

**Минобрнауки России**

**Юго-Западный государственный университет**

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА  
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Программно-информационная поисковая система с графическим  
интерфейсом на основе библиотеки регулярных выражений

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

А. А. Стародубцев

(инициалы, фамилия)

Группа ПО-216

Руководитель ВКР

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

# **Минобрнауки России**

## **Юго-Западный государственный университет**

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

---

(подпись, инициалы, фамилия)

«\_\_\_\_\_» \_\_\_\_\_ 20\_\_\_\_ г.

### **ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА**

Студента Стародубцев А.А., шифр 22-06-0164, группа ПО-21б

1. Тема «Программно-информационная поисковая система с графическим» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1584-с.

2. Срок предоставления работы к защите «9» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) провести анализ предметной области регулярных выражений и методов их реализации;

2) разработать концептуальную модель библиотеки регулярных выражений;

3) спроектировать программную систему библиотеки регулярных выражений на языке Си;

4) спроектировать C API библиотеки регулярных выражений для её интеграции в программы на Си-подобных языках;

5) реализовать библиотеку регулярных выражений средствами языка Си;

6) спроектировать программную систему поискового приложения с графическим интерфейсом;

7) реализовать поисковое приложение на языке C++ с использованием фреймворка Qt;

8) провести тестирование разработанных программных систем.

### 3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для библиотеки регулярных выражений являются: строка шаблона регулярного выражения, входной текст для поиска или валидации, строка замены (для операций замены).

2) Выходными данными для библиотеки регулярных выражений являются: захваченные подстроки, начальные позиции и длины совпадений, результат валидации строки, строка с выполненными заменами, коды и описания ошибок в шаблоне.

3) Входными данными для поискового приложения являются: текстовые файлы, открываемые для просмотра и редактирования, шаблоны регулярных выражений, вводимые пользователем, строки замены, путь к директории для рекурсивного поиска.

4) Выходными данными для поискового приложения являются: подсвеченные совпадения в тексте редактора, список найденных совпадений с номерами строк и столбцов, результат замены текста, список файлов с совпадениями при поиске по директории.

## 4. Содержание работы (по разделам):

### 4.1. Введение.

#### 4.1. Анализ предметной области.

4.2. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.3. Технический проект: общие сведения о программной системе, проект данных программной системы, проектирование архитектуры программной системы, проектирование программного интерфейса системы, проектирование пользовательского интерфейса программной системы.

4.4. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы, сборка компонентов программной системы.

4.5. Заключение.

4.6. Список использованных источников.

5. Перечень графического материала.

Руководитель ВКР

---

(подпись, дата)

А. А. Чаплыгин

---

(инициалы, фамилия)

Задание принял к исполнению

---

(подпись, дата)

А. А. Стародубцев

---

(инициалы, фамилия)

## РЕФЕРАТ

Объем работы равен 120 страницам. Работа содержит 16 иллюстраций, 20 таблиц, 26 библиографических источников. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: регулярные выражения, метасимволы, шаблон, библиотека, компиляция шаблона, конечный автомат, рекурсивный спуск, поиск с возвратами, группы захвата, обратные ссылки, символьные классы, квантификатор, абстрактное синтаксическое дерево, C API, Qt, текстовый редактор, графический интерфейс, поиск по директории, UTF-8, язык Си, C++, кроссплатформенность.

Объектом разработки являются библиотека регулярных выражений на языке Си с переносимым C API и десктопное поисковое приложение с графическим интерфейсом на основе фреймворка Qt, интегрирующее разработанную библиотеку.

Целью выпускной квалификационной работы является проектирование и реализация программно-информационной поисковой системы с графическим интерфейсом на основе библиотеки регулярных выражений собственной разработки.

Библиотека регулярных выражений реализована на языке Си методом рекурсивного нисходящего парсера: шаблон компилируется в абстрактное синтаксическое дерево непосредственно в процессе однократного разбора без построения промежуточных представлений, а сопоставление с текстом выполняется алгоритмом поиска с возвратами над полученным деревом.

Разработанная библиотека регулярных выражений может быть использована как самостоятельный компонент в любом проекте на языке Си или C++, требующем поиска, валидации или обработки текстовых данных. Поисковое приложение может применяться разработчиками и системными администраторами для интерактивного поиска и редактирования текстовых файлов и журналов с использованием регулярных выражений.

## ABSTRACT

The volume of work is 120 pages. The work contains 16 illustrations, 20 tables, 26 bibliographic sources. The number of appendices is 2. Graphic material is presented in appendix A. Source code fragments are presented in appendix B.

Keywords: regular expressions, wildcards, pattern, library, pattern compilation, finite automaton, recursive descent, backtracking, capture groups, back-references, character classes, quantifier, abstract syntax tree, C API, Qt, text editor, graphical user interface, directory search, UTF-8, C language, C++, cross-platform.

The object of development is a regular expression library written in C with a portable C API, and a desktop search application with a graphical user interface based on the Qt framework, integrating the developed library.

The purpose of the graduation thesis is to design and implement a software information retrieval system with a graphical user interface based on a custom-developed regular expression library.

The regular expression library is implemented in C using recursive descent parsing: the pattern is compiled into an abstract syntax tree directly during a single-pass parse without constructing intermediate representations, and text matching is performed by a backtracking search algorithm over the resulting tree.

The developed regular expression library can be used as a standalone component in any C or C++ project requiring text search, validation, or processing. The search application can be used by developers and system administrators for interactive search and editing of text files and logs using regular expressions.

## СОДЕРЖАНИЕ

|                                                              |    |
|--------------------------------------------------------------|----|
| ВВЕДЕНИЕ                                                     | 11 |
| 1 Анализ предметной области                                  | 14 |
| 1.1 Регулярные выражения                                     | 14 |
| 1.2 Методы реализации обработчика регулярных выражений       | 15 |
| 1.2.1 Интерпретация синтаксического дерева                   | 15 |
| 1.2.2 Симуляция конечного автомата                           | 16 |
| 1.2.3 Рекурсивный спуск с возвратами                         | 17 |
| 1.2.4 Сравнительный анализ подходов                          | 18 |
| 1.3 Язык Си и C API                                          | 20 |
| 1.4 Фреймворк Qt                                             | 21 |
| 2 Техническое задание                                        | 24 |
| 2.1 Основание для разработки                                 | 24 |
| 2.2 Назначение разработки                                    | 24 |
| 2.3 Библиотека регулярных выражений                          | 25 |
| 2.3.1 Функционал библиотеки регулярных выражений             | 25 |
| 2.3.2 Синтаксис шаблона регулярного выражения                | 25 |
| 2.3.3 API библиотеки регулярных выражений                    | 27 |
| 2.3.4 Экспортируемые функции библиотеки регулярных выражений | 27 |
| 2.4 Поисковое приложение                                     | 32 |
| 2.5 Требования к оформлению документации                     | 34 |
| 3 Технический проект                                         | 35 |
| 3.1 Общая характеристика организации решения задачи          | 35 |
| 3.2 Архитектура системы                                      | 35 |
| 3.3 Диаграмма компонентов                                    | 36 |
| 3.4 Библиотека регулярных выражений                          | 37 |
| 3.4.1 Внутренние структуры данных                            | 37 |
| 3.4.2 Внутреннее представление скомпилированного шаблона     | 39 |
| 3.4.3 Формальная грамматика шаблона регулярного выражения    | 41 |
| 3.4.4 Алгоритм компиляции шаблона                            | 42 |

|         |                                                   |     |
|---------|---------------------------------------------------|-----|
| 3.4.5   | Алгоритм сопоставления с образцом                 | 44  |
| 3.4.6   | Коды ошибок компиляции                            | 45  |
| 3.4.7   | Алгоритмы экспортируемых функций                  | 47  |
| 3.4.8   | Управление памятью и правила владения             | 49  |
| 3.5     | Поисковое приложение                              | 50  |
| 3.5.1   | Общая архитектура приложения                      | 50  |
| 3.5.2   | Модуль TextEditor                                 | 52  |
| 3.5.3   | Модуль SearchDialog                               | 54  |
| 3.5.4   | Модуль SearchDir                                  | 56  |
| 3.5.5   | Интеграция библиотеки регулярных выражений        | 58  |
| 4       | Рабочий проект                                    | 59  |
| 4.1     | Библиотека регулярных выражений                   | 59  |
| 4.1.1   | Состав библиотеки и условия применения            | 59  |
| 4.1.2   | Сборка библиотеки                                 | 59  |
| 4.1.3   | Экспортируемые функции библиотеки                 | 60  |
| 4.1.4   | Тестирование библиотеки регулярных выражений      | 68  |
| 4.1.4.1 | Модуль check_correctness                          | 69  |
| 4.1.4.2 | Модуль matchpos                                   | 71  |
| 4.1.4.3 | Модуль matchstart                                 | 74  |
| 4.1.4.4 | Модуль is_match                                   | 75  |
| 4.1.4.5 | Модуль matcharr                                   | 76  |
| 4.1.4.6 | Модуль matches                                    | 76  |
| 4.1.4.7 | Модуль replace                                    | 77  |
| 4.1.4.8 | Итоги тестирования                                | 78  |
| 4.2     | Поисковое приложение                              | 78  |
| 4.2.1   | Описание классов приложения                       | 78  |
| 4.2.2   | Тестирование поискового приложения                | 84  |
|         | ЗАКЛЮЧЕНИЕ                                        | 94  |
|         | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ                  | 95  |
|         | ПРИЛОЖЕНИЕ А Представление графического материала | 100 |
|         | ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы   | 104 |



|                                                                                            |        |
|--------------------------------------------------------------------------------------------|--------|
| На отдельных листах (CD-RW в прикрепленном конверте)                                       | 120    |
| Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)                               | Лист 1 |
| Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)             | Лист 2 |
| Синтаксис регулярных выражений (Графический материал / Синтаксис регулярных выражений.png) | Лист 3 |

## ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

КА – конечный автомат.

ДКА – детерминированный конечный автомат.

НКА – недетерминированный конечный автомат.

АСД – абстрактное синтаксическое дерево.

BNF (Backus Naur Form) – формальная грамматика в форме Бэкуса-Наура.

API (Application Programming Interface) – набор правил, типов, имен функций и контрактов, через которые одна программная сущность взаимодействует с другой на уровне исходного кода.

ABI (Application Binary Interface) – соглашение о взаимодействии на уровне двоичного кода (машинный код, объектные файлы, библиотеки) между скомпилированными компонентами.

UB (Undefined Behavior) – неопределенное поведение; ситуация, когда в определенных граничных случаях поведение программного продукта может меняться неконтролируемым образом и приводить к некорректным результатам.

ПО – программное обеспечение.

РП – рабочий проект.

ТЗ – техническое задание.

ТП – технический проект.

UML (Unified Modelling Language) – язык графического описания для объектного моделирования в области разработки программного обеспечения.

## ВВЕДЕНИЕ

Современные программные системы всё чаще сталкиваются с необходимостью анализа и обработки текстовых данных: от валидации пользовательского ввода и разбора логов до поиска по большим корпусам документов. Универсальным инструментом для решения подобных задач служат регулярные выражения – формальный язык описания шаблонов, позволяющий компактно выразить сложные критерии поиска и замены текста. Регулярные выражения встроены в большинство современных языков программирования и широко применяются в текстовых редакторах, системах сборки, анализаторах логов и средствах обработки данных.

Несмотря на широкую распространённость готовых решений, разработка собственной библиотеки регулярных выражений представляет значительный теоретический и практический интерес: она требует глубокого понимания теории формальных языков, алгоритмов разбора и сопоставления строк, а также проектирования переносимых программных интерфейсов. Создание такой библиотеки на языке Си с экспортируемым C API обеспечивает её применимость в широком спектре программных систем без зависимости от среды выполнения или стороннего окружения.

Наряду с библиотекой практическую ценность представляет поисковое приложение с графическим интерфейсом, непосредственно демонстрирующее возможности разработанной библиотеки. Интерактивный текстовый редактор с поддержкой поиска, замены и рекурсивного обхода директорий является типичным примером программного обеспечения, в котором эффективный механизм регулярных выражений оказывает прямое влияние на удобство работы пользователя.

*Цель настоящей работы* – проектирование и реализация программно-информационной поисковой системы с графическим интерфейсом на основе библиотеки регулярных выражений собственной разработки. Для достижения поставленной цели необходимо решить *следующие задачи*:

- провести анализ предметной области и методов реализации движков регулярных выражений;
- разработать концептуальную модель библиотеки регулярных выражений;
- спроектировать программную систему библиотеки регулярных выражений на языке Си;
- спроектировать переносимый C API библиотеки для её интеграции в программы на Си-подобных языках;
- реализовать библиотеку регулярных выражений средствами языка Си;
- спроектировать программную систему поискового приложения с графическим интерфейсом;
- реализовать поисковое приложение на языке C++ с использованием фреймворка Qt.

*Структура и объем работы.* Отчёт состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст выпускной квалификационной работы равен 120 страницам.

*Во введении* сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

*В первом разделе* на стадии описания технической характеристики предметной области приводится анализ регулярных выражений, методов их реализации, языка Си и C API, а также фреймворка Qt.

*Во втором разделе* на стадии технического задания приводятся требования к разрабатываемой библиотеке регулярных выражений, её C API и поисковому приложению с графическим интерфейсом.

*В третьем разделе* на стадии технического проектирования представлены проектные решения для библиотеки регулярных выражений: внутреннее структуры данных, формальная грамматика поддерживаемого синтаксиса, алгоритмы компиляции и сопоставления, описание экспортируемого API, а также архитектура поискового приложения и его модулей.

*В четвёртом разделе* на стадии рабочего проекта приводится описание реализации разработанных программных систем, проводится тестирование библиотеки регулярных выражений и поискового приложения.

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал. В приложении Б представлены фрагменты исходного кода.

## 1 Анализ предметной области

### 1.1 Регулярные выражения

Регулярные выражения (*regex* – от англ. *Regular Expressions*) – это формальный язык, используемый для поиска и осуществления манипуляций с фрагментами текста, основанный на использовании метасимволов (англ. *wildcards*). Для поиска используется шаблон регулярного выражения – строка-образец, состоящая из символов и метасимволов и позволяющая анализировать входящий текст удобным образом. Для манипуляций с текстом дополнительно задаётся строка замены, которая также может содержать в себе специальные символы.

Регулярные выражения широко используются в различных областях программирования и обработки данных, такими как:

- текстовые редакторы и IDE (расширенный поиск и подстановка текста на основе шаблонов);
- валидация данных (проверка формата вводимых данных, таких как адреса электронной почты, номера телефонов, почтовые индексы, гиперссылки);
- парсинг данных (извлечение информации из неструктурированных текстов, таких как HTML, XML или лог-файлы);
- обработка данных в базах данных;
- фильтрация спама в системах почты.

Истоки регулярных выражений лежат в теории автоматов и формальных языков. Эти области изучают вычислительные модели (автоматы) и способы описания и классификации формальных языков. В 1940-х гг. Уоррен Маккалак и Уолтер Питтс описали нервную систему, используя простой автомат в качестве модели нейрона. Математик Стивен Клини позже описал эти модели, используя свою систему математических обозначений, названную «регулярные множества». Кен Томпсон построил их в редактор QED, а затем в редактор `ed` под UNIX. С этого времени регулярные выражения ста-

ли широко использоваться в Unix и Unix-подобных утилитах, например: `expr`, `awk`, `Emacs`, `vi`, `lex` и `Perl`.

Регулярные выражения в `Perl` и `Tcl` происходят от реализации, написанной Генри Спенсером. Филип Хазэль разработал библиотеку `pcre` (англ. *Perl Compatible Regular Expressions* – `Perl`-совместимые регулярные выражения), которая используется во многих современных инструментах, таких как `PHP` и `Apache`.

Существует несколько разновидностей регулярных выражений:

1. Регулярные выражения `POSIX` – стандарт, используемый в Unix и Unix-подобных системах. Делится на два типа: «базовые» (`BRE`) и «расширенные» (`ERE`).

2. Регулярные выражения, совместимые с `Perl` (`PCRE`) – расширенная версия регулярных выражений, используемая в `Perl` и поддерживаемая многими языками, такими как `PHP`, `R` и `JavaScript`.

3. Регулярные выражения `JavaScript` – подобны `PCRE`, поддерживают флаги, такие как `g` (глобальный поиск) и `i` (игнорирование регистра).

4. Шаблоны `Lua` – упрощённая версия регулярных выражений, используемая в языке `Lua`. Имеет менее мощный синтаксис, чем в других языках, не поддерживает некоторые сложные конструкции.

## **1.2 Методы реализации обработчика регулярных выражений**

В теории и практике компиляторов и обработки формальных языков сложились три основных подхода к реализации механизма сопоставления регулярных выражений с текстом: интерпретация синтаксического дерева (дерева разбора), симуляция конечного автомата и рекурсивный спуск с возвратом.

### **1.2.1 Интерпретация синтаксического дерева**

При данном подходе шаблон регулярного выражения сначала разбирается синтаксическим анализатором (парсером) и представляется в виде абстрактного синтаксического дерева (АСД, от англ. *Abstract Syntax Tree*, `AST`)

– иерархической структуры данных, в которой каждый внутренний узел соответствует операции (конкатенация, альтернатива, квантификатор), а листовые узлы – атомарным элементам шаблона (конкретным символам или символьным классам). Сопоставление с текстом выполняется путём рекурсивного обхода этого дерева: для каждого узла определяется, может ли соответствующий фрагмент текста быть сопоставлен с данной ветвью дерева.

Достоинствами данного подхода являются концептуальная простота реализации и близость структуры данных к грамматике языка регулярных выражений, что упрощает поддержку расширенных возможностей: именованных групп, условных конструкций, атомарных групп. Помимо этого, данный подход удобен для дальнейшей трансляции – дерево разбора служит естественным промежуточным представлением для последующей генерации кода автомата или байт-кода виртуальной машины.

Среди недостатков следует отметить, что наивная интерпретация дерева с возвратами (без мемоизации) обладает экспоненциальной сложностью в худшем случае и может быть подвержена атакам типа ReDoS (*Regular Expression Denial of Service*). Эффективность работы напрямую зависит от применяемых оптимизаций дерева и стратегии обхода.

### **1.2.2 Симуляция конечного автомата**

Данный подход основан на классическом результате теории формальных языков: каждому регулярному выражению соответствует конечный автомат, допускающий тот же язык. Реализация включает два этапа. На первом этапе шаблон регулярного выражения транслируется в недетерминированный конечный автомат (НКА) с использованием алгоритма Томпсона, который строит НКА в виде ориентированного графа переходов за время и пространство, линейное по размеру шаблона. На втором этапе выполняется симуляция НКА на входной строке: вместо перебора с возвратами одновременно отслеживается множество активных состояний, в которых может находиться автомат после обработки каждого символа.



Альтернативой является предварительная детерминизация: НКА преобразуется в детерминированный конечный автомат (ДКА) посредством алгоритма построения подмножеств, после чего сопоставление с текстом выполняется за строго линейное время относительно длины входной строки. Детерминизация может производиться заранее (статически) или лениво – в процессе сопоставления (алгоритм Хопкрофта–Карпа, используемый, в частности, в библиотеке RE2 и пакете `regex` языка Go).

Главным достоинством автоматного подхода является гарантированная линейная сложность сопоставления относительно длины входной строки для всего класса регулярных языков, что исключает возможность ReDoS. Это делает подход наиболее предпочтительным для применений, критичных к безопасности.

К ограничениям следует отнести сложность поддержки ряда расширений, выходящих за рамки регулярных языков: обратных ссылок (*backreferences*), предпросмотров (*lookahead/lookbehind*) и условных конструкций. Кроме того, полная детерминизация может приводить к экспоненциальному росту числа состояний ДКА (экспоненциальный взрыв), что делает хранение полного ДКА нецелесообразным для сложных шаблонов; именно поэтому в практических реализациях применяется ленивая детерминизация с ограничением кэша.

### 1.2.3 Рекурсивный спуск с возвратами

Рекурсивный спуск с возвратами (*backtracking*) – наиболее распространённый подход в практических реализациях (Perl, PCRE, Python `re`, большинство языков программирования). В этом подходе сопоставление выполняется прямым рекурсивным обходом шаблона параллельно с обходом входной строки: при неудачном сопоставлении в некоторой точке выполняется возврат к последней точке выбора и испытывается альтернативная ветвь.

Несмотря на экспоненциальную сложность в худшем случае, данный подход на практике эффективен для большинства реальных шаблонов. Его ключевым преимуществом является возможность естественной реализа-

ции расширений, нарушающих регулярность: обратных ссылок, атомарных групп, посессивных (сверхжадных) квантификаторов, предпросмотров и ретроспективных проверок, именованных групп захвата. Перечисленные конструкции недоступны или труднодостижимы в чистом автоматном подходе.

К недостаткам относятся: уязвимость к ReDoS при специально подобранных шаблонах и входных строках, а также зависимость производительности от порядка альтернатив в шаблоне. Ряд реализаций (PCRE2, Oniguruma) частично устраняют эти проблемы с помощью оптимизаций: мемоизации промежуточных результатов, посессивных квантификаторов и атомарных групп.

#### 1.2.4 Сравнительный анализ подходов

Перечисленные подходы не являются взаимоисключающими: многие промышленные библиотеки сочетают их. Так, можно построить АСД на этапе разбора шаблона, затем транслировать его в НКА для основного сопоставления, а для конструкций вне класса регулярных языков переключаться на механизм с возвратами. Выбор подхода определяется требованиями к поддерживаемому синтаксису, гарантиям производительности и сложности реализации.

Разница в производительности между подходами на патологических шаблонах может быть колоссальной. Классический пример – шаблон  $(a^+)+b$ , применённый к строке из  $n$  символов  $a$  без завершающего  $b$ : алгоритм с возвратами совершает порядка  $2^n$  проверок (например, при  $n = 30$  это свыше миллиарда операций и время выполнения в несколько минут), тогда как симуляция НКА обрабатывает ту же строку за  $O(n \cdot m)$  шагов и завершается мгновенно. На реальных данных разница обычно менее выражена, однако на специально подобранных входных данных может достигать от тысячи до миллиона раз и более – именно это используется в атаках ReDoS.

В таблице 1.1 приведено сравнение рассмотренных подходов по ключевым критериям.

Таблица 1.1 – Сравнение методов реализации обработчика регулярных выражений

| Критерий                | АСД<br>(интерпретация<br>дерева)                    | Симуляция КА<br>(НКА/ДКА)                                   | Возврат<br>(backtracking) |
|-------------------------|-----------------------------------------------------|-------------------------------------------------------------|---------------------------|
| Сложность сопоставления | $O(2^n)$ в худшем случае                            | $O(n \cdot m)$ или $O(m)$                                   | $O(2^n)$ в худшем случае  |
| Потребляемая память     | $O(n)$ для стека рекурсии                           | $O(2^n)$ для ДКА в худшем случае;<br>$O(n \cdot m)$ для НКА | $O(n)$ для стека вызовов  |
| Обратные ссылки         | Поддерживаются                                      | Возможны на НКА, не поддерживаются на ДКА                   | Поддерживаются            |
| Предпросмотры           | Поддерживаются                                      | Затруднены                                                  | Поддерживаются            |
| Устойчивость к ReDoS    | Низкая                                              | Высокая<br>(линейная)                                       | Низкая                    |
| Сложность реализации    | Средняя                                             | Высокая                                                     | Ниже среднего             |
| Типичные реализации     | Промежуточное представление в PCRE, RE2, Rust regex | RE2, Go regexr                                              | PCRE, Python, Perl        |

### 1.3 Язык Си и C API

C – процедурный язык программирования общего назначения, в котором программы описываются наборами функций, оперирующих над данными через явные типы, указатели и структуры. Язык спроектирован так, чтобы давать программисту прямой и предсказуемый контроль над памятью и аппаратурой при минимальной абстракции над машиной.

C был создан в начале 1970-х Деннисом Ритчи в Bell Labs как развитие языка B и инструмент для разработки операционной системы UNIX. В ранних версиях язык уже поддерживал понятия типов, структур и побайтной работы с памятью, что сделало его удобным для системного программирования. Книга Кернигана и Ритчи «The C Programming Language» (1978) закрепила синтаксис и стиль, став практическим эталоном языка. В 1989 году появился стандарт ANSI C (C89/C90), затем шли ревизии C99, C11, C17/C18 и значительное обновление C23 – все они добавляли новые возможности, уточняли поведение и расширяли стандартную библиотеку, сохраняя при этом совместимость в широкой степени.

Дизайн C опирается на несколько ключевых идей: минимализм синтаксиса, предсказуемость выполнения и высокая эффективность. Язык предоставляет минимальный набор строительных блоков – переменные, выражения, управляющие конструкции, функции, указатели и структуры – и поручает программисту управление ресурсами (выделение и освобождение памяти, контроль границ буферов). Такая философия делает C особенно пригодным там, где важны скорость, компактность и контроль над аппаратной частью.

C широко применяют в областях, где требуется низкоуровневое управление и высокая производительность. Это реализация ядра операционных систем и системных утилит, написание драйверов устройств и сетевых стеков, разработка встроенного ПО для микроконтроллеров и реального времени, создание высокопроизводительных серверных компонентов и баз данных, реализация частей компиляторов и рантаймов для других языков (например, ядро CPython), а также оптимизированные библиотеки для численных вычис-

лений, графики и игровых движков. Благодаря стабильности ABI и простоте компоновки, многие библиотеки предоставляют C-интерфейс даже если их внутренняя реализация написана на других языках.

C API – это распространенный способ сделать библиотеку доступной внешним программам: набор заголовочных файлов с декларациями, соответствующая бинарная библиотека и ясные правила владения ресурсами. В C API обычно экспортируют функции с единым префиксом (чтобы избежать конфликтов имен), используют инкапсуляцию с помощью непрозрачных указателей (opaque pointers) для скрытия внутренней реализации, возвращают коды ошибок и документируют, кто отвечает за выделение и освобождение памяти. Другие важные аспекты – четкое версионирование ABI, требования по потокобезопасности и совместимость с C++ через extern "C".

Типовая схема небольшого C API выглядит как набор функций для создания и уничтожения контекста, операций над этим контекстом и вспомогательных функций для управления выделенной библиотекой памятью. Конtrakты функций должны быть явно задокументированы: допустимые входные значения, поведение при ошибках, правила владения результирующими ресурсами.

При проектировании C API важно установить однозначные правила владения памятью, избегать утечек и неопределенного поведения, инкапсулировать реализацию за упаковыми структурами и сохранять ABI-совместимость при развитии библиотеки. Для практической безопасности следует предусмотреть проверки размеров буферов, возвращать информативные коды ошибок и документировать, какие функции потокобезопасны.

## **1.4 Фреймворк Qt**

Qt – кроссплатформенный фреймворк для разработки приложений с графическим интерфейсом и сопутствующих подсистем (сеть, файловые операции, сериализация, многопоточность и т.д.), в основе которого лежит набор библиотек на C++. Qt предоставляет высокоуровневые абстракции для созда-

ния нативно выглядящих приложений на настольных системах, мобильных платформах и встраиваемых устройствах.

Это одна из наиболее зрелых и широко используемых кроссплатформенных библиотек для GUI-разработки: проект начат в начале 1990-х Хаавардом Нордом и Эйриком Чамбе-Энгом, впоследствии основавших компанию Trolltech, с тех пор прошел путь от коммерческой реализации к модели с открытым исходным кодом и коммерческой поддержкой. Qt стандартизировал подход к декларативному описанию интерфейсов (Qt Quick, QML) и расширил традиционные C++-возможности собственными механизмами метаобъектов (signals/slots, свойства, динамическая информация о типах).

Qt строится вокруг идеи модульности: рамки фреймворка разбиты на независимые модули – Qt Core, Qt GUI, Qt Widgets, Qt Quick, Qt Network, Qt SQL и другие – каждый из которых решает свой набор задач и предоставляет свой набор типов и сервисов.

В центре архитектуры лежит объектная модель на C++ с расширенной метаобъектной системой. Она реализует механизм сигналов и слотов для гибкой коммуникации между объектами и поддерживает свойства, которые легко интегрируются с QML и системой метаданных.

Qt предоставляет единый, кроссплатформенный API, благодаря чему приложения ведут себя и выглядят нативно на различных операционных системах – Windows, macOS, Linux, iOS и Android – и при этом могут использовать системные сервисы каждой платформы через единый интерфейс.

Для описания современных, декларативных интерфейсов Qt предлагает QML и движок Qt Quick: QML – это язык разметки с поддержкой анимаций и встроенного JavaScript, который позволяет быстро проектировать UI и при необходимости связывать его с производительным C++-бэкендом.

Qt также сопровождается развитой экосистемой инструментов: интегрированной средой разработки Qt Creator и визуальным редактором Qt Designer, официальными средствами сборки (qmake и поддержкой CMake), обширной документацией и множеством примеров, что облегчает разработку и распространение приложений на базе фреймворка.

Qt активно применяется при разработке графического интерфейса программ. Области применения фреймворка включают в себя:

- десктопные приложения с богатым GUI (офисные программы, IDE, визуальные редакторы);
- встраиваемые и промышленные интерфейсы управления (панели приборов, терминалы);
- мобильные приложения и гибридные решения (Qt поддерживает сборку под iOS/Android);
- приложения с требованиями к визуализации и мультимедиа (CAD, графические редакторы, специализированные визуализаторы);
- инструменты разработки и среды исполнения (редакторы, отладчики, средства моделирования);
- сетевые и серверные компоненты (используя Qt Network и асинхронные механизмы).

## **2 Техническое задание**

### **2.1 Основание для разработки**

Основанием для разработки является задание по выпускной квалификационной работе «Поисковое приложение с графическим интерфейсом на Qt на базе библиотеки регулярных выражений».

### **2.2 Назначение разработки**

Основными задачами выпускной квалификационной работы являются:

1. Проектирование и разработка библиотеки регулярных выражений на языке Си;
2. Проектирование и разработка графического приложения на Qt для поиска в документах с интеграцией разработанной библиотеки регулярных выражений.

Задачами данной разработки являются:

- провести анализ предметной области;
- разработать концептуальную модель библиотеки регулярных выражений;
- спроектировать программную систему библиотеки регулярных выражений;
- спроектировать API библиотеки регулярных выражений для её лёгкой интеграции в программы на Си-подобных языках;
- реализовать библиотеку регулярных выражений средствами языка Си;
- спроектировать программную систему поискового приложения с графическим интерфейсом;
- реализовать поисковое приложение на языке C++ с использованием фреймворка Qt для графического интерфейса.



## **2.3 Библиотека регулярных выражений**

### **2.3.1 Функционал библиотеки регулярных выражений**

Библиотека регулярных выражений на основе строки шаблона регулярного выражения позволяет:

- откомпилировать шаблон в объект регулярного выражения для дальнейшего многократного использования в обработке текста;
- получить вердикт, подходит ли строка под шаблон;
- получить массив захваченных подстрок из заданной строки;
- заменить захваченные подстроки в заданной строке на указанный текст, а также получить число выполненных замен;
- в случае некорректности шаблона регулярного выражения получить описание допущенной ошибки.

### **2.3.2 Синтаксис шаблона регулярного выражения**

Шаблон регулярного выражения должен позволять обрабатывать:

1. ASCII-символы и последовательности байтов UTF-8 в буквальном виде.
2. Метасимволы:
  - 2.1. `.` – любой одиночный символ;
  - 2.2. `^` – результат должен включать начало строки;
  - 2.3. `$` – результат должен включать конец строки;
  - 2.4. `*` – предыдущий токен (символ или группа) встречается 0 или более раз;
  - 2.5. `+` – предыдущий токен встречается 1 или более раз;
  - 2.6. `?` – предыдущий токен опционален;
  - 2.7. `*, +?, ??` – ленивый режим захвата для данного токена;
  - 2.8. `{n}` – предыдущий токен повторяется ровно  $n$  раз;
  - 2.9. `{n,m}` – предыдущий токен повторяется от  $n$  до  $m$  раз;
  - 2.10. `[abc]` – символ равен одному из символов в квадратных скобках;

2.11.  $[\wedge abc]$  – символ равен любому символу, которого нет в квадратных скобках;

2.12.  $abc|def$  – соответствует либо шаблону слева вертикальной черты, либо шаблону справа от нее (альтерация);

2.13.  $(abc)$  – определяет группу захвата, а также ограничивает область шаблона, рассматриваемого альтерацией;

2.14.  $(?:abc)$  – определяет не-захватываемую группу и ограничивает область шаблона, рассматриваемого альтерацией;

2.15.  $\backslash n$  – обратная ссылка на захваченную ранее при поиске группу, где  $x$  – номер группы от 1 до 9.

### 3. Символьные классы:

3.1.  $\backslash d$  – любая цифра;

3.2.  $\backslash D$  – любой нецифровой символ;

3.3.  $\backslash s$  – пробельный символ;

3.4.  $\backslash S$  – любой непробельный символ;

3.5.  $\backslash w$  – буква латинского алфавита, цифра или символ подчёркивания;

3.6.  $\backslash W$  – любой символ, кроме буквенно-цифрового и знака подчёркивания;

3.7.  $\backslash a$  – буква латинского алфавита;

3.8.  $\backslash A$  – любой символ, кроме латинских букв;

3.9.  $\backslash p$  – символы пунктуации;

3.10.  $\backslash P$  – любой непунктуационный символ;

3.11.  $\backslash n$  – символ новой строки;

3.12.  $\backslash t$  – символ табуляции.

4. Экранирование метасимволов: для буквальной интерпретации метасимволов, перечисленных в пункте 2, перед ними необходимо поставить экранирующий символ:  $\backslash$ . Сам экранирующий символ также нужно экранировать для буквальной интерпретации.

5. Подстановка захваченной группы при замене с помощью регулярного выражения ( $\$n$  или  $\backslash n$ , где  $n$  – номер группы, захваченный регулярным выражением, или всей захваченной подстроки при  $n = 0$ ).

6. Модификаторы регулярного выражения:

6.1. *m* – многострочный режим (^ и \$ означают начало и конец каждой строки);

6.2. *s* – метасимвол '.' включает в себя переход на новую строку;

6.3. *i* – поиск без учёта регистра (только латинские буквы);

6.4. *I* – поиск без учёта регистра (только базовая латиница и кириллица, текст обязательно должен быть в кодировке UTF-8).

### **2.3.3 API библиотеки регулярных выражений**

Библиотека регулярных выражений должна иметь переносимый C API для её интеграции в другие программы на Си-подобных языках посредством заголовочного (.h) файла, содержащего API-функции с пояснениями про принимаемые и возвращаемые ими параметры.

API библиотеки должно реализовывать следующие возможности:

- компиляция шаблона регулярного выражения для дальнейшего повторного использования при обработке строк;
- валидация строки с помощью шаблона регулярного выражения;
- получение захваченной подстроки из заданной строки шаблона или из откомпилированного регулярного выражения;
- получение массива со всеми захваченными подстроками из заданной строки шаблона или из откомпилированного регулярного выражения;
- получение строки – результата замены захваченных подстрок (всех или до указанного ограничения на количество) в заданной строке на указанный текст, а также числа выполненных замен.

### **2.3.4 Экспортируемые функции библиотеки регулярных выражений**

#### **2.3.4.1 Компиляция шаблона регулярного выражения**

Функции данного семейства выполняют разбор строки шаблона и создают внутреннее представление регулярного выражения в виде объекта

struct re\_chain, пригодного для многократного использования без повторного разбора шаблона.

1. struct re\_chain\* compile\_re(char\* pattern) — компиляция шаблона в объект регулярного выражения. Возвращает указатель на объект регулярного выражения; в случае некорректности шаблона возвращает NULL. Чтобы понять, в чём ошибка, нужно вызвать полную версию функции (compile\_re2) со всеми параметрами.

2. struct re\_chain\* compile\_re2(char\* pattern, int\* errcode, int\* errpos) — то же, что и для функции выше, но принимает ещё 2 параметра-указателя, заполняемые в случае некорректности шаблона: errcode — код ошибки и errpos — номер символа в строке шаблона, при разборе которого произошла ошибка (может указывать на нуль-терминатор строки, если ожидался закрывающий символ). Допустимо передать NULL-указатели, тогда эти поля не будут заполнены.

3. void free\_re(struct re\_chain\* re\_obj) — освобождение памяти, занимаемой объектом регулярного выражения, созданным функциями compile\_re и compile\_re2

Листинг 1 — Пример компиляции шаблона с диагностикой ошибок

```
1 char* pattern = "(\\d{2})\\. (\\d{2})\\. (\\d{4})";
2
3 int errcode, errpos;
4 struct re_chain *re = compile_re2(pattern, &errcode, &errpos);
5 if (!re) {
6     // errpos - позиция символа шаблона, вызвавшего ошибку
7     fprintf(stderr, "Ошибка с кодом %d, позиция %d\\n", errcode, errpos);
8     return 1;
9 }
10
11 free_re(re); // освобождение скомпилированного объекта
```

#### 2.3.4.2 Поиск позиции совпадения

Данные функции возвращают позицию и длину первого совпадения регулярного выражения в тексте, не выделяя памяти под содержимое захваченной подстроки.

1. int matchpos(char\* pattern, char\* text, int\* capture\_data) — возвращает данные о захваченной подстроке из исходного текста по строке шаблона регулярного выражения. При успешном захвате возвращает номер первого символа

подстроки и записывает её длину в переменную по указателю `capture_data`, если он не равен `NULL`. При неудачном захвате возвращает `-1`. При некорректности шаблона возвращает отрицательное число меньше `-1` — код ошибки, а также записывает номер символа в строке шаблона, при разборе которого произошла ошибка, в `capture_data`. Если по указателю `capture_data` передать положительное число, то поиск в строке `text` начнётся с этой позиции.

2. `int obj_matchpos(struct re_chain* re_obj, char* text, int* capture_data)` — то же, что и для функции выше, но вместо строки с шаблоном принимает откомпилированный объект регулярного выражения.

#### Листинг 2 – Последовательный проход по всем совпадениям в тексте

```
1 char* text = "В 2023 году вышло 42 релиза, а в 2024 году - уже 58";
2 struct re_chain* re = compile_re("\\d+");
3
4 int pos = 0;
5 int len;
6 while (1) {
7     len = pos;
8     int start = obj_matchpos(re, text, &len); // поиск начиная с позиции len
9     if (start < 0)
10         break;
11
12     printf("позиция %d: \".*s\\\"\\n", start, len, text + start);
13     pos = start + len; // сдвиг за конец найденного совпадения
14 }
15 free_re(re); // освобождение объекта регулярного выражения
```

#### 2.3.4.3 Валидация строки

Функции данного семейства проверяют, соответствует ли строка целиком шаблону регулярного выражения, и возвращают булев результат без извлечения подстрок.

1. `int is_match(char* pattern, char* text)` — выполняет валидацию строки с помощью строки шаблона регулярного выражения: при успехе возвращает `1`, при неудаче — `0`, при некорректности шаблона возвращает отрицательное число меньше `-1` — код ошибки.

2. `int obj_is_match(struct re_chain* re_obj, char* text)` — то же, что и для функции выше, но вместо строки с шаблоном принимает откомпилированный объект регулярного выражения.

### Листинг 3 – Валидация e-mail адресов

```
1 struct re_chain* re_email = compile_re("^([\\w.+]+@[\\w-]+\\.([\\w.]+)$");
2 char* inputs[] = {
3     "user@example.com", "not-an-email",
4     "wrong@abc.", "abc@af-y.ru"
5 };
6
7 for (int i = 0; i < 4; i++) {
8     int email_ok = obj_is_match(re_email, inputs[i]); // 1 - совпадение, 0 -
9     нет
10    printf("%s' - %s\n", inputs[i], email_ok == 1 ? "корректен" : "не
11    корректен!");
12 }
13 free_re(re_email);
```

#### 2.3.4.4 Извлечение одного совпадения

Функции данного семейства возвращают первое совпадение регулярного выражения вместе с содержимым всех групп захвата в виде NULL-терминированного массива строк.

1. `char** match(char* pattern, char* text)` — возвращает NULL-терминированный массив malloc-строк аналогично функции `match()` в JavaScript: первый элемент — главный захват, далее — содержимое групп захвата. При отсутствии совпадений возвращает пустой массив, при некорректности шаблона — NULL.

2. `char** obj_match(struct re_chain* re_obj, char* text)` — то же, что и для функции выше, но вместо строки с шаблоном принимает откомпилированный объект регулярного выражения.

#### Листинг 4 – Извлечение даты в формате ДД.ММ.ГГГГ из набора строк

```
1 struct re_chain* re = compile_re("(\\d{2})\\. (\\d{2})\\. (\\d{4})");
2 char* texts[] = {
3     "Дата рождения: 01.04.1987",
4     "Дата выдачи: 15.08.2003",
5     "Дата истечения: 31.12.2030"
6 };
7
8 for (int i = 0; i < 3; i++) {
9     char** m = obj_match(re, texts[i]); // первый захват + группы
10    if (m && m[0]) {
11        printf("%s: день=%s месяц=%s год=%s\n", m[0], m[1], m[2], m[3]);
12        for (int j = 0; m[j]; j++) free(m[j]);
13    }
14    free(m);
15 }
16 free_re(re);
```

#### 2.3.4.5 Извлечение всех совпадений

Функции данного семейства возвращают все непересекающиеся совпадения регулярного выражения в тексте в виде NULL-терминированного массива строк, аналогично вызову `match()` с флагом `g` в JavaScript.

1. `char** matches(char* pattern, char* text)` — возвращает NULL-терминированный массив `malloc`-строк — все захваченные подстроки регулярного выражения. При отсутствии совпадений возвращает пустой массив, при некорректности шаблона — `NULL`.

2. `char** obj_matches(struct re_chain* re_obj, char* text)` — то же, что и для функции выше, но вместо строки с шаблоном принимает откомпилированный объект регулярного выражения.

Листинг 5 — Извлечение всех слов из текста

```
1 // сбор всех слов из лог-строки для анализа
2 char* log_line = "[ERROR] 2024-01-15 DatabaseConnection: timeout after 30s"
3 ;
4 struct re_chain* re = compile_re("\\a+");
5 char** words = obj_matches(re, log_line);
6 if (words) {
7     printf("Найденные слова:\n");
8     for (int i = 0; words[i]; i++) {
9         printf(" [%d] %s\n", i, words[i]);
10        free(words[i]);
11    }
12    free(words);
13 }
14 free_re(re);
```

#### 2.3.4.6 Замена совпадений

Функции данного семейства выполняют замену совпадений регулярного выражения в исходной строке на строку замены, которая может содержать ссылки на группы захвата (`$n` или `\n`).

1. `int re_nreplace(char* pattern, char* repl, char* text, char* dest, int n, int limit)` — выполняет замену подстрок, совпадающих с `pattern`, на строку `repl` в тексте `text`, максимум `limit` раз (0 — без ограничений). Результат записывается в буфер `dest` размером `n` байт. Возвращает число выполненных замен, -1 при

переполнении буфера или отрицательный код ошибки при некорректном шаблоне.

2. `int obj_re_nreplace(struct re_chain* re_obj, char* repl, char* text, char* dest, int n, int limit)` – то же, что и для функции выше, но вместо строки с шаблоном принимает откомпилированный объект регулярного выражения.

3. `char* re_dupreplace(char* pattern, char* repl, char* text, int limit, int* repl_cnt)` – выполняет замену подстрок, совпадающих с `pattern`, на строку `repl` в тексте `text`, максимум `limit` раз (0 – без ограничений). Результат записывается в возвращаемую `malloc`-строку. Записывает по указателю `repl_cnt` число выполненных замен или отрицательный код ошибки при некорректном шаблоне

4. `char* obj_re_dupreplace(struct re_chain* re_obj, char* repl, char* text, int limit, int* repl_cnt)` – то же, что и для функции выше, но вместо строки с шаблоном принимает откомпилированный объект регулярного выражения.

#### Листинг 6 – Замена подстрок с группами захвата

```
1 // Приведение дат ДД.ММ.ГГГГ к формату ISO (ГГГГ-ММ-ДД)
2 char* text = "Договор от 15.03.2023, срок действия до 31.12.2025";
3 char dest[256];
4
5 struct re_chain* re = compile_re("(\\d{2})\\.(\\d{2})\\.(\\d{4})");
6 // $1 $2 $3 - ссылки на группы захвата (день, месяц, год)
7 int count = obj_re_nreplace(re, "$3-$2-$1",
8     text, dest, sizeof(dest), 0); // 0 - без ограничений
9 if (count >= 0)
10     printf("результат (%d замен(ы)): %s\\n", count, dest);
11 else
12     fprintf(stderr, "переполнение буфера или ошибка шаблона\\n");
13
14 free_re(re);
15 // вывод: Договор от 2023-03-15, срок действия до 2025-12-31
```

## 2.4 Поисковое приложение

Десктопное приложение для поиска и замены текста в документах должно обладать следующими функциональными и техническими требованиями:

1. Работа с файлами и текстом:



- а) позволять пользователю открывать произвольный текстовый файл и выполнять в нём поиск посредством регулярных выражений, подсвечивать все найденные совпадения и опционально выводить их в список с возможностью навигации по документу с его помощью;
- б) позволять выполнять замену текста в файле, используя шаблон регулярного выражения как в интерактивном режиме, так и в пакетном;
- с) сохранять файл после замены с возможностью перезаписать оригинал или сохранить результат как новый файл;
- д) отображать номера строк в редакторе, а также текущую позицию курсора (строка, столбец) в строке состояния;
- е) определять кодировку файла при открытии и выполнять автоматическую конвертацию содержимого в кодировку UTF-8 для дальнейшей работы.

## 2. Поиск и навигация:

- а) иметь возможность выполнять поиск без учёта и с учётом регистра, выводить на экран ошибки в составлении шаблона при их наличии, а также выполнять простой поиск по тексту без использования регулярных выражений;
- б) отображать общее число найденных совпадений;
- с) поддерживать переход к следующему и предыдущему совпадению по горячим клавишам;
- д) отображать номер строки для каждого совпадения в списке результатов;

## 3. Поиск по папке:

- а) обеспечивать возможность рекурсивного поиска по регулярному выражению во всех текстовых файлах в выбранной папке, выводить результаты поиска и при нажатии на совпадение открывать содержащий его файл для редактирования в поисковом приложении;
- б) поддерживать фильтрацию файлов по расширению при рекурсивном поиске (например, только \*.txt, \*.log);
- с) отображать число совпадений на файл в дереве результатов поиска.

#### 4. Пользовательский интерфейс:

- а) хранить историю последних использованных шаблонов регулярных выражений и предоставлять к ней доступ через выпадающий список поля ввода;
- б) отображать справку по синтаксису поддерживаемых регулярных выражений непосредственно в приложении;
- с) поддерживать светлую и темную тему оформления средствами Qt.

#### 5. Технические требования:

- а) быть написано с использованием фреймворка Qt на языке C++ для обеспечения кроссплатформенности среди десктопных операционных систем;
- б) интегрировать API разработанной библиотеки регулярных выражений.

### **2.5 Требования к оформлению документации**

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

### **3 Технический проект**

#### **3.1 Общая характеристика организации решения задачи**

Разрабатываемая система состоит из двух взаимосвязанных компонентов: библиотеки регулярных выражений на языке Си и десктопного поискового приложения на C++ с использованием фреймворка Qt.

Библиотека регулярных выражений реализована в виде единственного файла исходного кода на языке Си и заголовочного файла с описанием экспортируемого C API. Такой способ поставки обеспечивает максимальную переносимость: библиотека может быть включена в любой проект на языке Си или C++ без предварительной сборки в отдельный бинарный модуль. Заголовочный файл содержит объявления всех экспортируемых функций, обернутые в блок `extern "C"`, что обеспечивает корректное связывание при компиляции в составе C++-проекта.

Поисковое приложение реализовано на C++ с использованием фреймворка Qt и включает в себя текстовый редактор с поддержкой поиска и замены, а также модуль рекурсивного поиска по файлам в директории. Приложение интегрирует библиотеку регулярных выражений напрямую через ее C API, подключая заголовочный файл и компилируя исходный файл библиотеки совместно с исходными файлами приложения.

#### **3.2 Архитектура системы**

Система разделена на два независимых компонента – библиотеку регулярных выражений и поисковое приложение, – каждый из которых включает в себя несколько модулей.

Библиотека регулярных выражений состоит из следующих модулей:

1. Модуль парсера – разбор строки шаблона регулярного выражения и построение внутреннего представления.
2. Модуль сопоставления – проверка входной строки на соответствие скомпилированному объекту регулярного выражения.
3. Модуль замены – выполнение замены совпадений в строке.

4. Модуль управления памятью – выделение и освобождение внутренних структур данных.

5. Модуль автотестов – набор автоматизированных тестов для верификации корректности работы библиотеки.

Поисковое приложение состоит из следующих модулей:

1. main – точка входа в приложение, инициализация Qt и запуск главного окна.

2. TextEditor – модуль текстового редактора: открытие, отображение и редактирование файлов, подсветка совпадений, поиск и замена в тексте.

3. SearchDialog – диалог поиска и замены в текстовом редакторе.

4. SearchDir – модуль рекурсивного поиска по файлам в директории с отображением результатов и возможностью открытия найденного файла в редакторе.

### **3.3 Диаграмма компонентов**

На рисунке 3.1 изображена диаграмма компонентов для проектируемой системы. Она включает модули:

- модуль парсера;
- модуль сопоставления;
- модуль поиска и валидации;
- модуль замены;
- модуль управления памятью;
- модуль автотестов;
- главный модуль приложения (main);
- модуль текстового редактора (TextEditor);
- модуль диалога поиска и замены (SearchDialog);
- модуль рекурсивного поиска по директории (SearchDir).

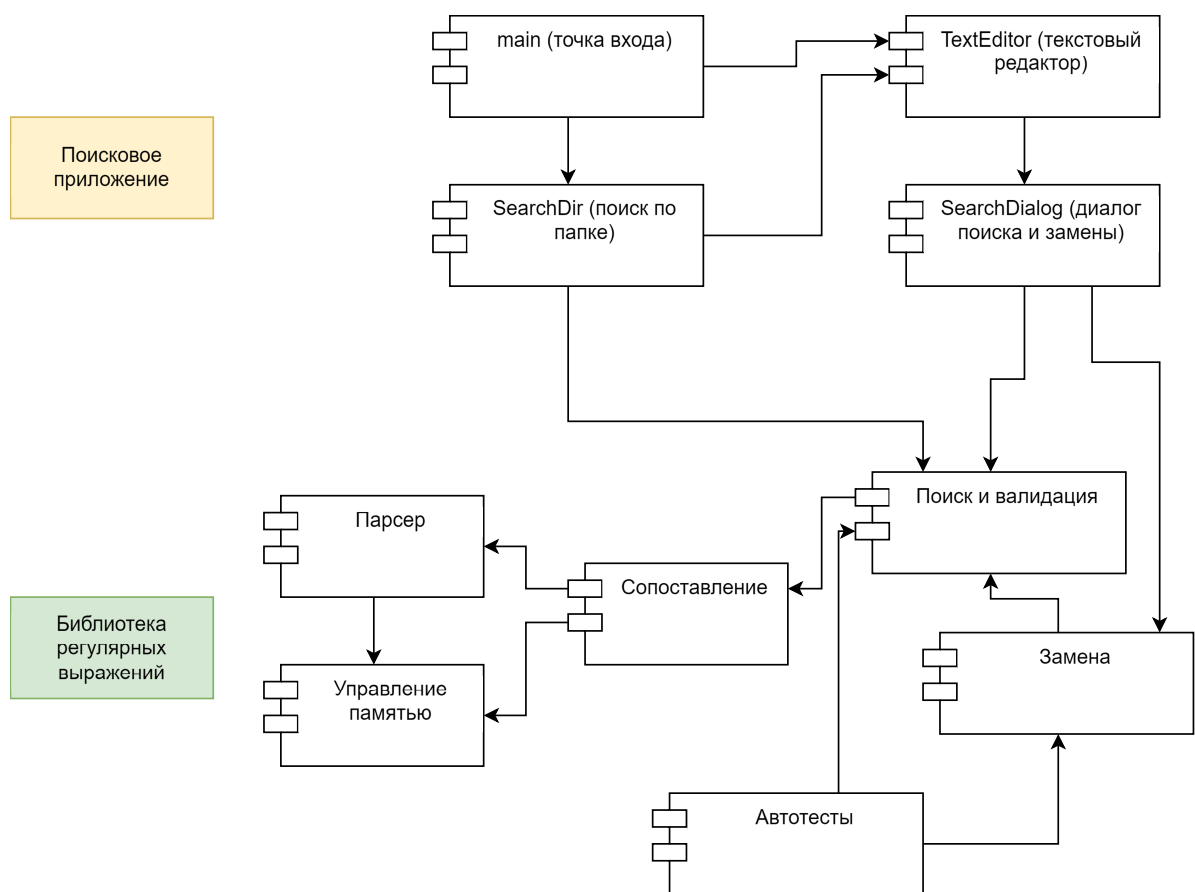


Рисунок 3.1 – Диаграмма компонентов проектируемой системы

## 3.4 Библиотека регулярных выражений

### 3.4.1 Внутренние структуры данных

Скомпилированное регулярное выражение представляется в памяти в виде связанного списка объектов типа `struct re_chain` – звеньев цепочки. Каждое звено соответствует одной альтерации регулярного выражения (разделяемых в шаблоне `|`). Внутри звена хранится односвязный список объектов типа `struct re_block` – блоков, каждый из которых соответствует одному токenu шаблона. Структура `struct re_chain` содержит следующие поля:

- `next` – указатель на следующее звено цепочки (следующую альтерацию) или `NULL`;
- `head`, `tail` – указатели на первый и последний блок в списке блоков данной альтернативы;
- `bl_count` – число блоков в списке;

- `incl_start`, `incl_end` – флаги наличия якорей `^` и `$` в данной альтернативе;
- `ch_data` – для корневой цепочки: суммарное число групп захвата в регулярном выражении; для вложенных цепочек: порядковый номер соответствующей группы захвата;
- `flags` – модификаторы регулярного выражения, хранятся только в корневом звене цепочки; ненулевое значение является признаком того, что данное звено является корневым.

Структура `struct re_block` содержит следующие поля:

- `type` – тип блока, определяющий интерпретацию поля `data`. Возможные типы блоков приведены в таблице 3.1;
- `data` – объединение (`union`) с данными блока в зависимости от его типа (см. таблицу 3.1);
- `rep_min`, `rep_max` – минимальное и максимальное число повторений блока согласно квантификатору;
- `lazy` – флаг ленивого режима захвата;
- `next` – указатель на следующий блок в цепочке.

Таблица 3.1 – Типы блоков регулярного выражения

| Тип   | Описание                                                                                     | Данные блока                |
|-------|----------------------------------------------------------------------------------------------|-----------------------------|
| PLAIN | Последовательность символов, воспринимаемых буквально; квантификаторы к блоку не применяются | Указатель на строку<br>text |
| ONE   | Одиночный символ, возможно с квантификатором                                                 | Символ sym                  |
| ANY   | Метасимвол «.» – любой одиночный символ                                                      | Отсутствуют                 |

Таблица 3.1 – Типы блоков регулярного выражения

| Тип       | Описание                                          | Данные блока                                       |
|-----------|---------------------------------------------------|----------------------------------------------------|
| CHARCLASS | Символьный класс (\d, \w, \s и др.)               | Символ sym, представляющий символьный класс        |
| GROUP     | Набор символов в квадратных скобках [...]         | Указатель на строку text с набором символов        |
| REVGROUP  | Инвертированный набор символов [^...]             | Указатель на строку text с набором символов        |
| CHAIN     | Группа в круглых скобках                          | Указатель на вложенную цепочку альтераций subchain |
| BACKREF   | Обратная ссылка на захваченную группу от \1 до \9 | Номер группы захвата cap_num                       |

Для хранения информации о захваченных подстроках в процессе сопоставления используется вспомогательная структура `struct re_matches`, содержащая массивы начальных позиций (`starts`) и длин (`lengths`) захватов, их емкость (`capacity`) и текущие флаги модификаторов (`flags`).

### 3.4.2 Внутреннее представление скомпилированного шаблона

Результатом компиляции шаблона регулярного выражения является абстрактное синтаксическое дерево, реализованное в виде двух взаимосвязанных односвязных списков. Верхний уровень образуют звенья типа `struct re_chain`, каждое из которых соответствует одной альтернативе шаблона (разделяемые оператором `|`); звенья связаны через поле `next`. Внутри каждого звена хранится список токенов конкатенации – объектов типа `struct re_block`,

также связанных через поле `next`. Блок типа `CHAIN` (скобочная группа) рекурсивно содержит указатель на вложенную цепочку альтераций.

Корневое звено цепочки отличается от вложенных ненулевым значением поля `flags`, которое хранит модификаторы регулярного выражения (`RE_ROOT`, `RE_M`, `RE_S`, `RE_I`, `RE_IRU`). В поле `ch_data` корневого звена хранится суммарное число групп захвата в шаблоне; в поле `ch_data` вложенных звеньев – порядковый номер соответствующей группы захвата.

На рисунке 3.2 приведена диаграмма внутреннего представления для шаблона  $(a+|^bc)d\{2\}$ . Корневая цепочка содержит два блока: блок типа `CHAIN` (группа захвата №1) с квантификатором  $\{1,1\}$  и блок типа `CHARCLASS` (символьный класс `\d`) с квантификатором  $\{2,2\}$ . Блок `CHAIN` ссылается на вложенную цепочку из двух альтернатив: первая содержит блок типа `ONE` (символ `a` с квантификатором  $\{1,\infty\}$ ), вторая – с флагом захвата с начала строки и блоком типа `PLAIN` (буквальная строка `"bc"`).

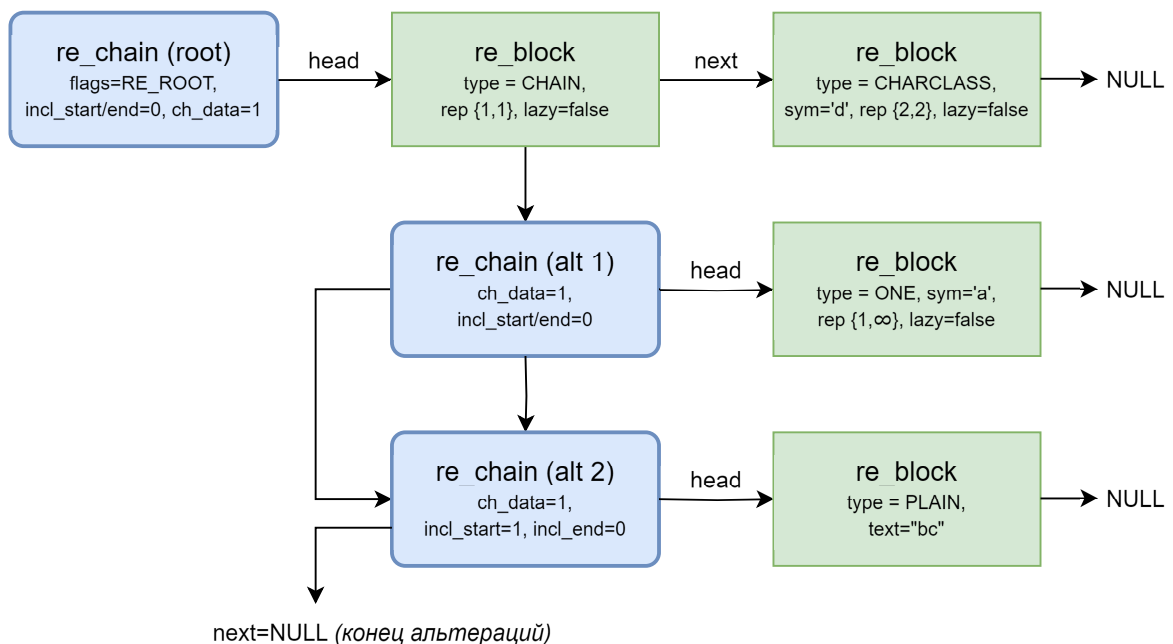


Рисунок 3.2 – Диаграмма абстрактного синтаксического дерева регулярного выражения



### 3.4.3 Формальная грамматика шаблона регулярного выражения

Синтаксис шаблонов регулярных выражений, поддерживаемых библиотекой, описывается формальной грамматикой в форме Бэкуса-Наура (BNF). Грамматика отражает приоритет операций: квантификаторы связывают сильнее конкатенации, конкатенация – сильнее альтерации.

Листинг 7 – Грамматика шаблона регулярного выражения в форме Бэкуса–Наура

```
1  <шаблон>          ::= <альтерация>
2  <альтерация>      ::= <конкатенация>
3  | <альтерация>    "|" <конкатенация>
4  <конкатенация>    ::= <токен>
5  | <конкатенация> <токен>
6  <токен>           ::= <атом>
7  | <атом> <квантификатор>
8  | <атом> <квантификатор> "?"
9  <квантификатор>   ::= "*" | "+" | "?"
10 | "{" <число> "}"
11 | "{" <число> "," <число> "}"
12 <атом>            ::= <символ>
13 | "\" <метасимвол>
14 | "\" <символьный-класс>
15 | "\" <цифра-от-1-до-9>
16 | "."
17 | "^"
18 | "$"
19 | <набор-символов>
20 | <группа>
21 <группа>          ::= "(" <шаблон> ")"
22 | "(?:" <шаблон> ")"
23 | "(?flags)"
24 <набор-символов>  ::= "[" <элементы-набора> "]"
25 | "[^" <элементы-набора> "]"
26 <элементы-набора> ::= <элемент-набора>
27 | <элемент-набора> <элементы-набора>
28 <элемент-набора>  ::= <символ-набора>
29 | "\" <символьный-класс>
30 | "\" <символ-набора>
31 | <символ-набора> "-" <символ-набора>
32 <символьный-класс> ::= "d" | "D" | "w" | "W" | "s" | "S" | "a" | "A" | "p" |
    "P" | "n" | "t"
33 <модификатор>     ::= "m" | "s" | "i" | "I"
34 <символ>          ::= любой символ ASCII или байтовая последовательность
35 UTF-8, кроме метасимволов
```

|    |                   |                                                          |
|----|-------------------|----------------------------------------------------------|
| 36 | <метасимвол>      | ::= "."   "^"   "\$"   "*"   "+"   "?"   "["   "]"   "{" |
|    |                   | "}"   " "   "("   ")"   "\"                              |
| 37 | <символ-набора>   | ::= любой символ, кроме "]" и "\"                        |
| 38 | <цифра-от-1-до-9> | ::= "1"   "2"   "3"   "4"   "5"   "6"   "7"   "8"   "9"  |
| 39 | <число>           | ::= <цифра>   <число> <цифра>                            |
| 40 | <цифра>           | ::= "0"   "1"   "2"   "3"   "4"   "5"   "6"   "7"   "8"  |
|    |                   | "9"                                                      |

Несколько пояснений к грамматике:

- форма (?flags) задает модификаторы регулярного выражения (m, s, i, l) и не производит захвата; обратная ссылка \n (где *n* – цифра от 1 до 9) ссылается на ранее захваченную группу и допустима только в позиции атома;
- в строке замены допускаются ссылки вида \$n или \n (где *n* от 0 до 9), которые не являются частью шаблона и в грамматике не описываются;
- диапазон {*n,m*} требует  $n \leq m$ ; оба числа не должны превышать INT\_MAX.

### 3.4.4 Алгоритм компиляции шаблона

Компиляция шаблона выполняется рекурсивным нисходящим парсером, реализованным в функции `_parse_chain`. Результатом компиляции является абстрактное синтаксическое дерево (АСД) – иерархическая структура, в которой альтерации представлены звеньями списка `re_chain`, а токены каждой альтернативы – блоками `re_block`, образующими вложенное дерево через блоки типа `CHAIN`. АСД формируется непосредственно в процессе однопроходного разбора строки шаблона без построения промежуточных представлений, что сокращает число выделений памяти и упрощает реализацию.

При встрече с открывающей круглой скобкой парсер рекурсивно вызывает себя для разбора вложенной группы, результат которого записывается в блок типа `CHAIN`. При встрече с символом альтерации «|» создается новое звено цепочки, и разбор продолжается в нем. Квантификаторы (\*, +, ?, {*n,m*}) обрабатываются вспомогательной функцией `process_quantif` непосредственно после распознавания предшествующего токена.

Проверка корректности шаблона выполняется в процессе разбора без отдельного прохода. При обнаружении синтаксической ошибки парсер немедленно прерывает работу через механизм нелокального перехода `setjmp/longjmp`, возвращая управление в функцию `compile_re2` с кодом ошибки и позицией символа в шаблоне, при разборе которого ошибка была обнаружена.

Если в шаблоне заданы флаги регистронезависимого поиска (`i` или `I`), после завершения разбора над всем внутренним представлением выполняется однократный проход `re_tolower_inplace`, приводящий все литеральные данные в блоках к нижнему регистру. В процессе сопоставления аналогичное приведение выполняется над входной строкой, что позволяет реализовать регистронезависимый поиск без изменения алгоритма сопоставления.

Точкой входа при компиляции является экспортируемая функция `compile_re2`, вызывающая `_parse_chain` с начальными параметрами. Упрощенная обертка `compile_re` вызывает `compile_re2` с `NULL`-указателями вместо параметров диагностики ошибок.

На рисунке 3.3 изображена диаграмма активностей алгоритма компиляции шаблона регулярного выражения.

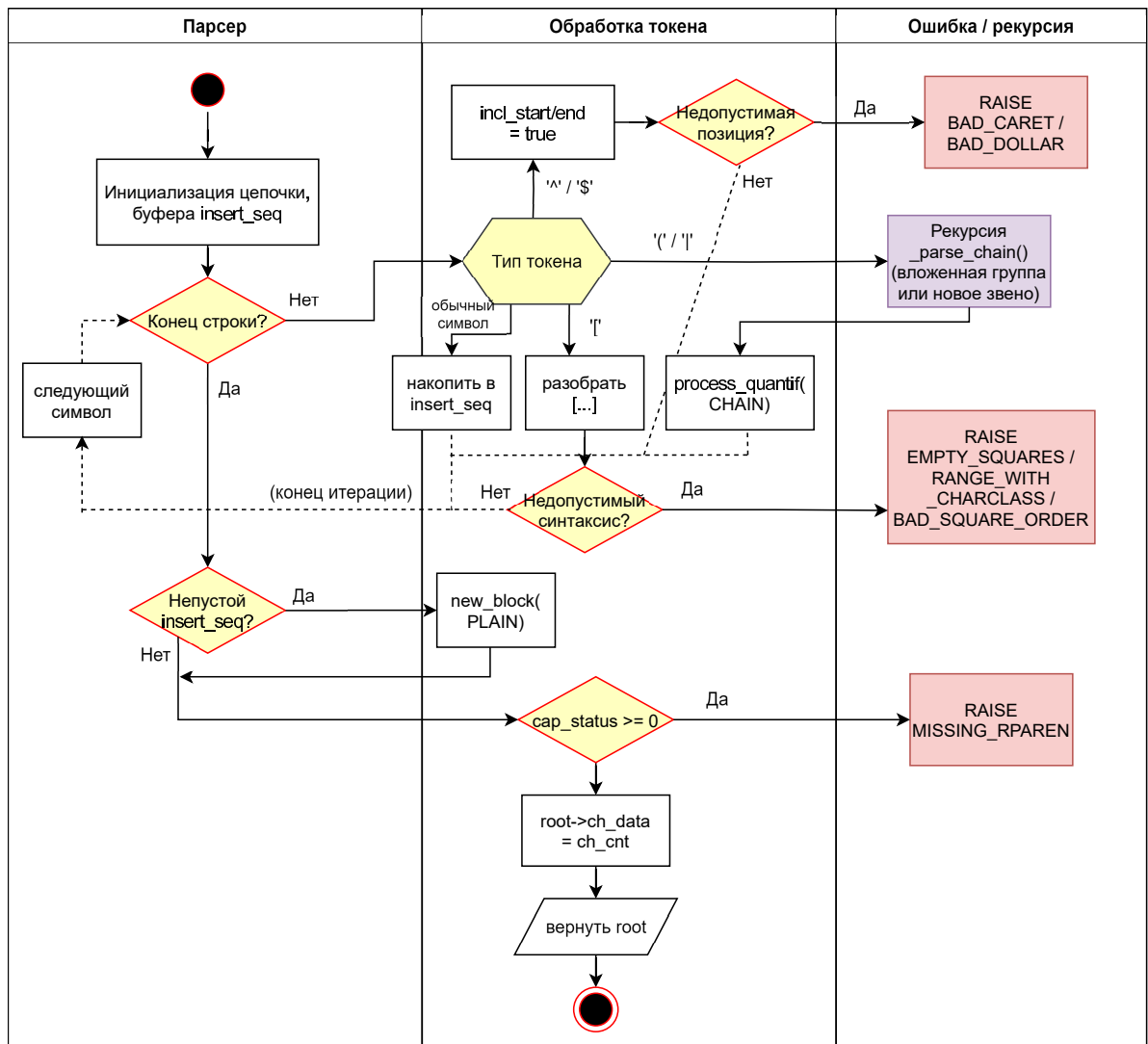


Рисунок 3.3 – Диаграмма активностей алгоритма компиляции

### 3.4.5 Алгоритм сопоставления с образцом

Для проверки соответствия входной строки скомпилированному регулярному выражению применяется алгоритм поиска с возвратами (метод перебора с откатом). Алгоритм реализован в функции `_match_chain` и работает следующим образом.

Для каждой стартовой позиции во входной строке алгоритм последовательно обходит блоки текущего звена цепочки. Для каждого блока он жадно (или лениво, если задан флаг `lazy`) пытается захватить максимально возможное число повторений, соответствующих квантификатору. Если следующий блок не сопоставляется, алгоритм возвращается к предыдущему блоку и уменьшает число захваченных им повторений на единицу, после чего пред-

принимает новую попытку. Таким образом осуществляется полный перебор допустимых комбинаций с откатами.

Для блоков типа `CHAIN` (вложенных групп) сопоставление выполняется рекурсивно через функцию `_match_group`, которая перебирает все альтернативы вложенной цепочки и выбирает ту из них, начало захвата которой расположено как можно раньше во входной строке, – в соответствии с поведением регулярных выражений в других языках.

Позиции и длины захваченных групп в процессе сопоставления накапливаются в структуре `struct re_matches` и используются как для возврата результата вызывающей стороне, так и для разрешения обратных ссылок (`BACKREF`) в ходе того же сопоставления.

Подготовка к сопоставлению (нормализация входной строки при регистронезависимом поиске, обработка начальной позиции поиска) выполняется оберткой `_match_wrapper`, которую вызывают все экспортируемые функции сопоставления.

### 3.4.6 Коды ошибок компиляции

При обнаружении синтаксической ошибки в шаблоне регулярного выражения функция `compile_re2` возвращает `NULL` и записывает по указателю `errcode` один из кодов ошибки, перечисленных в таблице 3.2. По указателю `errpos` записывается индекс символа в строке шаблона, при разборе которого ошибка была обнаружена. Человекочитаемое описание ошибки на английском языке можно получить вызовом отладочной функции `get_error_msg(errcode)`.

Таблица 3.2 – Коды ошибок компиляции регулярного выражения

| Код  | Константа              | Описание                                                                                     |
|------|------------------------|----------------------------------------------------------------------------------------------|
| –100 | <code>BAD_CARET</code> | Неэкранированный символ <code>^</code> не в начале регулярного выражения или набора символов |

Таблица 3.2 – Коды ошибок компиляции регулярного выражения

| Код | Константа            | Описание                                                      |
|-----|----------------------|---------------------------------------------------------------|
| –99 | BAD_DOLLAR           | Неэкранированный символ \$ не в конце регулярного выражения   |
| –98 | PREV_NOT_QUANTIF     | Предыдущий токен не квантифицируем                            |
| –97 | EMPTY_SQUARES        | Пустой набор символов в квадратных скобках                    |
| –96 | MISSING_LSQUARE      | Отсутствует открывающая квадратная скобка «[»                 |
| –95 | MISSING_RSQUARE      | Отсутствует закрывающая квадратная скобка «]»                 |
| –94 | RANGE_WITH_CHARCLASS | Диапазон с символьным классом недопустим (например, [\w -z])  |
| –93 | BAD_ESC_CHAR         | Отсутствует символ после экранирующего символа                |
| –92 | BAD_INSIDE_CURLY     | Недопустимые символы внутри квантификатора в фигурных скобках |
| –91 | BAD_CURLY_ORDER      | Неверный порядок чисел в квантификаторе в фигурных скобках    |
| –90 | HUGE_CURLY_RANGE     | Диапазон квантификатора слишком велик                         |
| –89 | MISSING_LPAREN       | Отсутствует открывающая круглая скобка «(»                    |
| –88 | MISSING_RPAREN       | Отсутствует закрывающая круглая скобка «)»                    |
| –87 | BAD_SQUARE_ORDER     | Неверный порядок символов в диапазоне набора символов         |

Таблица 3.2 – Коды ошибок компиляции регулярного выражения

| Код | Константа     | Описание                                       |
|-----|---------------|------------------------------------------------|
| –86 | BAD_CHARCLASS | Некорректный (несуществующий) символьный класс |
| –85 | INVALID_FLAG  | Неверный модификатор регулярного выражения     |

### 3.4.7 Алгоритмы экспортируемых функций

Все экспортируемые функции библиотеки построены по единой схеме: публичная функция является тонкой обёрткой над внутренней реализацией, принимающей как строку шаблона, так и скомпилированный объект. Если передана строка шаблона, внутренняя функция компилирует её через `compile_re2`, выполняет операцию и немедленно освобождает объект через `free_re`. Если передан готовый объект, этапы компиляции и освобождения пропускаются. Такая схема позволяет избежать дублирования логики между `pattern-` и `obj-`версиями каждой функции.

#### 3.4.7.1 Семейство `matchpos`

Внутренняя функция `_matchpos` принимает либо строку шаблона, либо объект регулярного выражения. При получении строки шаблона она компилирует его через `compile_re2`; при ошибке компиляции записывает позицию ошибочного символа по указателю `capture_data` и возвращает код ошибки. При успешной компиляции вызывает `_match_wrapper`, передавая значение `*capture_data` как начальную позицию поиска, если оно положительно. Возвращает начальную позицию совпадения и записывает его длину по указателю `capture_data`. Если совпадений не найдено, возвращает `–1`.

#### 3.4.7.2 Семейство `is_match`

Внутренняя функция `_is_match` аналогична `_matchpos`, но передаёт в `_match_wrapper` `NULL` вместо указателя на длину захвата, поскольку длина результата не требуется. Возвращаемое значение нормализуется: `–1` от

`_match_wrapper` (совпадений нет) преобразуется в 0, любое неотрицательное значение – в 1.

#### **3.4.7.3 Семейство `match`**

Внутренняя функция `_match` выделяет структуру `re_matches` с массивами начальных позиций и длин групп захвата ёмкостью, равной числу групп захвата в скомпилированном объекте. После вызова `_match_wrapper` формирует результирующий массив: первым элементом записывается malloc-строка с полным совпадением, затем последовательно – строки для каждой группы захвата с непустым результатом. Массив завершается указателем на `NULL`. При отсутствии совпадений массив содержит только `NULL`. Временная структура `re_matches` освобождается до возврата.

#### **3.4.7.4 Семейство `matches`**

Внутренняя функция `_matches` выполняет итеративный поиск всех непесекающихся совпадений. На каждой итерации вызывается `_match_wrapper` с текущей позицией поиска `pos`. При успешном захвате подстрока копируется в malloc-строку и добавляется в результирующий массив; позиция `pos` сдвигается за конец найденного совпадения. При нулевом захвате позиция сдвигается на один символ, чтобы исключить зависание на месте. Цикл завершается при первом неудачном поиске или при достижении конца строки.

Результирующий массив строк реализован с динамическим наращиванием ёмкости: начальная ёмкость равна 10 элементам; при её исчерпании массив перевыделяется через `re_realloc` с удвоением ёмкости. Такая стратегия обеспечивает амортизированную стоимость добавления элемента  $O(1)$  и позволяет не знать заранее число совпадений. Массив завершается указателем `NULL`.

#### **3.4.7.5 Семейства `re_nreplace` и `re_dupreplace`**

Оба семейства реализованы через единую внутреннюю функцию `_re_replace`, поведение которой определяется значением указателя `*dest_ptr`: если он равен `NULL`, активируется режим динамического выделения памяти (`re_dupreplace`), иначе результат записывается в буфер фиксированного размера (`re_nreplace`).



Алгоритм замены работает в цикле: на каждой итерации вызывается `_match_wrapper` с текущей позицией `curr_char`. При нахождении совпадения в результирующий буфер сначала копируется префикс – часть исходной строки от `curr_char` до начала совпадения, – затем обрабатывается строка замены `repl`. В строке замены распознаются обратные ссылки вида `$n` и `\n` (где  $n$  от 0 до 9): `$0` заменяется полным совпадением, `$1`–`$9` – содержимым соответствующей группы захвата, если она была захвачена. Позиции и длины групп захвата хранятся в локальной структуре `re_matches` с массивами `starts` и `lengths` ёмкостью 10 элементов, расположенными на стеке. При нулевом захвате в буфер дополнительно записывается символ исходной строки на позиции совпадения, после чего позиция сдвигается на единицу (поведение при нулевом захвате не стандартизировано и различается между реализациями; в данной библиотеке принята модель PCRE/JavaScript как наиболее распространённая среди промышленных библиотек). После завершения цикла в буфер дописывается остаток исходной строки, начиная с `curr_char`.

В режиме фиксированного буфера копирование на каждом шаге ограничивается числом оставшихся байтов; при исчерпании буфера функция возвращает `-1`. В режиме динамического выделения начальный размер буфера равен длине исходной строки плюс один байт под нуль-терминатор. При нехватке места буфер перевыделяется через `re_realloc` с запасом, равным длине исходной строки, – это снижает число перевыделений при многократных заменах на строки, длиннее исходных совпадений. Функция возвращает число выполненных замен, а указатель на выделенный буфер записывается по `dest_ptr`.

### 3.4.8 Управление памятью и правила владения

Библиотека придерживается явных и однозначных правил владения памятью, которые должны соблюдаться вызывающей стороной.

Объекты регулярных выражений, созданные функциями `compile_re` и `compile_re2`, должны быть освобождены вызовом `free_re` после окончания использования. Библиотека не хранит глобальных указателей на такие объекты

и не освобождает их самостоятельно, кроме случаев внутреннего использования в pattern-версиях функций.

Массивы строк, возвращаемые функциями `match` и `matches`, выделены через `malloc` в два уровня: сам массив указателей и каждая строка в нём. Вызывающая сторона обязана освободить каждую строку через `free`, а затем освободить массив указателей. Строки, возвращаемые `re_dupreplace` и `obj_re_dupreplace`, также выделены через `malloc` и должны быть освобождены вызывающей стороной.

Буфер `dest` в функциях `re_nreplace` и `obj_re_nreplace` предоставляется вызывающей стороной и библиотекой не освобождается.

Все внутренние выделения памяти выполняются через обёртки `re_malloc` и `re_realloc`, завершающие процесс с сообщением об ошибке при исчерпании памяти, что исключает необходимость проверки возвращаемых значений внутри библиотеки. Скомпилированный объект `struct re_chain` в процессе сопоставления не модифицируется – все промежуточные данные о захватах хранятся в структуре `struct re_matches`, создаваемой на стеке при каждом вызове. Глобального изменяемого состояния библиотека не использует. Поэтому параллельное использование одного скомпилированного объекта из нескольких потоков безопасно без какой-либо синхронизации. Независимые объекты, созданные в разных потоках, также могут использоваться параллельно без ограничений.

## **3.5 Поисковое приложение**

### **3.5.1 Общая архитектура приложения**

Поисковое приложение построено по архитектурному шаблону, принятому во фреймворке Qt для приложений с графическим интерфейсом: каждому окну или диалогу соответствует пара классов – класс окна и автоматически генерируемый из `.ui`-файла класс `Ui::*`, инкапсулирующий виджеты интерфейса. Бизнес-логика сосредоточена в классах окон; прямое обращение к виджетам из внешних модулей не допускается.

Приложение работает в двух режимах: режиме текстового редактора с поиском и заменой в открытом файле и режиме поиска по директории. Переключение между режимами осуществляется открытием соответствующего диалогового окна из главного меню редактора.

Интеграция библиотеки регулярных выражений выполнена на уровне исходного кода: файлы `regex-lib.c` и `regex-lib.h` включены непосредственно в проект Qt и компилируются вместе с исходными файлами приложения. Заголовочный файл подключается через блок `extern "C"` во всех модулях, использующих API библиотеки.

Приложение состоит из пяти программных модулей, описанных в таблице 3.3.

Таблица 3.3 – Модули поискового приложения

| Модуль       | Класс Qt              | Назначение                                                                                                              |
|--------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------|
| main         | —                     | Точка входа; создаёт QApplication и главное окно TextEditor                                                             |
| texteditor   | QMainWindow           | Главное окно: текстовый редактор, меню, строка состояния, масштабирование                                               |
| searchdialog | QDialog               | Немодальный диалог поиска и замены в открытом файле                                                                     |
| searchdir    | QDialog               | Немодальный диалог рекурсивного поиска по файлам директории                                                             |
| searchworker | QObject,<br>QRunnable | Фоновые классы поиска:<br>SearchController управляет обходом директории, FileSearchWorker выполняет поиск в одном файле |

### 3.5.2 Модуль `TextEditor`

Класс `TextEditor` наследует `QMainWindow` и является главным окном приложения. Центральным виджетом служит `QPlainTextEdit` – `text_edit_main`, в котором отображается и редактируется содержимое открытого файла. Строка состояния `QStatusBar` отображает текущую позицию курсора (строка и столбец), тип переноса строк и масштаб отображения текста.

Модуль реализует следующие функции.

*Открытие файла.* Слот `on_action_open_triggered` открывает диалог выбора файла `QFileDialog`. Файл читается как байтовый массив; двоичные файлы отклоняются с диалогом ошибки. Кодировка определяется автоматически: при обнаружении файла с кодировкой CP1251 происходит его конвертация в UTF-8 для дальнейшей работы. Тип переноса строк (LF, CR, CRLF) определяется функцией `detectLineEnding` и сохраняется для последующей записи.

*Сохранение файла.* Слот `on_action_save_triggered` сохраняет текущее содержимое редактора в исходный файл с восстановлением оригинального типа переноса строк. Слот `on_action_save_as_triggered` открывает диалог сохранения `QFileDialog` и позволяет выбрать новое имя или путь.

*Масштабирование.* Слоты `on_action_zoom_in_triggered`, `on_action_zoom_out_triggered` и `on_action_zoom_100_triggered` изменяют размер шрифта редактора по списку стандартных значений `ZOOM_SIZES`. Масштабирование также доступно через `Ctrl+Колёсико мыши` посредством фильтра событий `eventFilter`. Текущий масштаб в процентах отображается в строке состояния.

*Подсветка совпадений.* Для подсветки найденных совпадений применяется механизм дополнительных выделений `QPlainTextEdit::setExtraSelections`, управляемый классом `SearchDialog`. Список выделений сбрасывается при изменении текста через соединение сигнала `textChanged` со слотом `SearchDialog::clearHighlights`.

*Отображение позиции курсора.* Сигнал `cursorPositionChanged` редактора соединён со слотом `onCursorPositionChanged`, который вычисляет текущие номер

строки и столбец через QTextCursor::blockNumber и QTextCursor::positionInBlock и отображает их в постоянном виджете status\_cursor строки состояния.

*Взаимодействие с диалогом поиска.* Объект SearchDialog создаётся при инициализации TextEditor и существует на протяжении всего времени жизни главного окна. Диалог является немодальным. Слот on\_action\_find\_triggered показывает диалог; при первом показе диалог позиционируется справа от главного окна, если позволяет место на экране, иначе – по центру экрана.

Поля класса TextEditor приведены в таблице 3.4.

Таблица 3.4 – Поля класса TextEditor

| Поле                 | Тип             | Назначение                                                      |
|----------------------|-----------------|-----------------------------------------------------------------|
| ui                   | Ui::TextEditor* | Указатель на виджеты, сгенерированные из .ui-файла              |
| search_dialog        | SearchDialog*   | Немодальный диалог поиска и замены                              |
| search_dir_dialog    | SearchDir*      | Немодальный диалог поиска по директории; nullptr если не открыт |
| current_file_path    | QString         | Путь к текущему открытому файлу; пуст, если файл не открыт      |
| current_line_ending  | LineEnding      | Тип переноса строк открытого файла (LE_LF, LE_CR, LE_CRLF)      |
| current_zoom_percent | int             | Текущий масштаб отображения в процентах                         |
| status_cursor        | QLabel*         | Постоянный виджет строки состояния: позиция курсора             |
| status_line_ending   | QLabel*         | Постоянный виджет строки состояния: тип переноса строк          |
| status_zoom          | QLabel*         | Постоянный виджет строки состояния: масштаб                     |

### 3.5.3 Модуль SearchDialog

Класс SearchDialog наследует QDialog и реализует немодальный диалог поиска и замены. Диалог принимает указатель на QPlainTextEdit главного окна и указатель на QStatusBar и напрямую работает с содержимым и курсором редактора.

Диалог поддерживает два режима поиска, выбираемых переключателями (QRadioButton): обычный текстовый поиск средствами QTextDocument::find и поиск по регулярному выражению с использованием API библиотеки regex-lib. В режиме регулярных выражений дополнительно доступны опции многострочного режима ((?m)) и режима «"." – новая строка» ((?s)); опция поиска целых слов в этом режиме недоступна и блокируется автоматически.

Область поиска может быть ограничена текущим выделением через флажок «Только в выделении». Поддерживается циклический поиск: при достижении конца текста поиск продолжается с начала с уведомлением в строке статуса.

Панель замены скрыта по умолчанию и разворачивается кнопкой-переключателем. В режиме поиска вместо кнопки «Заменить всё» отображается кнопка «Подсветить» – она подсвечивает все совпадения в области поиска через setExtraSelections.

Модуль реализует следующие операции.

*Поиск следующего совпадения.* Метод findNext\_regex определяет область поиска через getSearchScope, вычисляет начальную позицию в байтах UTF-8 от текущего положения курсора и вызывает matchpos. Функция selectInEditor переводит байтовое смещение в позицию символа Qt и выделяет найденный фрагмент. При нулевом захвате выделяется ближайший символ с уведомлением в строке статуса.

*Подсчёт и подсветка совпадений.* Метод collectMatches итеративно вызывает matchpos и заполняет список found\_matches. Метод highlightAll формирует список QTextEdit::ExtraSelection по этому списку и передаёт его в setExtraSelections.

*Замена одного совпадения.* Слот `on_btn_replace_clicked` в режиме регулярных выражений вызывает `re_dupreplace` с `limit=1`, применяя замену к фрагменту текста начиная с позиции текущего выделения, после чего выполняет поиск следующего совпадения.

*Замена всех совпадений.* Слот `on_btn_replace_all_highlight_clicked` в режиме замены вызывает `re_dupreplace` с `limit=0` для области поиска и заменяет соответствующий фрагмент содержимого редактора результатом. При ограниченной области поиска результат склеивается с частями текста за её пределами.

*Валидация шаблона.* Метод `validatePattern` компилирует шаблон через `compile_re2` и при обнаружении ошибки отображает HTML-форматированное сообщение с подчёркиванием ошибочного символа. Смещение позиции ошибки корректируется на длину автоматически добавленного префикса модификаторов.

Поля класса `SearchDialog` приведены в таблице 3.5.

Таблица 3.5 – Поля класса `SearchDialog`

| Поле                           | Тип                             | Назначение                                                                                   |
|--------------------------------|---------------------------------|----------------------------------------------------------------------------------------------|
| <code>ui</code>                | <code>Ui::SearchDialog*</code>  | Указатель на виджеты, сгенерированные из <code>.ui</code> -файла                             |
| <code>text_editor</code>       | <code>QPlainTextEdit*</code>    | Указатель на редактор главного окна                                                          |
| <code>main_status_bar</code>   | <code>QStatusBar*</code>        | Строка состояния главного окна                                                               |
| <code>replace_visible</code>   | <code>bool</code>               | Флаг видимости панели замены                                                                 |
| <code>found_matches</code>     | <code>QList&lt;Match&gt;</code> | Список найденных совпадений (байтовые позиции и длины) для подсветки                         |
| <code>current_match_idx</code> | <code>int</code>                | Индекс текущего активного совпадения в <code>found_matches</code> ; <code>-1</code> если нет |

### 3.5.4 Модуль SearchDir

Класс SearchDir наследует QDialog и реализует немодальный диалог рекурсивного поиска по регулярному выражению или обычному тексту во всех текстовых файлах выбранной директории. Диалог открывается из главного меню редактора после выбора директории.

Интерфейс диалога включает поле ввода пути к директории с кнопкой обзора, поле ввода шаблона, комбинированный список масок расширений файлов с предустановленными вариантами («Текстовые файлы», «Исходный код», «Все файлы»), флажки режимов поиска, кнопки запуска и остановки, прогресс-бар и дерево результатов QTreeWidget.

Дерево результатов организовано двухуровневой иерархией: узлы первого уровня соответствуют файлам с найденными совпадениями и содержат относительный путь к файлу и число совпадений в нём; узлы второго уровня соответствуют отдельным совпадениям и содержат номер строки, номер столбца и фрагмент текста вокруг совпадения. При двойном щелчке по любому узлу соответствующий файл открывается в редакторе, а курсор переводится на строку найденного совпадения.

Поиск выполняется в фоновом потоке через класс SearchController, помещаемый в QThread. SearchController обходит директорию через QDirIterator и создаёт объекты FileSearchWorker — по одному на каждый файл. Воркеры помещаются в локальный QThreadPool; при включённом многопоточном режиме пул использует все доступные ядра процессора, при выключенном — одно. Завершение поиска отслеживается через атомарный счётчик активных воркеров active\_workers.

Для автоматического исключения двоичных файлов из поиска применяется эвристический метод: из каждого файла считываются первые 8 КБ содержимого, и при обнаружении нулевого байта файл классифицируется как двоичный и пропускается. Данный подход основан на том, что текстовые файлы в кодировках ASCII, UTF-8 и CP1251 не содержат нулевых байтов, то-



гда как подавляющее большинство двоичных форматов содержит их в самом начале файла.

Остановка поиска реализована через атомарный флаг `stop_flag` типа `QAtomicInt`, проверяемый в каждой итерации воркера. Кнопка «Найти» блокируется на время завершения потока после запроса остановки и разблокируется по сигналу `searchFinished`.

Поля класса `SearchDir` приведены в таблице 3.6.

Таблица 3.6 – Поля класса `SearchDir`

| Поле                        | Тип                            | Назначение                                                                                              |
|-----------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>ui</code>             | <code>Ui::SearchDir*</code>    | Указатель на виджеты, сгенерированные из <code>.ui</code> -файла                                        |
| <code>text_editor</code>    | <code>TextEditor*</code>       | Указатель на главное окно для открытия найденных файлов                                                 |
| <code>re_obj</code>         | <code>struct re_chain*</code>  | Скомпилированный объект регулярного выражения; передаётся воркерам и освобождается по завершении поиска |
| <code>worker</code>         | <code>SearchController*</code> | Объект контроллера фонового поиска                                                                      |
| <code>thread</code>         | <code>QThread*</code>          | Фоновый поток, в котором выполняется <code>SearchController</code>                                      |
| <code>search_running</code> | <code>bool</code>              | Флаг активного поиска; управляет состоянием кнопки запуска/остановки                                    |
| <code>total_matches</code>  | <code>int</code>               | Суммарное число найденных совпадений по всем файлам                                                     |

### 3.5.5 Интеграция библиотеки регулярных выражений

В модулях SearchDialog и SearchDir взаимодействие с библиотекой регулярных выражений организовано по единой схеме. При запуске поиска шаблон компилируется однократно через `compile_re2`; полученный объект `struct re_chain*` передается во все последующие вызовы `obj_matchpos` и `obj_re_dupreplac`. По завершении работы с объектом вызывается `free_re`. Такой подход исключает повторную компиляцию одного и того же шаблона при итеративном поиске и особенно важен в модуле SearchDir, где один шаблон применяется последовательно к большому числу файлов.

Передача текста между Qt и библиотекой выполняется через промежуточное UTF-8-представление: `QString` преобразуется в `QByteArray` методом `toUtf8`, после чего указатель `QByteArray::data` передается в функции библиотеки. Результирующие позиции совпадений, возвращаемые в байтовых смещениях UTF-8, преобразуются обратно в позиции символов Qt вспомогательной функцией `selectInEditor` через `QString::fromUtf8`.

## **4 Рабочий проект**

В процессе разработки программно-информационной поисковой системы были реализованы два самостоятельных программных компонента: библиотека регулярных выражений на языке Си и поисковое приложение на C++ с использованием фреймворка Qt.

### **4.1 Библиотека регулярных выражений**

#### **4.1.1 Состав библиотеки и условия применения**

Библиотека регулярных выражений поставляется в виде двух файлов: `regex-lib.c` — реализация всех внутренних и экспортируемых функций, и `regex-lib.h` — заголовочный файл с объявлениями экспортируемых функций, непрозрачного типа `struct re_chain` и перечисления кодов ошибок `enum re_error_type`. Для подключения библиотеки к проекту достаточно добавить оба файла в систему сборки и включить заголовочный файл. В C++-проектах подключение выполняется без дополнительных настроек благодаря блоку `extern "C"` в заголовочном файле.

Библиотека протестирована на следующих платформах и компиляторах:

- Windows 10/11, Clang 17;
- Linux (Ubuntu 22.04), GCC 12;
- онлайн-компиляторы на базе GCC и Clang (OnlineGDB и аналоги).

При компиляции не возникает предупреждений. Проверка санитайзерами адресов и неопределённого поведения (`-fsanitize=address,undefined`) не выявила ошибок.

#### **4.1.2 Сборка библиотеки**

Для сборки библиотеки не требуется система автоматической сборки и сторонние зависимости — достаточно компилятора, поддерживающего стандарт C99 или новее. Команды сборки представлены в таблице 4.1.

Таблица 4.1 – Команды сборки библиотеки

| Режим                                  | Команда                                                                                              |
|----------------------------------------|------------------------------------------------------------------------------------------------------|
| Отладочная<br>сборка тестов            | <code>gcc -g -Wall -Wextra -fsanitize=address,undefined<br/>regex-tests.c -o regex-tests-dbg</code>  |
| Оптимизиро-<br>ванная сборка<br>тестов | <code>gcc -O2 regex-tests.c -o regex-tests</code>                                                    |
| Релизная<br>сборка<br>библиотеки       | <code>gcc -O2 -DNDEBUG -c regex-lib.c -o regex-lib.o<br/>gcc program.c regex-lib.o -o program</code> |

Команды идентичны при использовании Clang: достаточно заменить gcc на clang. Тестовый файл regex-tests.c включает реализацию библиотеки на-  
прямую директивой `#include "regex-lib.c"`, что позволяет собирать тесты един-  
ственной командой без отдельной компиляции. При интеграции библиотеки  
в сторонний проект рекомендуется отдельная компиляция.

### 4.1.3 Экспортируемые функции библиотеки

Библиотека предоставляет C API, объявленное в заголовочном файле  
regex-lib.h. Все экспортируемые функции разделены на семейства по выпол-  
няемой операции. Для каждого семейства предоставляются две версии: при-  
нимающая строку шаблона и выполняющая его компиляцию при каждом вы-  
зове, и версия с префиксом `obj_`, принимающая заранее скомпилированный  
объект типа `struct re_chain*`. Использование `obj_-`версий предпочтительно при  
многократном применении одного шаблона. Помимо экспортируемых функ-  
ций, заголовочный файл объявляет непрозрачный тип `struct re_chain` и пере-  
числение кодов ошибок `enum re_error_type`, что позволяет потребителю API  
обрабатывать конкретные коды ошибок без доступа к внутренней реализа-  
ции.

Дополнительно в файле исходного кода библиотеки доступны две вспомогательные функции, не входящие в публичный API и предназначенные для отладки: `print_re_blocks` – форматированный вывод внутреннего представления скомпилированного регулярного выражения в консоль и `get_error_msg` – получение текстового описания кода ошибки на английском языке.

#### 4.1.3.1 Компиляция шаблона регулярного выражения

Функции этого семейства выполняют разбор строки шаблона и построение внутреннего представления регулярного выражения, а также освобождение занимаемой им памяти. Аргументы функций приведены в таблицах 4.2 и 4.3.

Возвращаемое значение функций компиляции – указатель на непрозрачный тип `struct re_chain*`:

1. Не-NULL – компиляция успешна, возвращен объект регулярного выражения.
2. NULL – ошибка в строке шаблона.

Прототипы функций:

1. `struct re_chain* compile_re(char* pattern)`
2. `struct re_chain* compile_re2(char* pattern, int* errcode, int* errpos)`

Таблица 4.2 – Аргументы функций компиляции

| Аргумент                                                     | Описание                                    | Допускается NULL? |
|--------------------------------------------------------------|---------------------------------------------|-------------------|
| <code>char* pattern</code>                                   | Строка шаблона регулярного выражения        |                   |
| <code>int* errcode</code><br>(для <code>compile_re2</code> ) | Указатель для записи кода ошибки компиляции | Да                |

Таблица 4.2 – Аргументы функций компиляции

| Аргумент                      | Описание                                                                               | Допускается NULL? |
|-------------------------------|----------------------------------------------------------------------------------------|-------------------|
| int* errpos (для compile_re2) | Указатель для записи позиции символа в шаблоне, при разборе которого обнаружена ошибка | Да                |

Функция `free_re` освобождает всю память, выделенную под объект регулярного выражения и все его внутренние структуры. Должна быть вызвана для каждого объекта, созданного функциями `compile_re` или `compile_re2`, после окончания его использования. Функция ничего не возвращает.

Прототип функции:

1. `void free_re(struct re_chain* re_obj)`

Таблица 4.3 – Аргументы функции освобождения памяти

| Аргумент                | Описание                                                                                            | Допускается NULL? |
|-------------------------|-----------------------------------------------------------------------------------------------------|-------------------|
| struct re_chain* re_obj | Указатель на скомпилированный объект регулярного выражения, память которого должна быть освобождена |                   |

#### 4.1.3.2 Поиск позиции совпадения

Функции этого семейства находят первое совпадение шаблона в строке и возвращают его начальную позицию и длину. Аргументы функций приведены в таблице 4.4.

Возвращаемое значение – целое число:

1.  $\geq 0$  – захват успешен, его начальная позиция во входной строке.
2.  $-1$  – совпадений во входной строке не найдено.
3.  $< -1$  – ошибка в строке шаблона с возвращенным кодом.

Прототипы функций:

1. `int matchpos(char* pattern, char* text, int* capture_data)`
2. `int obj_matchpos(struct re_chain* re_obj, char* text, int* capture_data)`

Таблица 4.4 – Аргументы функций поиска позиции совпадения

| Аргумент                                             | Описание                                                                                                                                     | Допускается NULL? |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| <code>char* pattern / struct re_chain* re_obj</code> | Строка шаблона регулярного выражения / Откомпилированный объект регулярного выражения                                                        |                   |
| <code>char* text</code>                              | Входная строка для поиска                                                                                                                    |                   |
| <code>int* capture_data</code>                       | Указатель на целое: при вызове – начальная позиция поиска (если значение > 0); после вызова – длина найденного совпадения или позиция ошибки | Да                |

#### 4.1.3.3 Валидация строки

Функции этого семейства проверяют, содержит ли входная строка хотя бы одно совпадение с шаблоном, и возвращают булево значение. Аргументы функций приведены в таблице 4.5.

Возвращаемое значение – целое число:

1. 1 – входная строка удовлетворяет регулярному выражению.
2. 0 – входная строка не подходит под регулярное выражение.
3. < -1 – ошибка в строке шаблона с возвращенным кодом.

Прототипы функций:

1. `int is_match(char* pattern, char* text)`
2. `int obj_is_match(struct re_chain* re_obj, char* text)`

Таблица 4.5 – Аргументы функций валидации строки

| Аргумент                                      | Описание                                                                                   | Допускается NULL? |
|-----------------------------------------------|--------------------------------------------------------------------------------------------|-------------------|
| char* pattern /<br>struct re_chain*<br>re_obj | Строка шаблона регулярного выражения /<br>Скомпилированный объект регулярного<br>выражения |                   |
| char* text                                    | Входная строка для валидации                                                               |                   |

#### 4.1.3.4 Извлечение одного совпадения с группами захвата

Функции этого семейства возвращают NULL-терминированный массив строк, содержащий полное совпадение и все захваченные группы, аналогично функции `match()` в языке JavaScript. Аргументы функций приведены в таблице 4.6.

Возвращаемое значение – NULL-терминированный массив malloc-строк:

1. Указатель на указатель на строку – захват успешен: первый элемент массива – общий захват, последующие (пока не встретится указатель на NULL) – группы захвата.
2. Указатель на указатель на NULL – входная строка не подходит под регулярное выражение.
3. NULL – ошибка в строке шаблона с возвращенным кодом.

Прототипы функций:

1. char\*\* `match(char* pattern, char* text)`
2. char\*\* `obj_match(struct re_chain* re_obj, char* text)`



Таблица 4.6 – Аргументы функций извлечения одного совпадения

| Аргумент                                      | Описание                                                                                   | Допускается NULL? |
|-----------------------------------------------|--------------------------------------------------------------------------------------------|-------------------|
| char* pattern /<br>struct re_chain*<br>re_obj | Строка шаблона регулярного выражения /<br>Скомпилированный объект регулярного<br>выражения |                   |
| char* text                                    | Входная строка для поиска                                                                  |                   |

#### 4.1.3.5 Извлечение всех совпадений

Функции этого семейства возвращают NULL-терминированный массив строк, содержащий все непересекающиеся совпадения регулярного выражения во входной строке, аналогично функции `match()` с флагом `g` в языке JavaScript. Аргументы функций приведены в таблице 4.7.

Возвращаемое значение – NULL-терминированный массив `malloc`-строк:

1. Указатель на массив строк – все найденные совпадения; если совпадений нет, первым элементом массива будет NULL.
2. NULL – ошибка в строке шаблона.

Прототипы функций:

1. `char** matches(char* pattern, char* text)`
2. `char** obj_matches(struct re_chain* re_obj, char* text)`

Таблица 4.7 – Аргументы функций извлечения всех совпадений

| Аргумент                                      | Описание                                                                                   | Допускается NULL? |
|-----------------------------------------------|--------------------------------------------------------------------------------------------|-------------------|
| char* pattern /<br>struct re_chain*<br>re_obj | Строка шаблона регулярного выражения /<br>Скомпилированный объект регулярного<br>выражения |                   |
| char* text                                    | Входная строка для поиска                                                                  |                   |

#### 4.1.3.6 Замена совпадений в буфере фиксированного размера

Функции этого семейства выполняют замену совпадений шаблона на строку замены в предоставленном пользователем буфере фиксированного размера. Аргументы функций приведены в таблице 4.8.

Возвращаемое значение – целое число:

1.  $\geq 0$  – число успешно выполненных замен.
2.  $-1$  – в буфере dest закончилось место – результат неполный.
3.  $< -1$  – ошибка в строке шаблона с возвращенным кодом.

Прототипы функций:

1. int re\_nreplace(char\* pattern, char\* repl, char\* text, char\* dest, int n, int limit)
2. int obj\_re\_nreplace(struct re\_chain\* re\_obj, char\* repl, char\* text, char\* dest, int n, int limit)

Таблица 4.8 – Аргументы функций замены с фиксированным буфером

| Аргумент                                      | Описание                                                                                   | Допускается NULL? |
|-----------------------------------------------|--------------------------------------------------------------------------------------------|-------------------|
| char* pattern /<br>struct re_chain*<br>re_obj | Строка шаблона регулярного выражения /<br>Скомпилированный объект регулярного<br>выражения |                   |

Таблица 4.8 – Аргументы функций замены с фиксированным буфером

| Аргумент   | Описание                                                                                  | Допускается NULL? |
|------------|-------------------------------------------------------------------------------------------|-------------------|
| char* repl | Строка замены; может содержать ссылки на группы захвата вида \$n или \n (где n от 0 до 9) |                   |
| char* text | Исходная строка, в которой выполняется замена                                             |                   |
| char* dest | Указатель на буфер для записи результата замены                                           |                   |
| int n      | Размер буфера dest в байтах                                                               |                   |
| int limit  | Максимальное число замен; 0 – без ограничений                                             |                   |

#### 4.1.3.7 Замена совпадений с динамическим выделением памяти

Функции этого семейства выполняют замену совпадений шаблона на строку замены, автоматически выделяя результирующий буфер необходимого размера в динамической памяти. Аргументы функций приведены в таблице 4.9.

Возвращаемое значение – указатель на malloc-строку или NULL:

1. Не-NULL – указатель на строку с результатом замены (должна быть освобождена вызывающей стороной).
2. NULL – ошибка в строке шаблона (код ошибки при этом записывается по указателю repl\_cnt).

Прототипы функций:

1. char\* re\_dupreplace(char\* pattern, char\* repl, char\* text, int limit, int\* repl\_cnt)
2. char\* obj\_re\_dupreplace(struct re\_chain\* re\_obj, char\* repl, char\* text, int limit, int\* repl\_cnt)

Таблица 4.9 – Аргументы функций замены с динамическим выделением памяти

| Аргумент                                      | Описание                                                                                               | Допускается NULL? |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------|
| char* pattern /<br>struct re_chain*<br>re_obj | Строка шаблона регулярного выражения /<br>Скомпилированный объект регулярного<br>выражения             |                   |
| char* repl                                    | Строка замены; может содержать ссылки<br>на группы захвата вида \$n или \n (где <i>n</i> от<br>0 до 9) |                   |
| char* text                                    | Исходная строка, в которой выполняется<br>замена                                                       |                   |
| int limit                                     | Максимальное число замен; 0 – без<br>ограничений                                                       |                   |
| int* repl_cnt                                 | Указатель для записи числа выполненных<br>замен или отрицательного кода ошибки                         | Да                |

#### 4.1.4 Тестирование библиотеки регулярных выражений

Для верификации корректности библиотеки проведено модульное и интеграционное тестирование. Модульное тестирование охватывает каждое семейство экспортируемых функций в отдельности: тесты модулей `check_correctness`, `matchpos`, `matchstart` проверяют функции компиляции и сопоставления изолированно. Интеграционное тестирование охватывает совместную работу нескольких компонентов: модуль `is_match` проверяет цепочку «компиляция через `compile_re2` – сопоставление через `obj_is_match`»; модули `matcharr`, `matches` и `replace` проверяют полный конвейер «компиляция – сопоставление – извлечение подстрок / замена».

В файле `regex-tests.c` содержится встроенный набор автоматизированных тестов. Тестирование библиотеки выполняется в первых двух профилях сборки (отладочной и оптимизированной), описанных в таблице 4.1.

Профиль отладочной сборки используется для выявления ошибок работы с памятью и неопределённого поведения средствами санитайзеров `AddressSanitizer` и `UndefinedBehaviorSanitizer`. Время выполнения тестов в этом профиле не показательно из-за накладных расходов инструментации.

Профиль оптимизированной сборки выполняет тестирование в условиях, приближённых к промышленной эксплуатации и включает в себя агрессивные оптимизации компилятора, которые могут проявить скрытое неопределённое поведение (UB), не видимое при отладочной сборке.

В отладочном профиле вывод санитайзеров должен быть чистым – любое сообщение от `AddressSanitizer` или `UBSanitizer` означает ошибку в реализации. В оптимизированном профиле дополнительно фиксируется время выполнения тест-сюта, которое при сборке с `-O2` составляет 0,022 с.

Файл `regex-tests.c` включает реализацию библиотеки директивой `#include "regex-lib.c"`, что позволяет обойтись без отдельной компиляции при запуске тестов.

Тесты организованы в модули с помощью макросов `NEW_MODULE(name)` и `FINISH_MODULE()`. Результат отдельного теста учитывается макросами `TEST_OK(...)` и `TEST_FAIL(...)`, фиксирующими успешное и неудачное прохождение теста соответственно. По завершении каждого модуля выводится число пройденных тестов. Тесты, помеченные макросом `SPECIAL_TEST`, не учитываются в статистике – они проверяют граничные случаи, поведение в которых не стандартизировано и различается среди промышленных библиотек регулярных выражений (PCRE, Go `regexp`, Notepad++).

Тест-сют разделён на шесть модулей, описанных ниже.

#### **4.1.4.1 Модуль `check_correctness`**

Модуль проверяет корректность диагностики синтаксических ошибок в шаблонах регулярных выражений. Для каждого тестового шаблона функ-

ция `assert_check_correctness` вызывает `compile_re2` и сравнивает возвращённый код ошибки с эталонным значением из перечисления `re_error_type`. В выводе для каждого теста указывается код ошибки, её текстовое описание и позиция ошибочного символа в шаблоне.

Тесты охватывают все 16 кодов ошибок, определённых в библиотеке, включая корректные шаблоны (ожидаемый результат `RE_OK`), а также условную компиляцию для режима `STRICT_CHARCLASSES`, запрещающего экранирование букв, не используемых в символьных классах (например `j`, `q`). Всего в модуле 38 тестов.

Результат выполнения модуля `check_correctness` представлен на рисунке 4.1.

```
1  --- Module "check_correctness" ---
2  OK 0 "OK"
3  OK -100 "Unescaped ^ not in the beginning of regex or group" ('^' at 1)
4  OK -99 "Unescaped $ not in the end of regex" ('$' at 0)
5  OK -98 "Preceding token is not quantifiable" ('+' at 14)
6  OK -98 "Preceding token is not quantifiable" ('*' at 12)
7  OK -98 "Preceding token is not quantifiable" ('?' at 3)
8  OK -97 "Empty square brackets" (']' at 3)
9  OK -97 "Empty square brackets" (']' at 4)
10 OK -96 "Missing opening square bracket" (']' at 5)
11 OK -95 "Missing closing square bracket" ('[' at 8)
12 OK -94 "Range with character class is unacceptable" ('-' at 3)
13 OK -94 "Range with character class is unacceptable" ('-' at 2)
14 OK 0 "OK"
15 OK -93 "Missing character after escape character" ('\ ' at 4)
16 OK 0 "OK"
17 OK -100 "Unescaped ^ not in the beginning of regex or group" ('^' at 5)
18 OK 0 "OK"
19 OK 0 "OK"
20 OK 0 "OK"
21 OK 0 "OK"
22 OK -92 "Invalid characters inside curly brackets" ('a' at 8)
23 OK -92 "Invalid characters inside curly brackets" (',' at 7)
24 OK -91 "Quantifier range is out of order" ('}' at 7)
25 OK -90 "Quantifier range is too large" ('9' at 14)
26 OK 0 "OK"
27 OK 0 "OK"
28 OK -88 "Missing closing parenthesis" (')' at 3)
29 OK -89 "Missing opening parenthesis" ('(' at 2)
30 OK 0 "OK"
31 OK -88 "Missing closing parenthesis" (')' at 15)
32 OK -89 "Missing opening parenthesis" ('(' at 14)
33 OK 0 "OK"
34 OK -87 "Character range is out of order" ('y' at 8)
35 OK -85 "Invalid regex modifier" ('j' at 3)
```

```

36 OK -88 "Missing closing parenthesis" ( ' ' at 4)
37 OK 0 "OK"
38 OK 0 "OK"
39 OK -86 "Invalid character class" ( 'j' at 2)
40 All module tests (38) passed

```

Рисунок 4.1 – Результат выполнения модуля `check_correctness`

#### 4.1.4.2 Модуль `matchpos`

Модуль проверяет весь основной синтаксис регулярных выражений на основе функции `matchpos` – поиска первого совпадения шаблона в строке. Функция `assert_match` вызывает `matchpos` с нулевой начальной позицией и сравнивает найденную подстроку с эталонной строкой. Для тестов, ожидающих отсутствие совпадений, эталонным значением является `NULL`.

Тесты охватывают следующие возможности библиотеки:

- базовые метасимволы и квантификаторы (`.`, `*`, `+`, `?`, `{n,m}`);
- наборы символов (`[...]`, `[^...]`) и диапазоны;
- якоря `^` и `$` в том числе в составе альтераций;
- альтерацию с выбором наиболее раннего и наиболее длинного захвата;
- группы захвата, незахватываемые группы (`?:...`) и обратные ссылки `\1`;
- символьные классы `\d`, `\w`, `\s`, `\a`, `\r` и их инверсии;
- ленивые квантификаторы `*?`, `+?`, `??`;
- модификаторы `(?s)`, `(?m)`, `(?i)`, `(?I)` (в том числе регистронезависимый поиск с кириллицей в UTF-8);
- работу с пустыми шаблонами и пустыми строками.

Три теста помечены как особые (`SPECIAL_TEST`): выбор между альтерациями с одинаковой начальной позицией и ленивый захват внутри групп. Всего в модуле 136 тестов, из которых 3 особых.

Результат выполнения модуля `matchpos` представлен на рисунке 4.2.

```

1 --- Module "matchpos" ---
2 OK "abc"
3 OK "dksst"
4 OK "bfd"

```

```
5 OK "a123bc9"
6 OK "heo"
7 OK NULL
8 OK "abcktrrr"
9 OK "b 4d_"
10 OK NULL
11 OK ""
12 OK ""
13 OK ""
14 OK ""
15 OK ""
16 OK NULL
17 OK "aaaabw"
18 OK "test  string 12"
19 OK "Helo"
20 OK "привет"
21 OK "at"
22 OK "[][q[w"
23 OK "aaabab"
24 OK "c"
25 OK "1111"
26 OK "aa"
27 OK ""
28 OK "aaaaa"
29 OK ""
30 OK "aaaa"
31 OK "aba"
32 OK NULL
33 OK "1234"
34 OK "3456"
35 OK NULL
36 OK ""
37 OK ""
38 OK "27:15"
39 OK "2:15"
40 OK "1:27"
41 OK NULL
42 OK "https://swsu.ru"
43 OK NULL
44 OK NULL
45 OK "http://www.swsu.ru"
46 OK "abc@af-af.de"
47 OK NULL
48 OK NULL
49 OK NULL
50 OK "f"
51 OK "wq"
52 OK NULL
53 OK NULL
54 OK "def"
55 OK "abc"
56 OK ""
57 OK "bddd"
58 OK "1st"
```



```

59 OK "stecond"
60 OK NULL
61 OK "ddd"
62 OK NULL
63 * OK "aabcc"
64 OK "abcd a1 asdd asd asad da"
65 OK "abcd a1 asdd asd asa"
66 OK "abcd a1 asdd asd asad das"
67 OK NULL
68 OK "abcdef"
69 OK "abcabc"
70 OK ""
71 OK "1abcabc"
72 OK "qabqwcdab"
73 OK "aaaaaaaaa"
74 OK "aaaaaaaaaaaaa"
75 OK "abcdet2b!end"
76 OK NULL
77 OK "abcdet2end"
78 OK "cd"
79 OK ""
80 OK ""
81 OK "https://www.youtube.com/"
82 OK "https://youtube-nocookie.com/"
83 OK "http://youtu.be/"
84 OK NULL
85 OK NULL
86 OK "https://i.ytimg.com/"
87 OK "abed"
88 OK NULL
89 OK "abc"
90 OK NULL
91 OK "134"
92 OK "1345"
93 OK "13445"
94 OK "124"
95 OK "abABC! "
96 OK "abAB\|_ "
97 OK "abc45"
98 OK "abc_d1"
99 OK "ab#12$YZA"
100 OK "abc,          d!ef"
101 OK "a    b
102 "
103 OK "ba"
104 OK NULL
105 OK "ab"
106 OK "a"
107 OK "abcc"
108 OK "a"
109 OK ""witch" and her "broom""
110 OK ""witch""
111 OK "<p> <b> Wikipedia </b> encyclopedia. </p>"
112 OK "<p>"

```

```

113 OK "aa"
114 OK "aaa b"
115 OK "{FW}"
116 OK "869759002514931"
117 OK "3.1415926"
118 OK "y!y!"
119 OK NULL
120 OK "#y!y!"
121 OK NULL
122 OK "abab"
123 OK "abab"
124 OK "aa"
125 OK NULL
126 OK "'She's the one!'"
127 OK "_ 'a b c' 'd e f' !"
128 * FAIL NULL != "'She's the one!'"
129 * FAIL [] != [" test s ", "test s"]
130 OK "line1"
131 OK ".*?"
132 OK "line1
133 line2"
134 OK "b"
135 OK "b"
136 OK "bc"
137 OK "TEst"
138 OK "TEst"
139 OK "ATest"
140 OK "поддержка"
141 OK "поддержка"
142 OK "Тестовый поиск"
143 All module tests (136) passed

```

Рисунок 4.2 – Результат выполнения модуля matchpos

#### 4.1.4.3 Модуль matchstart

Модуль проверяет корректность поиска с заданной начальной позицией – передачи положительного значения по указателю capture\_data при вызове matchpos. Функция assert\_matchstart принимает дополнительный параметр start\_pos и вызывает matchpos с этим значением.

Тесты проверяют: поиск с ненулевой позиции, взаимодействие начальной позиции с якорем ^ (поиск не должен находить совпадение, если якорь не применим с заданной позиции), а также поиск с позицией, выходящей за пределы строки. Всего 4 теста.

Результат выполнения модуля matchstart представлен на рисунке 4.3.

```

1  --- Module "matchstart" ---
2  OK " b"
3  OK "</b>"
4  OK NULL
5  OK NULL
6  All module tests (4) passed

```

Рисунок 4.3 – Результат выполнения модуля matchstart

#### 4.1.4.4 Модуль is\_match

Модуль проверяет корректность функции obj\_is\_match – валидации строки на соответствие шаблону. Функция assert\_is\_match для каждого теста компилирует шаблон через compile\_re2, вызывает obj\_is\_match и сравнивает булев результат с эталонным. Использование obj\_-версии дополнительно проверяет корректность работы с предварительно скомпилированным объектом.

Тесты охватывают: базовое совпадение и его отсутствие, якоря в составе групп, а также практический шаблон проверки на простоту чисел в унарной системе счисления  $^1?\$|^((11+)\1+)$$  от 1 до 17, и шаблон валидации адреса электронной почты, используемый в поле HTML `<input type="email">`. Всего 20 тестов.

Результат выполнения модуля is\_match представлен на рисунке 4.4.

```

1  --- Module "is_match" ---
2  OK true
3  OK false
4  OK true
5  OK false
6  OK true
7  OK false
8  OK true
9  OK true
10 OK true
11 OK true
12 OK true
13 OK false
14 OK true
15 OK false
16 OK true
17 OK false
18 OK true
19 OK false
20 OK true
21 OK false
22 All module tests (20) passed

```

#### 4.1.4.5 Модуль matcharr

Модуль проверяет корректность функций match и matches через функции-обёртки assert\_matcharr и assert\_matches соответственно. Обе функции сравнивают возвращённый NULL-терминированный массив строк с эталонным массивом, переданным через вариативный макрос ARR(...).

Тесты функции match проверяют: возврат полного совпадения без групп, возврат групп захвата при простых и вложенных группах, незахватываемые группы (?:...), взаимодействие групп с квантификаторами, а также сложный вложенный шаблон с несколькими уровнями альтераций. Всего 14 тестов.

Результат выполнения модуля matcharr представлен на рисунке 4.5.

```
1  --- Module "matcharr" ---
2  OK ["x"]
3  OK ["x", "x"]
4  OK ["abcdef", "abc", "def"]
5  OK ["abcdef", "def"]
6  OK ["abcdef", "abc"]
7  OK ["aaaaaaaa", "aaaa", "aa"]
8  OK ["aaaaaaaa", "aaaaaaaa", "aaaa", "aa"]
9  OK ["cd"]
10 OK ["efcd", "cd"]
11 OK ["abcdet2b!end", "cdet2b!", "t2b!", "t2", "b!"]
12 OK ["bXcX", "c"]
13 OK ["aaa,b,c,d", "c,"]
14 OK ["word d e word", "word"]
15 OK ["nonnoncapturinggroups"]
16 All module tests (14) passed
```

Рисунок 4.5 – Результат выполнения модуля matcharr

#### 4.1.4.6 Модуль matches

Тесты функции matches проверяют: извлечение всех символов строки, поведение якорей ^ и \$ при глобальном поиске, обработку пустых совпадений (шаблоны .? и .??), поиск нескольких адресов электронной почты в тексте, и многострочный режим (?m) с якорем \$. Всего 10 тестов.

Результат выполнения модуля matches представлен на рисунке 4.6.

```

1  --- Module "matches" ---
2  OK ["a", "b", "c"]
3  OK ["a"]
4  OK ["bc"]
5  OK []
6  OK ["a", "b", "c", ""]
7  OK ["", "", "", ""]
8  OK ["abc@d.ef", "fb@swsu.ru"]
9  OK ["<three>", "<cool>", "<words>"]
10 OK [""]
11 OK ["", "", "", ""]
12 All module tests (10) passed

```

Рисунок 4.6 – Результат выполнения модуля matches

#### 4.1.4.7 Модуль replace

Модуль проверяет корректность функции `re_nreplace` – замены совпадений в буфере фиксированного размера. Функция `assert_re_nreplace` сравнивает содержимое буфера с эталонной строкой и число выполненных замен с эталонным счётчиком. При `ref_cnt = -1` число замен не проверяется (используется для особых тестов).

Тесты проверяют: удаление символов (пустая строка замены), замену с ограничением числа итераций, использование обратных ссылок `$0–$9` и `\n` в строке замены, многострочный режим с якорями `^` и `$`, экранирование метасимволов через замену, и практический сценарий удаления всяких пробелов из кода без нарушения отступов. Два теста помечены как особые (`SPECIAL_TEST`) – замена на символ при нулевом захвате. Всего 15 тестов.

Результат выполнения модуля `replace` представлен на рисунке 4.7.

```

1  --- Module "replace" ---
2  OK "abc", repl_cnt = 5
3  OK "ab c&", repl_cnt = 3
4  OK "abc", repl_cnt = 3
5  OK "Wikipedia encyclopedia. ", repl_cnt = 4
6  OK "abc_def__bla_bla", repl_cnt = 4
7  OK "abc_def_bla_bla", repl_cnt = 3
8  OK "def", repl_cnt = 1
9  OK "{
10     int a = 0;
11
12     char b = 'a';
13
14 }", repl_cnt = 3

```

```

15 OK "nums: 10, 200, 1040...", repl_cnt = 3
16 OK "_-t2b!          b!-", repl_cnt = 1
17 OK "li\\^te\\(r\\)al \\[match\\]\\!", repl_cnt = 6
18 OK "//a = 1;
19 //b = 2;
20 //" , repl_cnt = 3
21 OK "a = 1;", repl_cnt = 1
22 OK "a = 1;
23 b = 2;
24 c = 3;", repl_cnt = 3
25 OK "cd
26 aa
27 //" , repl_cnt = 1
28 * OK ".t.e.s.t.", repl_cnt = 5
29 * OK ".b..c.", repl_cnt = 4
30 All module tests (15) passed

```

Рисунок 4.7 – Результат выполнения модуля replace

#### 4.1.4.8 Итоги тестирования

Итоговая статистика тест-сьюта приведена на рисунке 4.8. Все 237 учитываемых тестов успешно пройдены; 5 особых тестов также дают ожидаемый результат. Суммарное время выполнения составляет 0,022 с.

```

1 -----
2
3 All tests (237) passed, elapsed time: 0.022 s

```

Рисунок 4.8 – Результат тестирования библиотеки

## 4.2 Поисковое приложение

### 4.2.1 Описание классов приложения

#### 4.2.1.1 Класс `TextEditor`

Класс `TextEditor` является главным окном приложения и наследует `QMainWindow`. Методы класса приведены в таблице 4.10.

Таблица 4.10 – Методы класса `TextEditor`

| Метод                                         | Описание                                                                                                                                                  |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>openFile(path)</code>                   | Открывает файл по заданному пути: читает содержимое, определяет тип переноса строк, конвертирует CP1251 в UTF-8 при необходимости, отображает в редакторе |
| <code>saveToFile(path)</code>                 | Сохраняет текст редактора в файл с восстановлением оригинального типа переноса строк (LF/CR/CRLF)                                                         |
| <code>setTabWidth(spaces)</code>              | Устанавливает визуальную ширину табуляции в заданное число пробелов                                                                                       |
| <code>setZoom(percent)</code>                 | Устанавливает масштаб отображения текста; размер шрифта привязывается к ближайшему стандартному значению из списка <code>ZOOM_SIZES</code>                |
| <code>detectLineEnding(raw)</code>            | Определяет тип переноса строк в текстовом файле: CRLF, CR или LF                                                                                          |
| <code>on_action_open_triggered()</code>       | Открывает диалог выбора файла; отклоняет двоичные файлы с сообщением об ошибке                                                                            |
| <code>on_action_save_triggered()</code>       | Сохраняет файл по текущему пути; при отсутствии пути (новый файл) вызывает «Сохранить как»                                                                |
| <code>on_action_save_as_triggered()</code>    | Открывает диалог сохранения файла                                                                                                                         |
| <code>on_action_find_triggered()</code>       | Показывает диалоговое окно поиска; позиционирует его справа от главного окна или по центру экрана при нехватке места                                      |
| <code>on_action_search_dir_triggered()</code> | Открывает диалог выбора директории и запускает <code>SearchDir</code>                                                                                     |

Таблица 4.10 – Методы класса `TextEditor`

| Метод                                                  | Описание                                                     |
|--------------------------------------------------------|--------------------------------------------------------------|
| <code>on_action_zoom_in / out / 100_triggered()</code> | Управление масштабом отображения текста в редакторе          |
| <code>onCursorPositionChanged()</code>                 | Обновляет индикатор строки и столбца в строке состояния      |
| <code>eventFilter(obj, event)</code>                   | Перехватывает <code>Ctrl+Колёсико</code> для масштабирования |

#### 4.2.1.2 Класс `SearchDialog`

Класс `SearchDialog` наследует `QDialog` и реализует немодальный диалог поиска и замены. Методы класса приведены в таблице 4.11.

Таблица 4.11 – Методы класса `SearchDialog`

| Метод                                     | Описание                                                                                                                                                                                                    |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>buildPattern()</code>               | Формирует строку шаблона регулярного выражения: добавляет в неё префикс модификаторов ( <code>?Ims</code> ) на основе состояния чекбоксов учёта регистра, многострочного режима и «.» включает новую строку |
| <code>buildFindFlags()</code>             | Формирует флаги <code>QTextDocument::FindFlags</code> для обычного режима поиска                                                                                                                            |
| <code>getSearchScope(text, offset)</code> | Определяет область поиска: весь текст или только выделенный участок; заполняет <code>search_text</code> и <code>scope_utf8_offset</code>                                                                    |
| <code>validatePattern()</code>            | Компилирует шаблон через <code>compile_re2</code> ; при ошибке отображает форматированное сообщение с выделением позиции ошибки                                                                             |



Таблица 4.11 – Методы класса SearchDialog

| Метод                                      | Описание                                                                                                                                   |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| collectMatches<br>(pattern, text, offset)  | Итеративно вызывает matchpos и заполняет список found_matches всеми совпадениями в области поиска                                          |
| highlightAll()                             | Формирует список QTextEdit::ExtraSelection из found_matches и передаёт в setExtraSelections                                                |
| clearHighlights()                          | Сбрасывает список совпадений и снимает подсветку                                                                                           |
| findNext_regex(wrap)                       | Поиск следующего совпадения по регулярному выражению с возможным циклическим переходом; переводит курсор редактора на найденное совпадение |
| findNext_normal(wrap)                      | Поиск следующего совпадения в обычном режиме через QTextDocument::find с учётом области поиска                                             |
| utf8ToQcharPos(utf8, pos)                  | Конвертирует байтовое смещение UTF-8 в позицию символа Qt                                                                                  |
| showStatus(html) /<br>clearStatus()        | Показывает/скрывает строку состояния в диалоге                                                                                             |
| on_btn_find_next_clicked()                 | Выполняет поиск следующего совпадения                                                                                                      |
| on_btn_count_clicked()                     | Подсчитывает число совпадений и отображает в строке состояния                                                                              |
| on_btn_replace_clicked()                   | Заменяет текущее выделенное совпадение через re_dupreplace с limit=1                                                                       |
| on_btn_replace_all<br>_highlight_clicked() | В режиме замены – заменяет все совпадения; в режиме поиска – подсвечивает в тексте все совпадения                                          |

Таблица 4.11 – Методы класса SearchDialog

| Метод                           | Описание                                                               |
|---------------------------------|------------------------------------------------------------------------|
| on_btn_toggle_replace_clicked() | Переключает видимость панели замены; меняет текст кнопки-переключателя |

#### 4.2.1.3 Класс SearchDir

Класс SearchDir наследует QDialog и реализует диалоговое окно рекурсивного многопоточного поиска по файлам выбранной директории. Методы класса приведены в таблице 4.12.

Таблица 4.12 – Методы класса SearchDir

| Метод                                         | Описание                                                                                                                          |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| startSearch()                                 | Компилирует шаблон, формирует список фильтров расширений, создаёт QThread и SearchController, соединяет сигналы и запускает поток |
| stopSearch()                                  | Устанавливает флаг остановки в SearchController через requestStop()                                                               |
| cleanupThread()                               | Завершает и удаляет рабочий поток после окончания поиска                                                                          |
| onMatchFound(result)                          | Добавляет дочерний узел совпадения в дерево результатов; при необходимости создаёт родительский узел файла                        |
| onFileProcessed<br>(processed, total)         | Обновляет прогресс-бар                                                                                                            |
| onSearchFinished<br>(files, matches, stopped) | Скрывает прогресс-бар, обновляет счётчики в узлах файлов, отображает итоговый статус                                              |

Таблица 4.12 – Методы класса SearchDir

| Метод                                        | Описание                                                                                          |
|----------------------------------------------|---------------------------------------------------------------------------------------------------|
| onTreeWidgetItemDoubleClicked<br>(item, col) | Открывает файл в редакторе через<br>TextEdit::openFile и переводит курсор на строку<br>совпадения |

#### 4.2.1.4 Классы SearchController и FileSearchWorker

Класс SearchController наследует QObject и выполняется в отдельном потоке. Он обходит директорию через QDirIterator и создаёт объекты FileSearchWorker – по одному на каждый файл. Воркеры помещаются в локальный QThreadPool; при многопоточном режиме (chk\_multithread) пул использует все доступные ядра процессора, при однопоточном – одно. Методы классов приведены в таблице 4.13.

Таблица 4.13 – Методы классов SearchController и FileSearchWorker

| Метод                               | Описание                                                                                                                                         |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| SearchController::setup(...)        | Инициализирует параметры поиска:<br>директорию, фильтры по расширениям, флаги<br>режима поиска, скомпилированный объект<br>регулярного выражения |
| SearchController::run()             | Вычисляет суммарный размер файлов, обходит<br>директорию, создаёт и запускает воркеры,<br>ожидает их завершения через QEventLoop                 |
| SearchController<br>::requestStop() | Устанавливает атомарный флаг остановки<br>stop_flag                                                                                              |

Таблица 4.13 – Методы классов SearchController и FileSearchWorker

| Метод                                     | Описание                                                                                                                                                                            |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FileSearchWorker::run()                   | Читает файл, проверяет на двоичность по наличию нулевых байтов в его первых 8 КБ, конвертирует CP1251 в UTF-8, итеративно ищет совпадения и испускает сигнал matchFound для каждого |
| FileSearchWorker<br>::isBinary(data)      | Проверяет буфер на наличие нулевых байтов                                                                                                                                           |
| FileSearchWorker<br>::extractContext(...) | Извлекает фрагмент текста вокруг совпадения для отображения в дереве результатов                                                                                                    |

## 4.2.2 Тестирование поискового приложения

Тестирование поискового приложения проводится вручную по сценариям, охватывающим позитивные, негативные и граничные случаи. Для каждого сценария приведены входные данные, ожидаемый результат и скриншот с результатом.

### 4.2.2.1 Сценарий 1. Открытие текстового файла

Входные данные: текстовый файл в кодировке UTF-8 с содержимым:

|   |               |
|---|---------------|
| 1 | Строка первая |
| 2 | Строка вторая |
| 3 | Строка третья |

Ожидаемый результат: файл открывается, содержимое отображается в редакторе, в строке состояния показаны «Стр: 1 Стб: 1», тип переноса строк «LF» и масштаб «100%», заголовок окна содержит имя файла.

Результат тестирования представлен на рисунке 4.1.

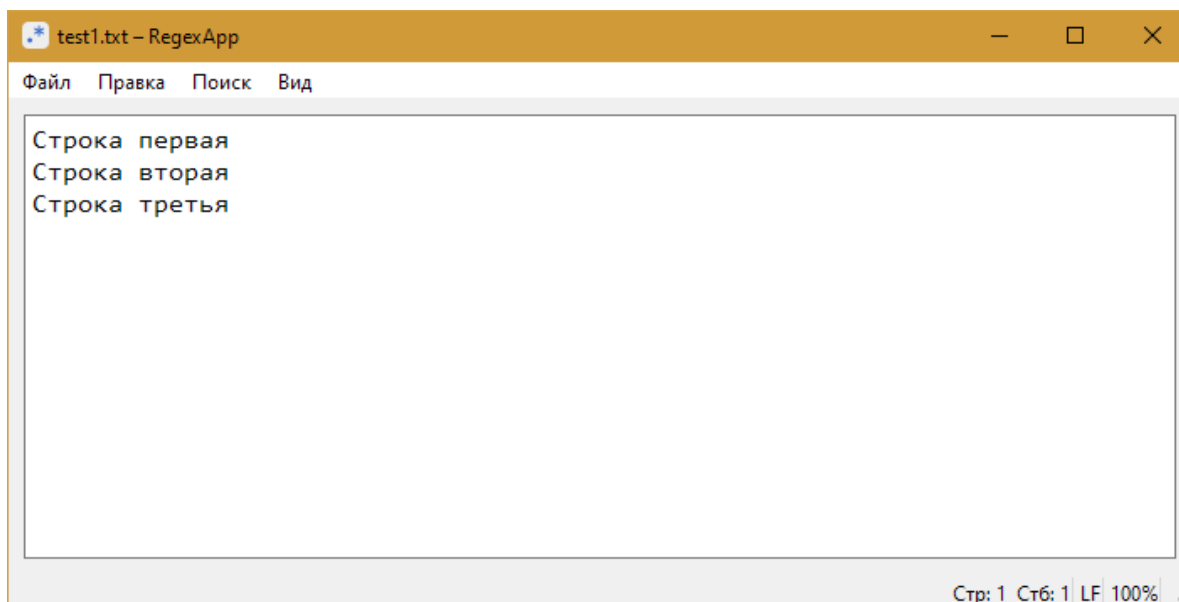


Рисунок 4.1 – Открытие текстового файла

#### 4.2.2.2 Сценарий 2. Открытие файла в кодировке CP1251

Входные данные: файл .txt в кодировке Windows-1251 с кириллическим текстом.

Ожидаемый результат: кириллица отображается корректно (автоматическая конвертация в UTF-8); в строке состояния отображается тип переноса строк (CRLF).

Результат тестирования представлен на рисунке 4.2.

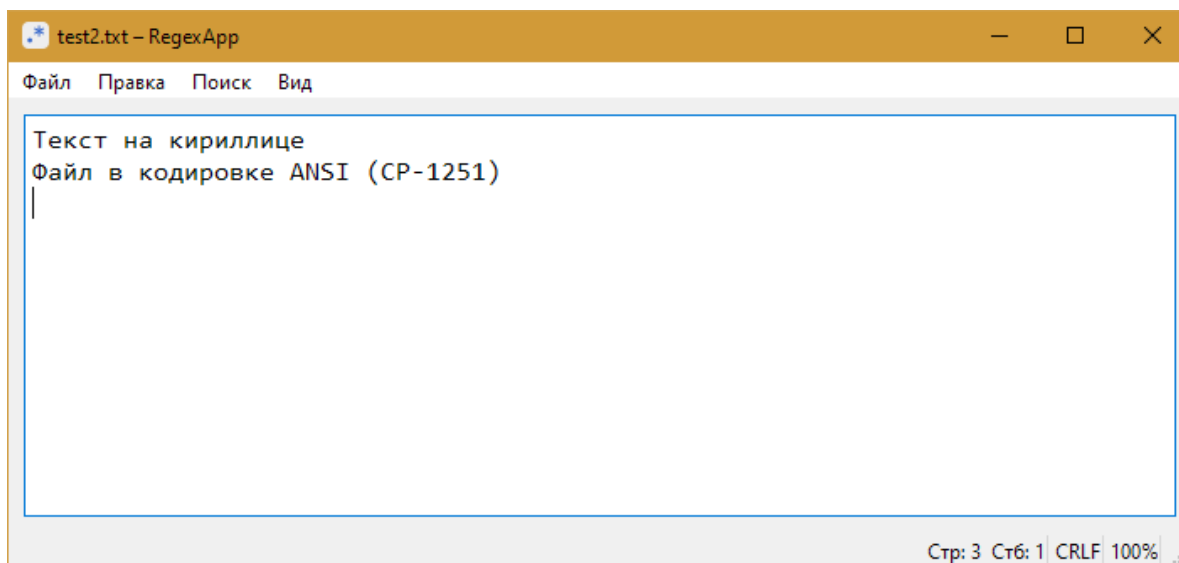


Рисунок 4.2 – Открытие файла в кодировке CP1251

#### 4.2.2.3 Сценарий 3. Попытка открытия двоичного файла

Входные данные: двоичный файл (например, .exe, .png или .bin).

Ожидаемый результат: появляется диалог с сообщением «Файл является двоичным и не может быть открыт в текстовом редакторе»; содержимое редактора не изменяется.

Результат тестирования представлен на рисунке 4.3.

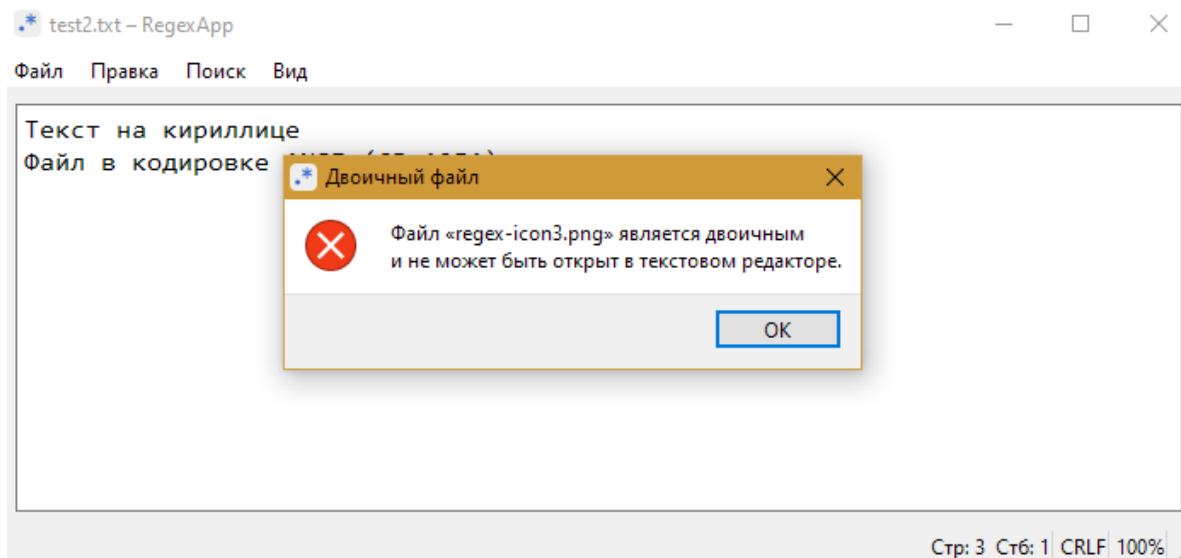


Рисунок 4.3 – Попытка открытия двоичного файла

#### 4.2.2.4 Сценарий 4. Поиск по регулярному выражению с подсветкой

Входные данные: открытый файл со следующим содержимым:

```
1 email: user@example.com
2 phone: +79161234567
3 email: admin@test.org
```

Шаблон: `[\w.+]+@[\\w-]+\.[\\w.]+`, режим – регулярное выражение, нажата кнопка «Подсветить».

Ожидаемый результат: оба адреса электронной почты подсвечены; в строке состояния отображается «Всего 2 совпадений в файле».

Результат тестирования представлен на рисунке 4.4.

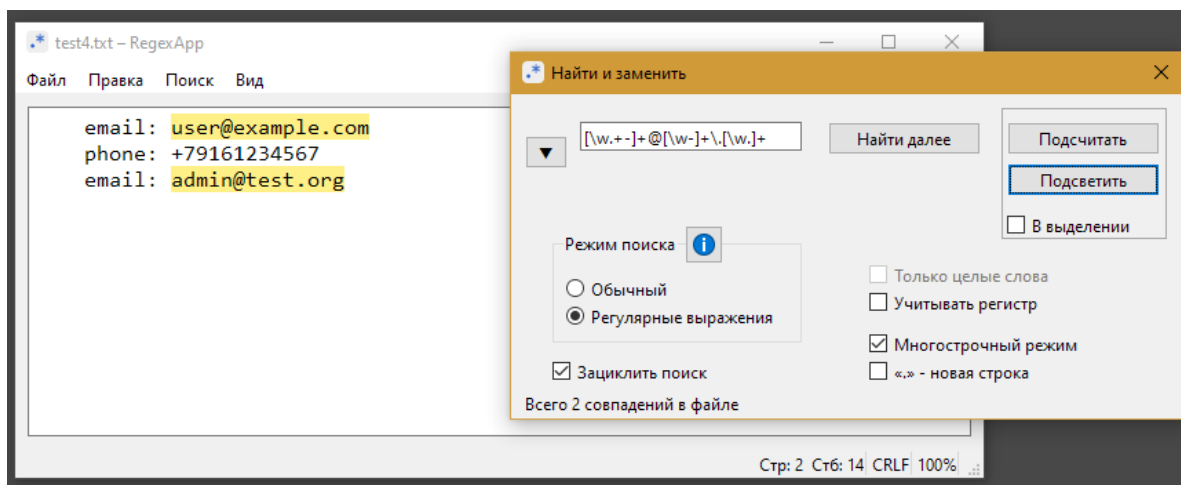


Рисунок 4.4 – Поиск по регулярному выражению с подсветкой

#### 4.2.2.5 Сценарий 5. Поиск с некорректным шаблоном

Входные данные: шаблон `.[\w-z]`, режим – регулярное выражение.

Ожидаемый результат: в строке состояния диалога отображается сообщение об ошибке красным цветом с подчёркиванием символа, вызвавшего ошибку (дефис, обозначающий диапазон символов в наборе символов); поиск не выполняется.

Результат тестирования представлен на рисунке 4.5.

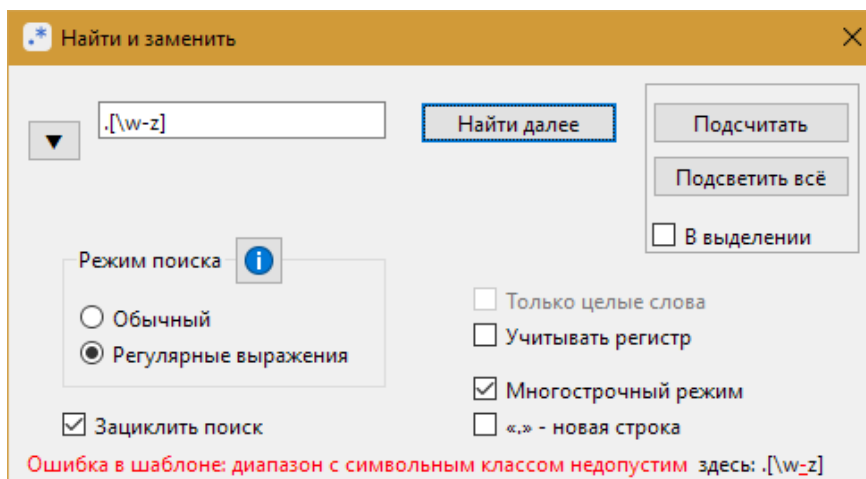


Рисунок 4.5 – Попытка поиска с некорректным шаблоном

#### 4.2.2.6 Сценарий 6. Некорректный шаблон с ошибкой в недостающем символе

Входные данные: шаблон (abc с ошибкой в недостающем символе (после текста шаблона)), режим – регулярное выражение.

Ожидаемый результат: в строке состояния диалога отображается сообщение об ошибке красным цветом с подчёркиванием места возникновения ошибки (нуль-терминатор шаблона – отсутствует закрывающая скобка); поиск не выполняется.

Результат тестирования представлен на рисунке 4.6.

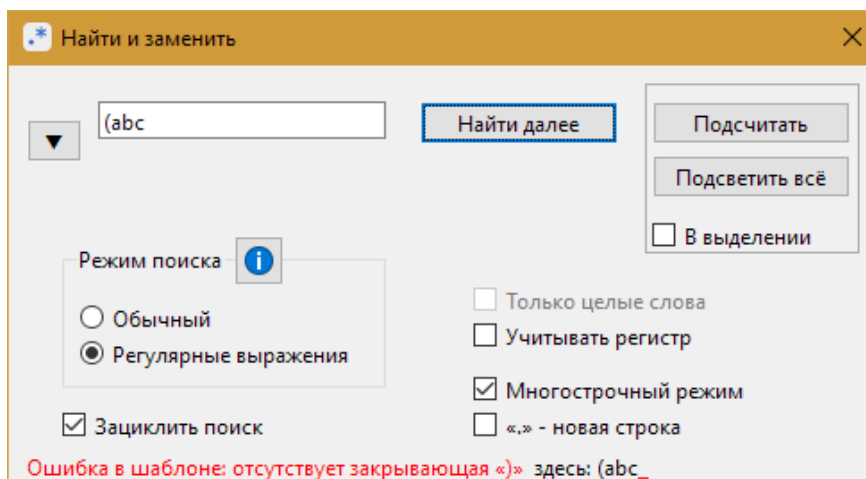


Рисунок 4.6 – Шаблон с ошибкой в недостающем символе

#### 4.2.2.7 Сценарий 7. Замена всех совпадений с обратными ссылками

Входные данные: открыт файл:

|   |                  |
|---|------------------|
| 1 | Дата: 15.03.2023 |
| 2 | Срок: 31.12.2025 |

Шаблон: `(\d{2})\.\(\d{2})\.\(\d{4})`, строка замены: `$3-$2-$1`, нажата кнопка «Заменить всё».

Ожидаемый результат: содержимое редактора принимает вид:

|   |                  |
|---|------------------|
| 1 | Дата: 2023-03-15 |
| 2 | Срок: 2025-12-31 |

В строке состояния отображается «Выполнено 2 замен».

Результат тестирования представлен на рисунке 4.7.



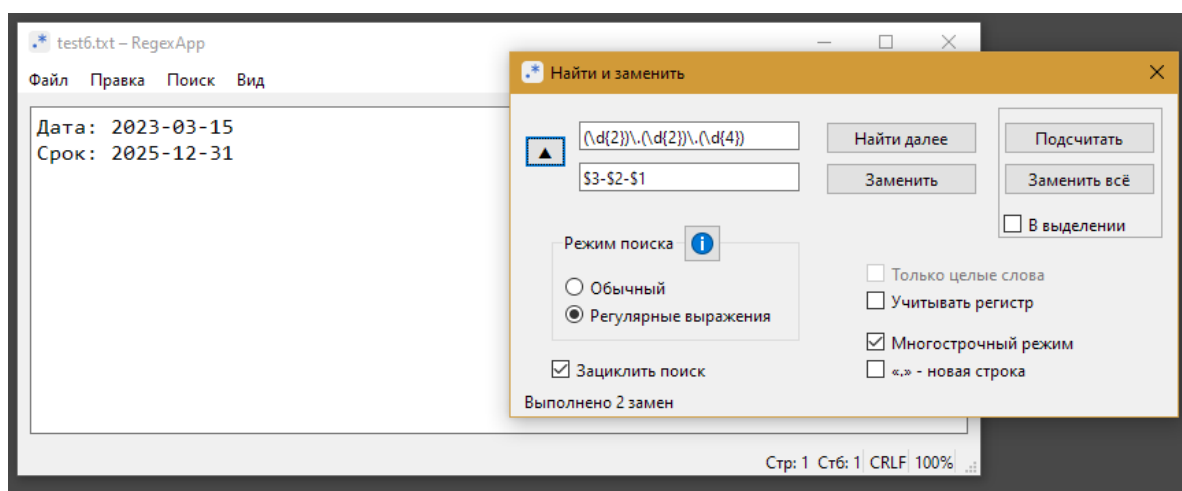


Рисунок 4.7 – Содержимое редактора после выполнения замены

#### 4.2.2.8 Сценарий 8. Поиск с циклическим переходом

Входные данные: файл с несколькими совпадениями, курсор на последнем совпадении, включена галочка «Зациклить поиск».

Ожидаемый результат: при нажатии «Найти далее» поиск переходит к первому совпадению; в строке состояния отображается «Достигнут конец, поиск продолжен с начала файла».

Результат тестирования представлен на рисунке 4.8.

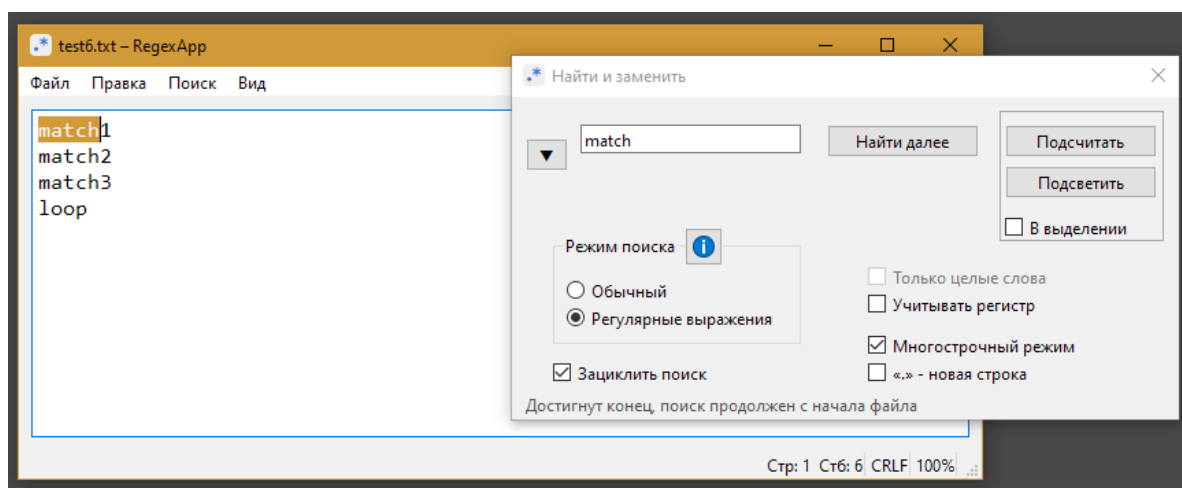


Рисунок 4.8 – Выделение после циклического перехода

#### 4.2.2.9 Сценарий 9. Поиск в выделенной области

Входные данные: выделен фрагмент текста, включена галочка «Только в выделении», задан шаблон.

Ожидаемый результат: подсветка и подсчёт совпадений выполняются только внутри выделенного фрагмента; символы за пределами выделения не затрагиваются.

Результат тестирования представлен на рисунке 4.9.

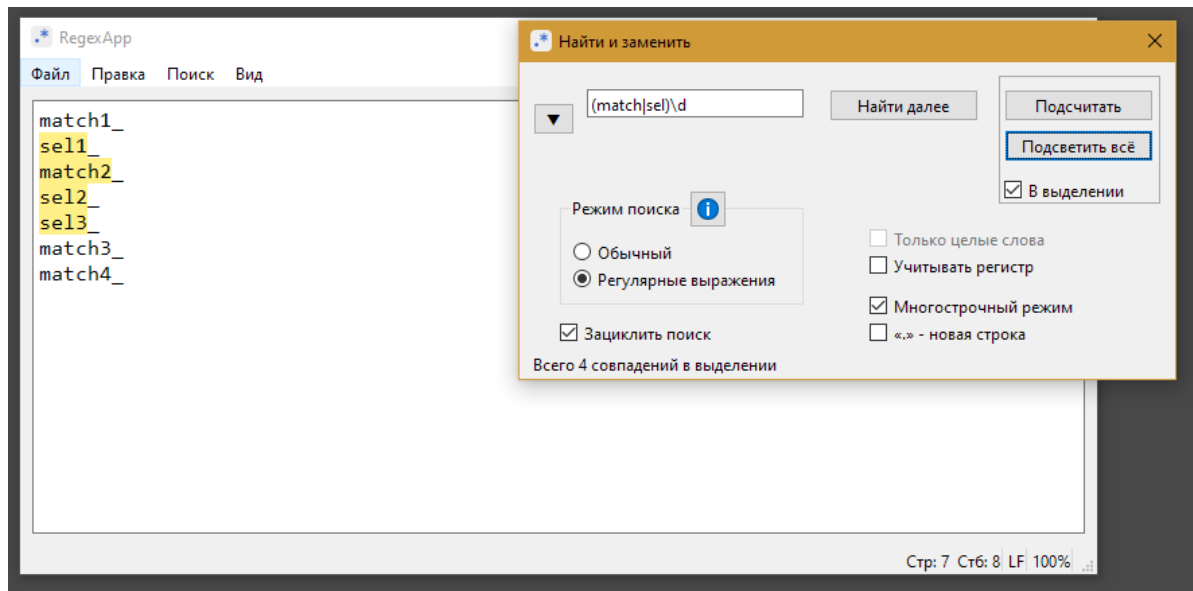


Рисунок 4.9 – Поиск в выделении

#### 4.2.2.10 Сценарий 10. Рекурсивный поиск по директории

Входные данные: директория с файлами:

```
1 file1.txt (содержит "TODO: fix bug")
2 subdir/file2.cpp (содержит "TODO: refactor")
3 subdir/image.png (двоичный файл)
```

Шаблон: TODO:.\*, фильтр по расширениям: \*, рекурсивный режим включён.

Ожидаемый результат: в дереве результатов отображаются два файла с одним совпадением каждый; файл image.png пропущен как двоичный; при двойном щелчке на совпадении файл открывается в редакторе с переходом на нужную строку.

Результат тестирования представлен на рисунке 4.10.

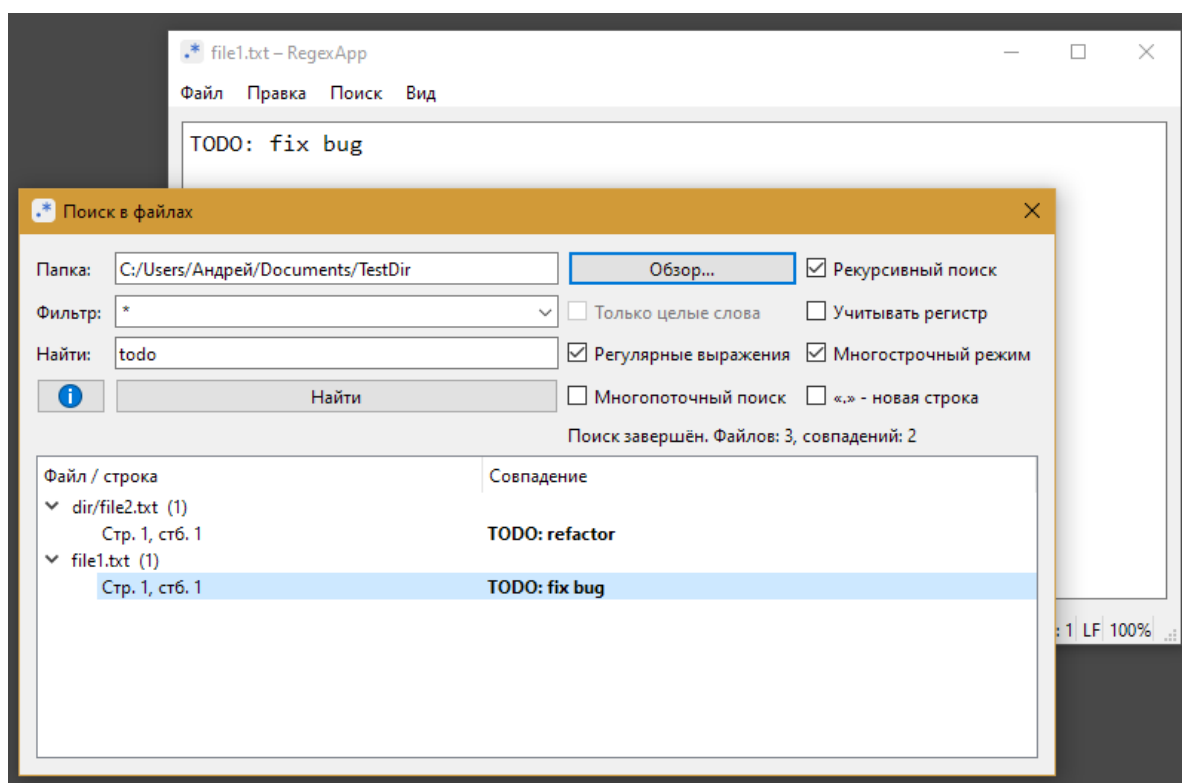


Рисунок 4.10 – Результаты рекурсивного поиска

#### 4.2.2.11 Сценарий 11. Прерывание поиска по директории

Входные данные: директория с большим числом файлов (более 1000), поиск запущен.

Ожидаемый результат: при нажатии кнопки «Прервать поиск» поиск останавливается; кнопка возвращается в состояние «Найти»; в строке состояния отображается «Поиск прерван»; уже найденные результаты остаются в дереве.

Результат тестирования представлен на рисунке 4.11.

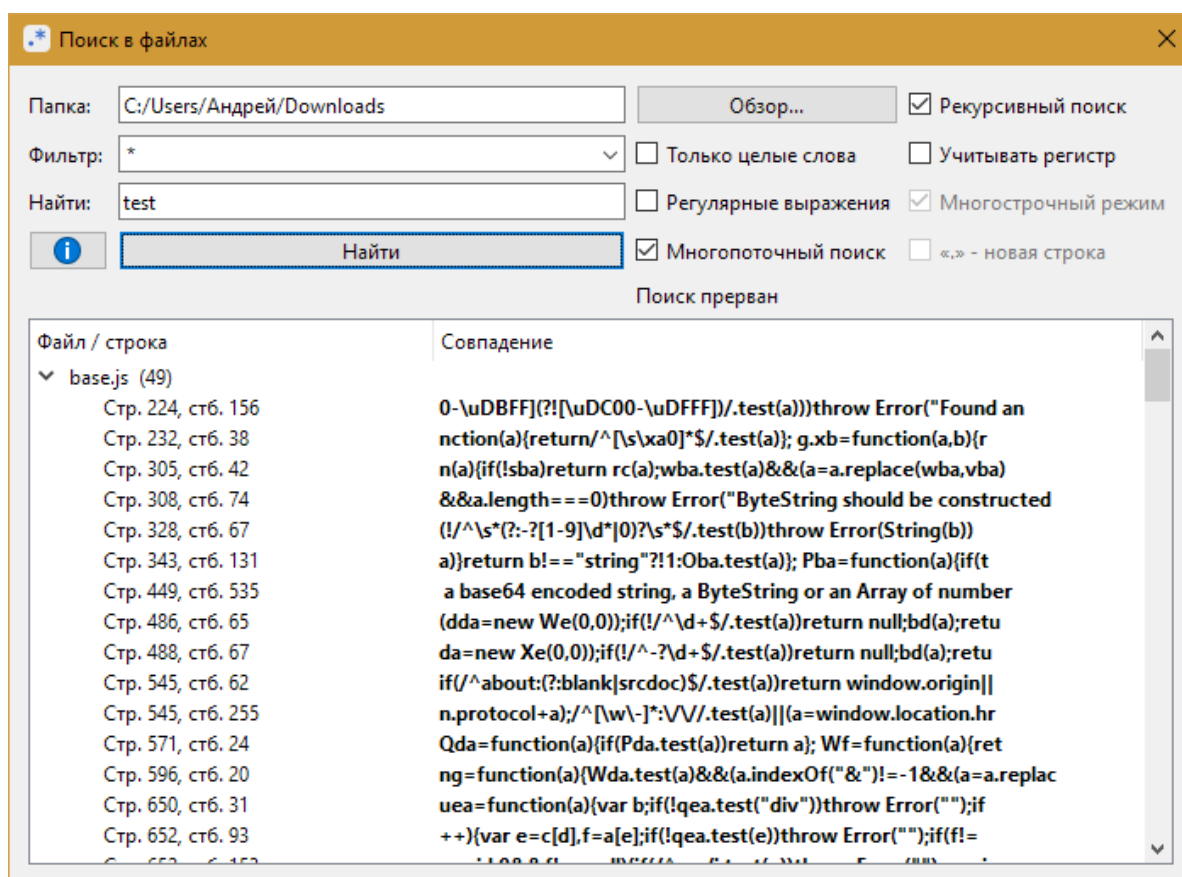


Рисунок 4.11 – Прерывание поиска по директории

#### 4.2.2.12 Сценарий 12. Граничный случай: совпадение нулевой длины

Входные данные: шаблон соответствует захвату нулевой длины (например, .??).

Ожидаемый результат: в строке состояния отображается «Совпадение нулевой длины – выделен ближайший символ»; ошибок не возникает.

Результат тестирования представлен на рисунке 4.12.

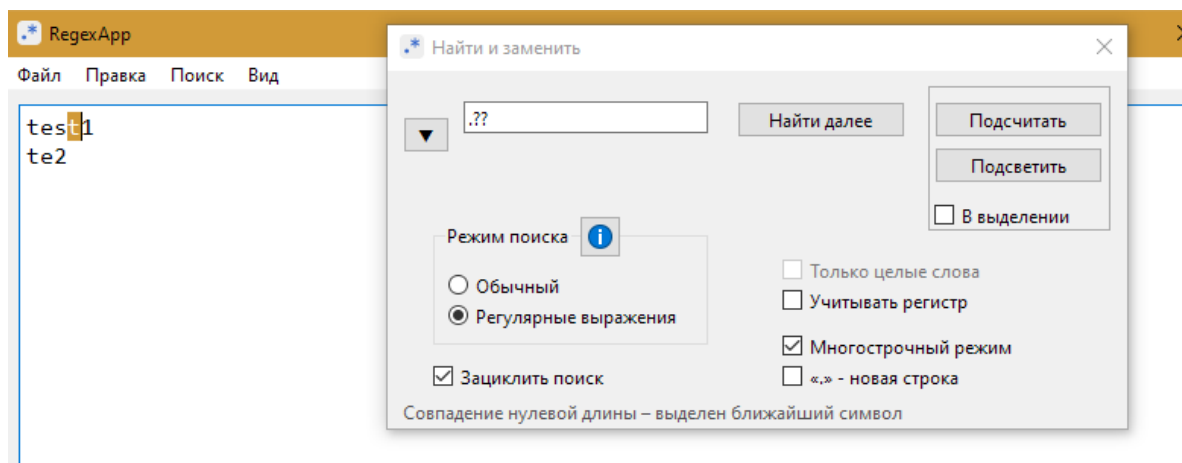


Рисунок 4.12 – Обработка совпадения нулевой длины

#### 4.2.2.13 Сценарий 13. Граничный случай: шаблон совпадает со всем текстом

Входные данные: шаблон `(?s).+` применяется к многострочному тексту, «Многострочный режим» отключён, режим «"." – новая строка» включается через модификатор в составе строки шаблона.

Ожидаемый результат: всё содержимое редактора подсвечивается как одно совпадение; подсчёт возвращает 1.

Результат тестирования представлен на рисунке 4.13.

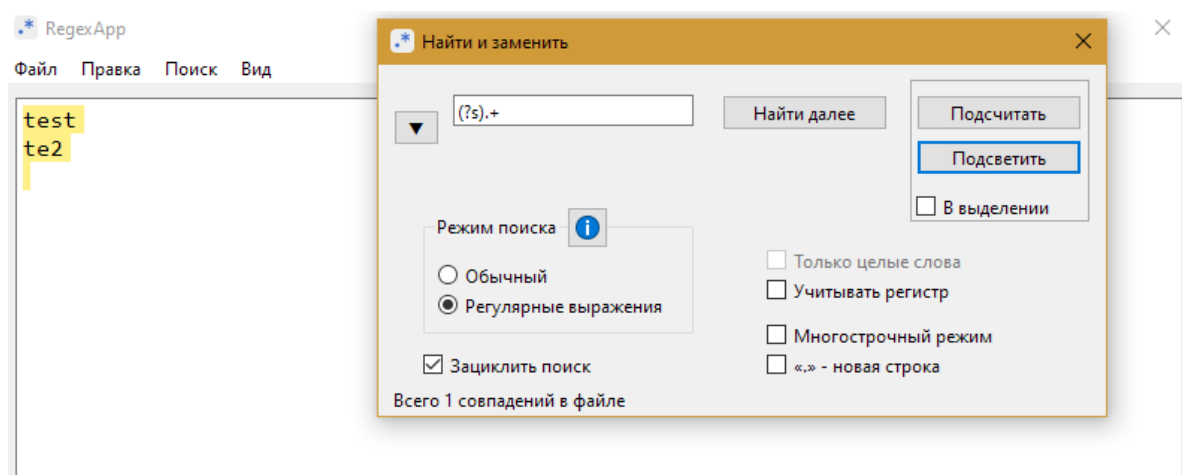


Рисунок 4.13 – Обработка шаблона, захватывающего весь многострочный текст

## ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана программно-информационная поисковая система с графическим интерфейсом, состоящая из библиотеки регулярных выражений на языке Си и десктопного поискового приложения на C++ с использованием фреймворка Qt.

Разработанная библиотека регулярных выражений представляет собой полностью функциональный однофайловый модуль на языке Си с переносимым C API, поддерживающий широкий синтаксис шаблонов: метасимволы, символьные классы, группы захвата, обратные ссылки, альтернатию, ленивые и жадные квантификаторы, модификаторы регулярных выражений, в том числе регистронезависимый поиск с поддержкой кириллицы в кодировке UTF-8. Поисковое приложение предоставляет пользователю текстовый редактор с подсветкой совпадений, интерактивный диалог поиска и замены, а также модуль рекурсивного поиска по файлам директории с отображением результатов и навигацией к найденным фрагментам.

Основные результаты работы:

1. Изучены методы реализации обработчиков регулярных выражений (интерпретация синтаксического дерева, симуляция конечного автомата, рекурсивный спуск с возвратами), язык Си и принципы проектирования C API, фреймворк Qt.
2. Разработана концептуальная модель библиотеки регулярных выражений. Определены требования к библиотеке, её C API и поисковому приложению.
3. Спроектирована программная система библиотеки регулярных выражений: определены внутренние структуры данных, формальная грамматика поддерживаемого синтаксиса шаблонов, алгоритмы компиляции и сопоставления.

4. Спроектирован переносимый C API библиотеки, обеспечивающий её интеграцию в программы на Си-подобных языках, в том числе в C++-проекты через блок extern "C".

5. Реализована библиотека регулярных выражений средствами языка Си. Разработан встроенный набор автоматизированных тестов, охватывающий все семейства экспортируемых функций.

6. Спроектирована программная система поискового приложения: определена архитектура модулей, схема интеграции библиотеки, механизмы взаимодействия компонентов.

7. Реализовано поисковое приложение на языке C++ с использованием фреймворка Qt, включающее текстовый редактор, диалог поиска и замены и модуль рекурсивного поиска по директории.

Все требования, объявленные в техническом задании, были полностью реализованы, все задачи, поставленные в начале разработки проекта, были также решены.

Таким образом, цель работы достигнута.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Фридл, Дж. Регулярные выражения / Дж. Фридл ; пер. с англ. А. Киселева. – 3-е изд. – Санкт-Петербург : Символ-Плюс, 2018. – 608 с. – ISBN 978-5-93286-215-5. – Текст : непосредственный.
2. Магазов, С. С. Теория формальных языков : Регулярные языки : учебно-методическое пособие / С. С. Магазов. – Москва : Издательство МГТУ им. Н. Э. Баумана, 2019. – 52 с. – ISBN 978-5-7038-5273-6. – Текст : непосредственный.
3. Малявко, А. А. Формальные языки и компиляторы : учебник / А. А. Малявко. – Новосибирск : Новосибирский государственный технический университет (НГТУ), 2014. – 431 с. – Текст : непосредственный.
4. Керниган, Б. Алгоритмические трюки для программистов / Б. Керниган, Р. Пайк ; пер. с англ. И. Красикова. – 2-е изд. – Москва : Вильямс, 2014. – 512 с. – ISBN 978-5-8459-1907-6. – Текст : непосредственный.
5. Прата, С. Язык программирования С (лекции и упражнения) / С. Прата ; пер. с англ. – 6-е изд. – Москва : Вильямс, 2015. – 928 с. – ISBN 978-5-8459-1950-2. – Текст : непосредственный.
6. Страуструп, Б. Язык программирования С++ / Б. Страуструп ; пер. с англ. С. Лунина. – 4-е изд. – Москва : Бином, 2015. – 1136 с. – ISBN 978-5-9518-0425-9. – Текст : непосредственный.
7. Осипов, Д. Л. Программирование на С++ в среде Qt / Д. Л. Осипов. – Санкт-Петербург : БХВ-Петербург, 2026. – (Профессиональное программирование). – ISBN 978-5-9775-1993-9. – Текст : непосредственный.
8. Шлее, М. Qt 5.10. Профессиональное программирование на С++ : наиболее полное руководство : для программистов / М. Шлее. – Санкт-Петербург : БХВ-Петербург, 2024. – 1052 с. – ISBN 978-5-9775-3678-3. – Текст : непосредственный.
9. Eng, L. Z. Application Development with Qt Creator : build cross-platform applications and GUIs using Qt 5 and C++ / L. Z. Eng, R. Rischpater.



– 3rd edition. – Birmingham ; Mumbai : Packt Publishing, 2020. – 413 p. – ISBN 978-1-78995-175-2.

10. Qt Documentation : QRegularExpression : [official documentation] / The Qt Company. – URL: <https://doc.qt.io/qt-6/qregularexpression.html> (дата обращения: 13.06.2025).

11. Гагарин, А. П. Современные основы проектирования компьютерных программ : учебное пособие / А. П. Гагарин ; Министерство науки и высшего образования Российской Федерации, Московский авиационный институт (национальный исследовательский университет). – Москва : Изд-во МАИ, 2024. – 158 с. – ISBN 978-5-4316-1190-2. – Текст : непосредственный.

12. Белик, А. Г. Проектирование и архитектура программных систем : учебное пособие / А. Г. Белик, В. Н. Цыганенко. – Омск : ОмГТУ, 2016. – 96 с. – ISBN 978-5-8149-2258-8. – Текст : непосредственный.

13. Вигерс, К. Разработка требований к программному обеспечению : практические приёмы сбора требований и управления ими при разработке программных продуктов : пер. с англ. / К. Вигерс, Д. Битти. – 3-е изд., доп. – Санкт-Петербург : BHV, 2020. – 736 с. – ISBN 978-5-9775-3348-5. – Текст : непосредственный.

14. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин ; пер. с англ. А. Кисилева. – Санкт-Петербург : Питер, 2018. – 351 с. – ISBN 978-5-4461-0772-8. – Текст : непосредственный.

15. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки // Техэксперт : [сайт]. – URL: <https://docs.cntd.ru/document/1200007652> (дата обращения: 07.05.2026).

16. ГОСТ 19.105-78. Единая система программной документации. Общие требования к программным документам // Техэксперт : [сайт]. – URL: <https://docs.cntd.ru/document/1200007655> (дата обращения: 07.05.2026).

17. ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению // Техэксперт : [сайт]. – URL: <https://docs.cntd.ru/document/1200007659> (дата обращения: 07.05.2026).

18. ГОСТ 34.201-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем. – Москва : Стандартинформ, 2020. – Текст : непосредственный.

19. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – Москва : Стандартинформ, 2020. – Текст : непосредственный.

20. Фаулер, М. Рефакторинг. Улучшение существующего кода / М. Фаулер. – Санкт-Петербург : Питер, 2019. – 448 с. – ISBN 978-5-4461-1288-7. – Текст : непосредственный.

21. Lischner, R. Exploring C++20 : The Programmer's Introduction to C++ / R. Lischner. – 3rd edition. – New York : Apress, 2020. – 653 p. – ISBN 978-1-4842-5960-3. – DOI: 10.1007/978-1-4842-5961-0.

22. Bancila, M. Modern C++ Programming Cookbook : Master C++ Core Language and Standard Library Features, with Over 100 Recipes, Updated to C++20 / M. Bancila. – 2nd edition. – Birmingham ; Mumbai : Packt Publishing, 2020. – 750 p. – ISBN 978-1-80020-898-8. – Text : direct.

23. Thompson, K. Regular Expression Search Algorithm / K. Thompson // Communications of the ACM. – 1968. – Vol. 11, no. 6. – P. 419–422. – DOI: 10.1145/363347.363387.

24. Cox, R. Regular Expression Matching Can Be Simple And Fast (but is slow in Java, Perl, PHP, Python, Ruby, ...) / R. Cox // swtch.com : [сайт]. – 2007. – URL: <https://swtch.com/~rsc/regexp/regexp1.html> (дата обращения: 07.05.2026).

25. C++11 Regular-Expressions : [документация] // cppreference.com : [сайт]. – URL: <https://en.cppreference.com/w/cpp/regex> (дата обращения: 07.05.2026).

26. Gough, B. An Introduction to GCC : for the GNU Compilers gcc and g++ / B. Gough, R. M. Stallman. – Revised edition. – Bristol : Network Theory Ltd, 2015. – 144 p. – ISBN 978-0-9541617-9-8. – Text : direct.

## ПРИЛОЖЕНИЕ А

### Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.3.

Рисунок А.1 – Сведения о ВКРБ

## Цель и задачи разработки

Цель настоящей работы – проектирование и реализация программно-информационной поисковой системы с графическим интерфейсом на основе библиотеки регулярных выражений собственной разработки.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и методов реализации движков регулярных выражений;
- разработать концептуальную модель библиотеки регулярных выражений;
- спроектировать программную систему библиотеки регулярных выражений на языке Си;
- спроектировать переносимый C API библиотеки для её интеграции в программы на Си-подобных языках;
- реализовать библиотеку регулярных выражений средствами языка Си;
- спроектировать программную систему поискового приложения с графическим интерфейсом;
- реализовать поисковое приложение на языке C++ с использованием фреймворка Qt.

|                              |                |            |          |
|------------------------------|----------------|------------|----------|
| ВКРБ 2206164.09.03.04.26.020 |                |            |          |
| Автор работы                 | Филиппов И. В. | Надпись    | Дата     |
| Рецензент                    | Степанов А. А. |            |          |
| Оценщик                      | Чепурин А. А.  |            |          |
| Подпись о ВКРБ               |                | Лист 2     | Листов 3 |
| Выполнен и одобрен           |                | 03.04.2016 |          |
| работы                       |                | по заданию |          |

Рисунок А.2 – Цель и задачи разработки

# Синтаксис регулярных выражений

## Метасимволы:

. – любой одиночный символ;  
 ^ – результат должен включать начало строки;  
 \$ – результат должен включать конец строки;  
 \* – предыдущий токен (символ или группа) встречается 0 или более раз;  
 + – предыдущий токен встречается 1 или более раз;  
 ? – предыдущий токен опционален;  
 \*, +, ? – ленивый режим захвата для данного токена;  
 {n} – предыдущий токен повторяется ровно  $n$  раз;  
 {n, m} – предыдущий токен повторяется от  $n$  до  $m$  раз;  
 [abc] – символ равен одному из символов в квадратных скобках;  
 [^abc] – символ равен любому символу, которого нет в квадратных скобках;  
 [abc|def] – соответствует либо шаблону слева вертикальной черты, либо шаблону справа от нее (альтерация);  
 (abc) – определяет группу захвата, а также ограничивает область шаблона, рассматриваемого альтерацией;  
 (? : abc) – определяет не-захватываемую группу и ограничивает область шаблона, рассматриваемого альтерацией;  
 \n – обратная ссылка на захваченную ранее при поиске группу, где  $x$  – номер группы от 1 до 9.

Для буквальной интерпретации метасимволов, перед ними необходимо поставить экранирующий символ: \  
 Сам экранирующий символ также нужно экранировать для буквальной интерпретации.

## Символьные классы:

\d – любая цифра;  
 \D – любой нецифровой символ;  
 \s – пробельный символ;  
 \S – любой непробельный символ;  
 \w – буква латинского алфавита, цифра или символ подчёркивания;  
 \W – любой символ, кроме буквенно-цифрового и знака подчёркивания;  
 \a – буква латинского алфавита;  
 \A – любой символ, кроме латинских букв;  
 \p – символы пунктуации;  
 \P – любой непунктуационный символ;  
 \n – символ новой строки;  
 \t – символ табуляции.

## Модификаторы:

$m$  – многострочный режим;  
 $s$  – метасимвол «.» включает в себя переход на новую строку;  
 $i$  – поиск без учёта регистра.

|                                  |               |            |        |
|----------------------------------|---------------|------------|--------|
| ВКРБ 2206164.09.03.04.26.020     |               |            |        |
| Имя                              | Фамилия И.О.  | Подпись    | Дата   |
| Исполнитель                      | Степанов А.А. |            |        |
| Проверенный                      | Чепурин А.А.  |            |        |
|                                  | Чепурин А.А.  |            |        |
| Подпись о ВКРБ                   |               | Дата       | Лист 3 |
| Выполнен и одобрен руководителем |               | 03.09.2016 |        |

Рисунок А.3 – Синтаксис регулярных выражений

## ПРИЛОЖЕНИЕ Б

### Фрагменты исходного кода программы

#### regex-lib.h

```
1 #ifndef REGEX_LIB_H
2 #define REGEX_LIB_H
3
4 #ifdef __cplusplus
5 extern "C" {
6 #endif
7
8 struct re_chain; // объект регулярного выражения
9
10 // Упрощённая версия вызова функции для парсинга шаблона, при возникновении
    // ошибки также возвращает NULL
11 // Чтобы понять, в чём ошибка, нужно вызвать полную версию функции (
    compile_re2) со всеми параметрами
12 // Её стоит использовать только при уверенности, что шаблон РВ точно не
    // содержит ошибок
13 struct re_chain* compile_re(char* pattern);
14
15 // То же, что и для функции выше, но принимает ещё 2 параметра-указателя,
    // заполняемые при некорректности шаблона:
16 // `errcode` - код ошибки и `errpos` - номер символа в строке шаблона, при
    // разборе которого произошла ошибка
17 // (может указывать на нуль-терминатор строки, если ожидался закрывающий
    // символ).
18 // Допустимо передать NULL-указатели, тогда эти поля не будут заполнены.
19 struct re_chain* compile_re2(char* pattern, int* errcode, int* errpos);
20
21 // Освобождение памяти, занимаемой объектом регулярного выражения, созданным
    // функциями compile_re
22 void free_re(struct re_chain* re_obj);
23
24 // Далее функции предоставляются парами: первая версия принимает на вход
    // строку шаблона регулярного выражения,
25 // версия obj_* вместо неё принимает откомпилированный объект регулярного
    // выражения
26
27 // Возвращает данные о первой захваченной подстроке из исходного текста и
    // строки шаблона регулярного выражения:
28 // При успешном захвате возвращает номер первого символа подстроки
29 // и записывает ее длину в переменную по указателю capture_data, если он не
    // равен NULL
30 // При неудачном захвате возвращает -1
31 // При некорректности шаблона возвращает отрицательное число меньше -1 -- код
    // ошибки,
32 // а также записывает номер символа в шаблоне, при разборе которого произошла
    // ошибка, в capture_data
33 // Если по указателю capture_data передать положительное число, то поиск в
    // строке text начнётся с этой позиции
34 int matchpos(char* pattern, char* text, int* capture_data);
35 int obj_matchpos(struct re_chain* re_obj, char* text, int* capture_data);
```

```

36
37 // Выполняет валидацию строки с помощью строки с шаблоном регулярного
    выражения:
38 // при успехе возвращает 1, при неудаче - 0,
39 // при некорректности шаблона возвращает отрицательное число меньше -1 (код
    ошибки)
40 int is_match(char* pattern, char* text);
41 int obj_is_match(struct re_chain* re_obj, char* text);
42
43 // Возвращает NULL-терминированный массив malloc-строк аналогично функции
    match() в JavaScript:
44 // первый элемент - главный захват, затем - его группы захвата
45 // при отсутствии захватов возвращает пустой массив, при некорректности
    шаблона - NULL
46 char** match(char* pattern, char* text);
47 char** obj_match(struct re_chain* re_obj, char* text);
48
49 // Возвращает NULL-терминированный массив malloc-строк - все захваченные
    подстроки регулярного выражения
50 // (аналогично функции match() с флагом 'g' в JavaScript)
51 // при отсутствии захватов возвращает пустой массив, при некорректности
    шаблона - NULL
52 char** matches(char* pattern, char* text);
53 char** obj_matches(struct re_chain* re_obj, char* text);
54
55 // Выполняет замену регулярным выражением pattern подстрок repl в строке text
    , максимум limit раз
56 // Если limit равен нулю - неограниченного число раз
57 // Функция возвращает результат в переданный буфер dest размером n байт
58 // Возвращает число выполненных замен или -1 при окончании места в буфере (то
    есть недостоверном результате)
59 // при некорректности шаблона возвращает отрицательное число меньше -1 (код
    ошибки)
60 int re_nreplace(char* pattern, char* repl, char* text, char* dest, int n, int
    limit);
61 int obj_re_nreplace(struct re_chain* re_obj, char* repl, char* text, char*
    dest, int n, int limit);
62
63 // Выполняет замену регулярным выражением pattern подстрок repl в строке text
    , максимум limit раз
64 // Если limit равен нулю - неограниченное число раз
65 // Возвращает malloc-строку с результатом или NULL при некорректном шаблоне
66 // Записывает по указателю repl_cnt число выполненных замен или отрицательный
    код ошибки
67 char* re_dupreplace(char* pattern, char* repl, char* text, int limit, int*
    repl_cnt);
68 char* obj_re_dupreplace(struct re_chain* re_obj, char* repl, char* text, int
    limit, int* repl_cnt);
69
70 enum re_error_type {
71     RE_OK                = 0,
72     BAD_CARET            = -100, // неэкранированный ^ не в начале
73     BAD_DOLLAR           = -99,  // неэкранированный $ не в конце
74     PREV_NOT_QUANTIF     = -98,  // предыдущий токен не квантифицируем

```



```

75     EMPTY_SQUARES      = -97,    // пустой набор символов []
76     MISSING_LSQUARE    = -96,    // отсутствует открывающая [
77     MISSING_RSQUARE    = -95,    // отсутствует закрывающая ]
78     RANGE_WITH_CHARCLASS = -94,   // диапазон с символьным классом
79     BAD_ESC_CHAR       = -93,    // отсутствует символ после '\'
80     BAD_INSIDE_CURLY    = -92,    // недопустимые символы внутри {}
81     BAD_CURLY_ORDER     = -91,    // неверный порядок чисел в {}
82     HUGE_CURLY_RANGE    = -90,    // диапазон квантификатора слишком велик
83     MISSING_LPAREN      = -89,    // отсутствует открывающая (
84     MISSING_RPAREN      = -88,    // отсутствует закрывающая )
85     BAD_SQUARE_ORDER    = -87,    // неверный порядок символов в диапазоне
86     BAD_CHARCLASS       = -86,    // некорректный символьный класс
87     INVALID_FLAG        = -85,    // неверный модификатор PB
88 };
89
90 #ifdef __cplusplus
91 }
92 #endif
93
94 #endif // REGEX_LIB_H

```

## regex-lib.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4  #include <string.h>
5  #include <limits.h>
6  #include <setjmp.h>
7  #include <ctype.h> // isalpha
8
9  #define ESC_CHAR '\\' // экранирующий символ
10
11 const char wildcards[] = "^$.*+?[]{}|()"; // метасимволы кроме ESC_CHAR
12 const char quantifiers[] = "*+?{";
13 const char char_classes[] = "dwsapDWSAP";
14 const char esc_seqs[] = "rnt";
15 #define IS_WILDCARD(elem) (elem ? strchr(wildcards, elem) != NULL : false)
16 #define IS_QUANTIF(elem) (elem ? strchr(quantifiers, elem) != NULL : false)
17 #define IS_CHARCLASS(elem) (elem ? strchr(char_classes, elem) != NULL :
    false)
18 #define IS_CLASS(elem) (elem ? (strchr(char_classes, elem) != NULL || strchr(
    esc_seqs, elem) != NULL) : false)
19
20 #define STRICT_CHARCLASSES // не разрешать экранирование букв, кроме
    используемых в символьных классах (как в PB в др. языках)
21
22 enum block_type {
23     PLAIN, // символы воспринимаются буквально, rep_* игнорируются
24     ONE, // 1 символ, повторяющийся rep_* раз
25     ANY, // .
26     CHARCLASS, // символьный класс, хранит символ-описание после ESC_CHAR
27     GROUP, // [...]
28     REVGROUP, // [^...]
29     CHAIN, // группа в круглых скобках, повторяющаяся rep_* раз

```

```

30     BACKREF // обратная ссылка на захваченную группу вида \1
31 };
32
33 enum re_mods {
34     RE_ROOT = 1 << 0, // флаг корня PB (задан всегда)
35     RE_M = 1 << 1, // ^ $ означают начало и конец каждой строки
36     RE_S = 1 << 2, // . - новая строка
37     RE_I = 1 << 3, // поиск без учёта регистра
38     RE_IRU = 1 << 4, // без учёта регистра с поддержкой кириллицы, формат
        строки строго UTF-8
39 };
40
41 union re_data {
42     char* text; // для PLAIN, GROUP, REVGROUP - указатель на malloc-строку
43     char sym; // для ONE, CHARCLASS - сам символ
44     struct re_chain* subchain; // для CHAIN - указатель на цепочку блоков в
        группе внутри круглых скобок
45     int cap_num; // для BACKREF - № группы от 1 до 9
46 };
47
48 // список цепочек блоков PB (для альтерации)
49 // порядок рассмотрения не важен (можно добавлять в конец), достаточно
    разобрать одну из них
50 // каждая цепочка - список блоков, все из которых необходимо разобрать по
    очереди для успеха
51 struct re_chain {
52     struct re_chain* next; // следующий элемент цепочки
53     bool incl_start; // могут быть в каждой альтерации (как в PB в других
        языках)
54     bool incl_end;
55     struct re_block* head; // начало списка блоков (для прохода с начала)
56     struct re_block* tail; // конец списка (для быстрого добавления)
57     int bl_count; // число блоков в списке (в текущей альтерации)
58     int ch_data; // для главной цепочки: число групп захвата в PB, иначе -
        № группы, к которой отн звено
59     int flags; // для главной цепочки: модификаторы PB (не бывает 0), иначе -
        0 (признак корневой цепочки)
60 };
61
62 struct re_block {
63     enum block_type type;
64     union re_data data;
65     int rep_min; // мин число повторов блока (* - 0, + - 1, ? - 0, {3-5} - 3)
66     int rep_max; // макс число повторов блока (* - беск (INT_MAX), + - беск,
        ? - 1, {3-5} - 5)
67     bool lazy; // ленивый режим захвата вместо жадного
68     struct re_block* next;
69 };
70
71 struct re_matches {
72     int* starts; // массивы длиной в capacity
73     int* lengths;
74     int capacity; // равна кол-ву цепочек в функции match(), иначе равна 10 (
        макс № обратной ссылки)

```

```

75     int flags; // модификаторы PB
76 };
77
78 enum re_error_type {
79     RE_OK = 0,
80     BAD_CARET = -100,
81     BAD_DOLLAR, // = -99 и т.д.
82     PREV_NOT_QUANTIF,
83     EMPTY_SQUARES,
84     MISSING_LSQUARE,
85     MISSING_RSQUARE,
86     RANGE_WITH_CHARCLASS,
87     BAD_ESC_CHAR,
88     BAD_INSIDE_CURLY,
89     BAD_CURLY_ORDER,
90     HUGE_CURLY_RANGE,
91     MISSING_LPAREN,
92     MISSING_RPAREN,
93     BAD_SQUARE_ORDER,
94     BAD_CHARCLASS,
95     INVALID_FLAG,
96 };
97 // вызов "исключения" (прыжок на jenv) при возникновении ошибки в шаблоне PB
   во время разбора
98 #define RAISE(err_code) { *erp = re_str + i; longjmp(*jenv, err_code); }
99
100 // безопасный аналог strncpy
101 void re_strncpy(char* dst, const char* src, size_t size)
102 {
103     size_t len = 0;
104     while (++len < size && *src)
105         *dst++ = *src++;
106     if (len <= size)
107         *dst = '\0';
108 }
109
110 // Вывод фатальной ошибки в stderr и завершение программы
111 #define RE_ERROR(...) \
112 { \
113     fprintf(stderr, "FATAL ERROR: "); \
114     fprintf(stderr, __VA_ARGS__); \
115     fflush(stderr); \
116     exit(-1); \
117 }
118
119 // Выделение памяти с завершением программы при её исчерпании
120 void* re_malloc(size_t size)
121 {
122     void* ptr = malloc(size);
123     if (ptr)
124         return ptr;
125     else
126         RE_ERROR("Out of memory");
127 }

```

```

128
129 void* re_realloc(void* block, size_t size)
130 {
131     void* ptr = realloc(block, size);
132     if (ptr)
133         return ptr;
134     else
135         RE_ERROR("Out of memory");
136 }
137
138 // Возвращает указатель на n-ый элемент списка блоков или NULL, если индекс
    вне диапазона
139 struct re_block* re_nth(struct re_chain* re, int n)
140 {
141     if (re == NULL || n < 0) return NULL;
142     struct re_block* cur = re->head;
143     int i = 0;
144     while (cur != NULL && i < n) {
145         cur = cur->next;
146         i++;
147     }
148     return cur;
149 }
150
151 // Добавляет новый блок в конец списка блоков РВ
152 void new_block(struct re_chain* re, enum block_type type, union re_data data,
    int rmin, int rmax, bool lazy)
153 {
154     struct re_block* new_bl = re_malloc(sizeof(struct re_block));
155     new_bl->next = NULL;
156     new_bl->type = type;
157     new_bl->data = data;
158     new_bl->rep_min = rmin;
159     new_bl->rep_max = rmax;
160     new_bl->lazy = lazy;
161
162     if (re->tail) {
163         re->tail->next = new_bl;
164         re->tail = new_bl;
165     }
166     else // пустой список
167         re->head = re->tail = new_bl;
168     re->bl_count++;
169 }
170
171 // Обрабатывает квантификатор после текущего токена РВ и создаёт новый блок
172 // Возвращает длину квантификатора (больше 1 для фигурных скобок)
173 // также способна вызывать "исключения", если в ходе распознавания шаблона
    выявлены ошибки
174 int process_quantif(struct re_chain* re, char* re_str, enum block_type type,
    union re_data data, jmp_buf* jenv, char** erp)
175 {
176     int rep_min, rep_max, i;
177     bool lazy = false;

```

```

178     int* curr_rep = &rep_min;
179     bool second_num = false;
180     switch (re_str[0]) {
181         case '*':
182             rep_min = 0;
183             rep_max = INT_MAX;
184             i = 1;
185             break;
186         case '+':
187             rep_min = 1;
188             rep_max = INT_MAX;
189             i = 1;
190             break;
191         case '?':
192             rep_min = 0;
193             rep_max = 1;
194             i = 1;
195             break;
196         case '{':
197             rep_min = 0, rep_max = INT_MAX, i = 1;
198             while ((re_str[i] >= '0' && re_str[i] <= '9') || re_str[i] == ',')
199                 {
200                     if (re_str[i] == ',') {
201                         if (!second_num) {
202                             second_num = true;
203                             curr_rep = &rep_max;
204                             if (re_str[i+1] != '}') // далее указано второе число
205                                 , ограничивающее диапазон
206                                 rep_max = 0;
207                         }
208                     }
209                     else {
210                         if (*curr_rep >= (INT_MAX - (int)(re_str[i] - '0')) / 10)
211                             // защита от переполнения
212                             RAISE(HUGE_CURLY_RANGE);
213                         *curr_rep = *curr_rep * 10 + (int)(re_str[i] - '0');
214                     }
215                     i++;
216                 }
217             if (!second_num)
218                 rep_max = rep_min;
219             if (re_str[i] != '}')
220                 RAISE(BAD_INSIDE_CURLY);
221             if (rep_min > rep_max)
222                 RAISE(BAD_CURLY_ORDER);
223             i++; // + закрывающая скобка
224             break;
225         default:
226             rep_min = 1;
227             rep_max = 1;
228             i = 0;
229     }

```

```

229     if (re_str[i] == '?') {
230         lazy = true;
231         i++;
232     }
233     new_block(re, type, data, rep_min, rep_max, lazy);
234     return i;
235 }
236
237 // отладочный вывод списка блоков PB в консоль
238 void print_re_blocks(struct re_chain* root, int depth)
239 {
240     if (root->flags != 0 && root->flags != RE_ROOT)
241         printf("Global flags: %s%s%s\n", (root->flags & RE_M) ? "m" : "", (
            root->flags & RE_S) ? "s" : "",
242             (root->flags & RE_I) ? "i" : "", (root->flags & RE_IRU) ? "I"
                : "");
243     int alt = 0;
244     for (struct re_chain* re = root; re != NULL; re = re->next) {
245         for (int i = 0; i < depth; i++) printf("    "); // форматирование
            вложенных групп
246         if (root->next) // если в PB несколько альтераций
247             printf("Alt %d, ", alt);
248         printf("Flags: %s%s, %d blocks%s",
249             re->incl_start ? '^' : '_', re->incl_end ? '$' : '_', re->
                bl_count, re->bl_count > 0 ? ": " : "");
250         struct re_block* cur = re->head;
251         char repeats[32];
252         while (cur) {
253             if (cur->rep_max == INT_MAX)
254                 snprintf(repeats, sizeof(repeats), "{%s%d-INF}", cur->lazy ?
                    "~" : "", cur->rep_min);
255             else if (cur->rep_min == cur->rep_max)
256                 snprintf(repeats, sizeof(repeats), "{%s%d}", cur->lazy ? "~"
                    : "", cur->rep_min);
257             else
258                 snprintf(repeats, sizeof(repeats), "{%s%d-%d}", cur->lazy ? "
                    ~" : "", cur->rep_min, cur->rep_max);
259             switch(cur->type) {
260                 case PLAIN:
261                     printf("\'%s\' ", cur->data.text);
262                     break;
263                 case ONE:
264                     printf("\'%c\'%s ", cur->data.sym, repeats);
265                     break;
266                 case ANY:
267                     printf("ANY%s ", repeats);
268                     break;
269                 case CHARCLASS:
270                     printf("CLASS_%s ", cur->data.sym, repeats);
271                     break;
272                 case GROUP:
273                 case REVGROUP:
274                     printf("'%s[%s]%' ", cur->type == REVGROUP ? "!" : "",
                        cur->data.text, repeats);

```

```

275         break;
276     case CHAIN:
277         printf("CHAIN#%dP/s:\n", cur->data.subchain->ch_data,
                repeats);
278         print_re_blocks(cur->data.subchain, depth + 1);
279         if (cur->next) {
280             putchar('\n');
281             for (int i = 0; i < depth; i++) printf("    ");
282             printf(".. ");
283         }
284         break;
285     case BACKREF:
286         printf("BACKREF#%dP/s \n", cur->data.cap_num, repeats);
287         break;
288     }
289     cur = cur->next;
290 }
291 if (depth == 0 || re->next)
292     putchar('\n');
293 alt++;
294 }
295 }
296
297 // Парсит цепочку списков блоков, для каждой альтерации создаёт для неё звено
    // цепочки,
298 // параллельно проверяя шаблон регулярного выражения на корректность
299 // возвращает число разобранных символов или вызывает "исключение", если
    // шаблон некорректен
300 int _parse_chain(struct re_chain** root, char* re_str, char* insert_seq, int
    cap_status, int* fl, int* ch_cnt, jmp_buf* jenv, char** erp)
301 {
302     struct re_chain* re = re_malloc(sizeof(struct re_chain));
303     memset(re, 0, sizeof(struct re_chain));
304     re->ch_data = cap_status > 0 ? (*root ? (*root)->ch_data : *ch_cnt) : 0;
305     re->next = *root;
306     *root = re;
307
308     int insert_len = 0;
309     int i = 0;
310     while (re_str[i]) {
311         if (re_str[i] == '^') {
312             if (i > 0)
313                 RAISE(BAD_CARET);
314             re->incl_start = true;
315             i++;
316         }
317         else if (re_str[i] == '$') {
318             if (!(re_str[i+1] == '\0' || re_str[i+1] == '|' || re_str[i+1] ==
                ')'))
319                 RAISE(BAD_DOLLAR);
320             re->incl_end = true;
321             i++;
322         }
323         else if (re_str[i] == '.') {

```

```

324         i += 1 + process_quantif(re, re_str+i+1, ANY, (union re_data){ .
            text = NULL }, jenv, erp);
325     }
326     else if (re_str[i] == '[') {
327         i++;
328         enum block_type bl_type = GROUP;
329         if (re_str[i] == '^') {
330             bl_type = REVGROUP;
331             i++;
332         }
333         if (re_str[i] == ']')
334             RAISE(EMPTY_SQUARES);
335
336         int group_start = i;
337         while (re_str[i] != ']') { // подсчёт длины группы в символах для
            // точного выделения памяти
338             if (!re_str[i])
339                 RAISE(MISSING_RSQUARE);
340             if (re_str[i] == ESC_CHAR)
341                 i += 2;
342             else i++;
343         }
344         char* data_str = re_malloc(i - group_start + 1);
345         int data_len = 0;
346         bool next_escaped = false;
347         bool escaped_hyphen = false; // экранированный дефис при наличии
            // помещается единожды в конец строки, и перед ним не может быть
            ESC_CHAR
348         i = group_start;
349         while (re_str[i]) {
350             // Метасимволы в квадратных скобках, помимо трёх ниже, не
            // имеют силы, поэтому воспринимаются буквально (так же, как
            // и в PB в других языках)
351             // Единственные символы, которые есть смысл экранировать:
            // ']', '-', ESC_CHAR
352             if (re_str[i] == ESC_CHAR && !next_escaped) {
353                 next_escaped = true;
354                 if (re_str[i+1] != '-')
355                     data_str[data_len++] = re_str[i];
356                 i++;
357                 if (IS_CHARCLASS(re_str[i]) && re_str[i+1] == '-' &&
                    re_str[i+2] != ']') {
358                     i++; // место ошибки '-'
359                     RAISE(RANGE_WITH_CHARCLASS);
360                 }
361                 continue;
362             }
363
364             if (re_str[i+1] == '-' && re_str[i+2] && re_str[i+2] != ']')
365             {
366                 int dist = re_str[i+2] == ESC_CHAR ? 3 : 2;
367                 if (re_str[i+2] == ESC_CHAR && IS_CHARCLASS(re_str[i +
                    dist])) {
                    i++;

```



```

368         RAISE(RANGE_WITH_CHARCLASS);
369     }
370     if (re_str[i] > re_str[i + dist])
371         RAISE(BAD_SQUARE_ORDER);
372     data_str[data_len++] = '-'; // для удобства в нотации
                                // записи группы сначала символ диапазона '-'
373     data_str[data_len++] = re_str[i]; // а затем его
                                // начальный и конечный символы
374     data_str[data_len++] = re_str[i + dist];
375     i += dist;
376 }
377 else if (re_str[i] == '-') // дефис, не относящийся к
                            // диапазону (воспринимается буквально)
378     escaped_hyphen = true;
379 else if (re_str[i] == ']') && !next_escaped)
380     break;
381 else
382     data_str[data_len++] = re_str[i];
383
384     i++;
385     next_escaped = false;
386 }
387
388 if (escaped_hyphen)
389     data_str[data_len++] = '-';
390 data_str[data_len] = '\0';
391 i += 1 + process_quantif(re, re_str+i+1, bl_type, (union re_data)
    { .text = data_str }, jenv, erp);
392 }
393 else if (re_str[i] == '|') { // начало новой альтернативы (цепочки
    блоков)
394     i++;
395     // разбор нового звена цепочки (списка блоков) с текущей позиции
    // и до конца альтерации/группы
396     i += _parse_chain(root, re_str + i, insert_seq, cap_status, fl,
        ch_cnt, jenv, erp);
397 }
398 else if (re_str[i] == '(') { // начало новой группы, разбор полностью
    осуществляется рекурсивным вызовом
399     i++;
400     bool next_capturable = true;
401     bool first_char = i == 1;
402     if (re_str[i] == '?') {
403         i++;
404         if (re_str[i] == ':') { // не-захватываемая группа
405             next_capturable = false;
406             i++;
407         } else {
408             while (re_str[i] != ')') { // список модификаторов
409                 switch (re_str[i]) {
410                     case 'm':
411                         *fl |= RE_M; break;
412                     case 's':
413                         *fl |= RE_S; break;

```

```

414         case 'i':
415             *fl |= RE_I; break;
416         case 'I':
417             *fl |= RE_IRU; break;
418         case '\0':
419             RAISE(MISSING_RPAREN);
420         default:
421             RAISE(INVALID_FLAG);
422     }
423     i++;
424 }
425 i++; // закрывающая скобка
426 if (first_char && re_str[i] == '^') { // в порядке
427     // исключения рассм. отдельно
428     re->incl_start = true;
429     i++;
430 }
431 continue;
432 }
433 } else
434     (*ch_cnt)++;
435 struct re_chain* new_subchain = NULL;
436 i += _parse_chain(&new_subchain, re_str + i, insert_seq,
437     next_capturable ? 1 : 0, fl, ch_cnt, jenv, erp);
438 i += 1 + process_quantif(re, re_str+i+1, CHAIN, (union re_data){
439     .subchain = new_subchain }, jenv, erp);
440 }
441 else if (re_str[i] == ')') { // окончание разбора группы
442     if (cap_status == -1)
443         RAISE(MISSING_LPAREN);
444     return i;
445 }
446 else { // символы, воспринимаемые буквально
447     if (re_str[i] == ESC_CHAR) {
448         i++;
449         if (IS_CLASS(re_str[i])) {
450             i += 1 + process_quantif(re, re_str+i+1, CHARCLASS, (
451                 union re_data){ .sym = re_str[i] }, jenv, erp);
452             continue;
453         }
454         else if (re_str[i] >= '1' && re_str[i] <= '9') {
455             i += 1 + process_quantif(re, re_str+i+1, BACKREF, (union
456                 re_data){ .cap_num = (re_str[i] - '0') }, jenv, erp);
457             continue;
458         }
459     }
460     #ifdef STRICT_CHARCLASSES
461     else if (isalpha(re_str[i]))
462         RAISE(BAD_CHARCLASS);
463     #endif
464 }
465 else { // эти метасимволы всегда рассматриваются совместно с
466     // другими и в норме не бывают открывающими
467     if (IS_QUANTIF(re_str[i]))
468         RAISE(PREV_NOT_QUANTIF);

```

```

462         if (re_str[i] == ']')
463             RAISE(MISSING_LSQUARE);
464     }
465
466     if (!re_str[i])
467         RAISE(BAD_ESC_CHAR);
468
469     if (!re_str[i+1] || IS_WILDCARD(re_str[i+1]) || (re_str[i+1] ==
ESC_CHAR && (IS_CLASS(re_str[i+2]) || isdigit(re_str[i+2]))))
    {
470         bool next_is_quantif = IS_QUANTIF(re_str[i+1]);
471         if (!next_is_quantif) {
472             insert_seq[insert_len] = re_str[i];
473             insert_len++;
474         }
475         if (insert_len > 0) {
476             char* data_str = re_malloc(insert_len+1);
477             re_strncpy(data_str, insert_seq, insert_len+1); // срез
строки от начала длиной insert_len
478             insert_len = 0;
479             new_block(re, PLAIN, (union re_data){ .text = data_str },
1, 1, false);
480         }
481         if (next_is_quantif) {
482             i += 1 + process_quantif(re, re_str+i+1, ONE, (union
re_data){ .sym = re_str[i] }, jenv, erp);
483         }
484         else i++;
485     }
486     else {
487         insert_seq[insert_len] = re_str[i];
488         i++;
489         insert_len++;
490     }
491 }
492 }
493 if (cap_status >= 0)
494     RAISE(MISSING_RPAREN);
495 return i;
496 }
497
498 void re_str_tolower(char* src, char* dest, int len, int flags) // src и dest
могут указывать на одну строку
499 {
500     for (int i = 0; i < len; i++) {
501         if ((flags & RE_IRU) && src[i] == (char)208 && (src[i+1] == (char)129
|| (src[i+1] >= (char)144 && src[i+1] <= (char)175))) {
502             if (src[i+1] == (char)129) { // буква Ё кодируется по особым
правилам
503                 dest[i] = (char)209;
504                 dest[i+1] = (char)145;
505             } else if (src[i+1] + 32 > (char)191) { // для символов длиной 2
байта возможный диапазон значений второго байта от 128 до 191
506                 dest[i] = (char)209;

```

```

507         dest[i+1] = src[i+1] - 32; // + 32 - 192 + 128
508     } else {
509         dest[i] = (char)208;
510         dest[i+1] = src[i+1] + 32;
511     }
512     i++;
513 } else
514     dest[i] = tolower(src[i]);
515 }
516 dest[len] = '\0';
517 }
518
519 // Приведение одного байта к нижнему регистру с учётом контекста (соседних
    байтов, образующих юникод-символ),
520 // поскольку приведение всей входной строки при каждом сопоставлении очень
    медленное, если строка длинная, а захваты частые
521 char re_char_tolower(char* context_str, int n, int flags)
522 {
523     char ctx[2+1];
524     if (!(flags & RE_IRU && context_str[n] & 0x80)) // ASCII символы
525         return tolower(context_str[n]);
526
527     // continuation byte (10xxxxxx) означает, что str[n] - второй байт
528     // leading byte (11xxxxxx) - значит первый байт
529     int sym_start = ((context_str[n] & 0xC0) == 0x80 && n > 0) ? n - 1 : n;
530     re_strncpy(ctx, context_str + sym_start, sizeof(ctx));
531     re_str_tolower(ctx, ctx, 2, flags);
532     return ctx[n - sym_start];
533 }
534
535 // Приведение регистра букв к нижнему в шаблоне PB во всех блоках, где может
    храниться текст
536 void re_tolower_inplace(struct re_chain* root, int flags)
537 {
538     for (struct re_chain* re = root; re != NULL; re = re->next) {
539         struct re_block* cur = re->head;
540         while (cur) {
541             if (cur->type == PLAIN || cur->type == GROUP || cur->type ==
                REVGROUP)
542                 re_str_tolower(cur->data.text, cur->data.text, strlen(cur->
                    data.text), flags);
543             else if (cur->type == ONE)
544                 cur->data.sym = tolower(cur->data.sym);
545             else if (cur->type == CHAIN)
546                 re_tolower_inplace(cur->data.subchain, flags);
547             cur = cur->next;
548         }
549     }
550 }
551
552 // Вспомогательная функция для вывода человекочитаемого сообщения об ошибке
    разбора
553 char* get_error_msg(enum re_error_type code) {
554     switch (code) {

```

```

555     case (RE_OK):
556         return "OK";
557     case (BAD_CARET):
558         return "Unescaped ^ not in the beginning of regex or group";
559     case (BAD_DOLLAR):
560         return "Unescaped $ not in the end of regex";
561     case (PREV_NOT_QUANTIF):
562         return "Preceding token is not quantifiable";
563     case (EMPTY_SQUARES):
564         return "Empty square brackets";
565     case (MISSING_LSQUARE):
566         return "Missing opening square bracket";
567     case (MISSING_RSQUARE):
568         return "Missing closing square bracket";
569     case (RANGE_WITH_CHARCLASS):
570         return "Range with character class is unacceptable";
571     case (BAD_ESC_CHAR):
572         return "Missing character after escape character";
573     case (BAD_INSIDE_CURLY):
574         return "Invalid characters inside curly brackets";
575     case (BAD_CURLY_ORDER):
576         return "Quantifier range is out of order";
577     case (HUGE_CURLY_RANGE):
578         return "Quantifier range is too large";
579     case (MISSING_LPAREN):
580         return "Missing opening parenthesis";
581     case (MISSING_RPAREN):
582         return "Missing closing parenthesis";
583     case (BAD_SQUARE_ORDER):
584         return "Character range is out of order";
585     case (BAD_CHARCLASS):
586         return "Invalid character class";
587     case (INVALID_FLAG):
588         return "Invalid regex modifier";
589     default:
590         return "Unknown regex pattern error!";
591 }
592 }
593
594 // Экспортируемая функция -
595 // Освобождение памяти, занимаемой объектом регулярного выражения, созданным
596 // функциями compile_re
597 void free_re(struct re_chain* root)
598 {
599     struct re_chain* ch_tmp = NULL;
600     struct re_block* bl_tmp = NULL;
601     struct re_chain* re = root;
602     while (re) {
603         struct re_block* cur = re->head;
604         while (cur) {
605             if (cur->type == PLAIN || cur->type == GROUP || cur->type ==
606                 REVGROUP)
607                 free(cur->data.text);
608             else if (cur->type == CHAIN)

```

```

607         free_re(cur->data.subchain);
608         bl_tmp = cur;
609         cur = cur->next;
610         free(bl_tmp);
611     }
612     ch_tmp = re;
613     re = re->next;
614     free(ch_tmp);
615 }
616 }
617
618 // Экспортируемая функция -
619 // Парсит шаблон регулярного выражения, перед этим проверяя его на
    корректность
620 struct re_chain* compile_re2(char* pattern, int* errcode, int* errpos)
621 {
622     char* insert_seq = re_malloc(strlen(pattern) + 1); // общий временный
        буфер для всех звеньев
623     char** err_pos_ptr = re_malloc(sizeof(char*)); // необходимо выделить на
        куче, чтобы безопасно вернуть сюда значение при longjmp
624     jmp_buf local_env;
625     int ret = setjmp(local_env);
626     if (ret == 0) {
627         struct re_chain* root = NULL; // инициализация цепочки
628         int chain_count = 0;
629         int flags = RE_ROOT;
630         _parse_chain(&root, pattern, insert_seq, -1, &flags, &chain_count, &
            local_env, err_pos_ptr);
631         // парсинг завершился успешно
632         root->ch_data = chain_count;
633         root->flags = flags;
634         free(insert_seq);
635         free(err_pos_ptr);
636         if (flags & (RE_I | RE_IRU)) // флаг может быть не в начале РВ,
            поэтому не получится делать это во время парсинга
637             re_tolower_inplace(root, root->flags);
638         return root;
639     }
640     // возникла ошибка в ходе парсинга с кодом ret
641     else {
642         if (errcode)
643             *errcode = ret;
644         if (errpos)
645             *errpos = *err_pos_ptr - pattern;
646     }
647     free(insert_seq);
648     free(err_pos_ptr);
649     return NULL;
650 }
651
652 struct re_chain* compile_re(char* pattern) {
653     return compile_re2(pattern, NULL, NULL);
654 }

```

Автор ВКР

\_\_\_\_\_  
(подпись, дата)

\_\_\_\_\_  
А. А. Стародубцев

Руководитель ВКР

\_\_\_\_\_  
(подпись, дата)

\_\_\_\_\_  
А. А. Чаплыгин

Нормоконтроль

\_\_\_\_\_  
(подпись, дата)

\_\_\_\_\_  
А. А. Чаплыгин

**Место для диска**